

Autonomous Semantic Drone Road-Following

Progress review of the semantic drone project at Prototype 1- α : subsystems built and verified, the segmentation rewrite, dependency and SITL engineering on the Jetson, the first outdoor autonomous flight (16 July 2026), and the remaining path to a corrected centreline-tracking flight.

Achyut Tiwari

Submitted

to Dr. Anubhav Mishra
Institute Namtech Research Park, IIT Gandhinagar
Repository ACHYUT13807/semantic-drone
Compiled July 2026

PX4 1.17.0 / MAVSDK-Python / ROS 2 Humble / Gazebo SITL / Jetson Orin Nano / Pixhawk 6C Mini / A* + HSV + U-Net

Contents

Autonomous Semantic Drone Road-Following System	1
Prototype 1-α — Progress Review	1
Purpose of this document	2
Structure	3
Executive Summary — Prototype 1-α	4
Part I — The Project at a Glance	6
1. What the System Does and Why	6
2. The End-to-End Pipeline	9
3. System Architecture and Data Flow	11
4. The Hardware and Software Stack	12
Part II — Robotics Software Foundations	14
5. ROS 2 Humble as Used in This Project	14
6. The Codebase, Node by Node	18
7. The asyncio ↔ ROS 2 Integration Problem	21
Part III — The Flight Stack	23
8. PX4 Architecture	23
9. MAVLink and MAVSDK-Python	25
10. Offboard Control in Practice	27
11. PX4 Parameters That Were Changed (and Why)	28
Part IV — Simulation	29
12. Gazebo + PX4 SITL	29
13. The Simulation-to-Reality Gap	30
Part V — Perception	31
14. The Camera Pipeline	31
15. HSV Colour Segmentation	32
16. Semantic Segmentation with U-Net	34
17. TensorRT and Inference on the Jetson	36
18. The Intel RealSense D455	37
19. Texture as a Second Segmentation Feature	39
Part VI — Planning	40
20. Occupancy Grids and Costmaps	40
21. A* Pathfinding	41
22. Automatic Goal Selection	44
23. Waypoint Management	49
24. Continuous Replanning and the Tuning Parameters	50
Part VII — Control	52
25. The Control Node and Loop Closure	52
26. Takeoff, the Flight Sequence, and Safety Integration	53
Part VIII — Hardware Bring-Up	55
27. The Pixhawk 6C Mini	55
28. Firmware: Flashing PX4 1.17.0	56
29. Sensor Calibration	57
30. Flight Modes and the Kill Switch	58
31. Actuator Mapping and ESC Direction (Remaining)	59
32. GPS, Positioning, and the Geofence Prearm (Remaining)	60
33. The Jetson Orin Nano Platform	61

Contents

Part IX — Electrical Integration and Mechanical Design	62
34. Power Architecture	63
35. The Pixhawk ↔ Jetson Serial Link	65
36. ESCs, DShot, and Motor Outputs	67
37. The Airframe Mount Plate	68
38. Bench Validation in QGroundControl	70
Part X — The Segmentation Rewrite	72
39. The Failure: Near-Empty Masks	73
40. Three Stacked Faults	74
41. The Replacement Architecture: MobileNetV2 U-Net	76
42. Gabor Texture Fusion	78
43. Dice + Cross-Entropy Loss, and the Result	80
Part XI — Environment and Dependency Engineering	83
44. The Protobuf Collision	84
45. The Resolution, and Its Fragility	85
46. Package Repositories, setuptools, and Other Environment Debris	87
47. The Environment as a Recorded Artefact	88
Part XII — Distributed SITL and Performance Diagnosis	89
48. The Distributed Topology	89
49. Diagnosing the 4.5 Hz Bottleneck	90
50. Why HITL Is Not Viable on This VM	92
Part XIII — Safety Architecture	93
51. The Pilot Gate: <code>_wait_for_pilot()</code>	93
52. The Layered Safety Model	95
Part XIV — Original Contribution	97
53. The Road-Occlusion Recovery Theory	97
Part XV — What Is Further Required	101
54. Immediate Hardware Blockers (on the critical path)	102
55. Immediate Software Tasks (near the critical path)	103
56. Perception Upgrades (parallel track)	104
57. Original Research: Occlusion Recovery	105
58. The Roadmap as a Dependency Graph	106
Appendix A — Glossary	107
Appendix B — Command Reference	110
B.1 ROS 2 (introspection and running)	110
B.2 The three diagnostics this project relies on	110
B.3 MAVSDK-Python (essential calls)	110
B.4 QGroundControl (GUI, not CLI) — what it is used for	110
B.5 Simulation	111
B.6 Jetson / build sanity	111
Appendix C — Parameter and Threshold Reference	112
C.1 PX4 behavioural parameters	112
C.2 Calibrations completed (stored as parameters)	112
C.3 Firmware / platform versions	112
C.4 HSV segmentation thresholds	112
C.5 Planner tuning parameters (behavioural)	113
C.6 Connection addresses	113

Contents

Appendix D — Troubleshooting Log (Bugs Found and Fixed)	114
D.1 core package imports fail under ros2 run	114
D.2 No images reaching the segmenter	114
D.3 HSV segmentation "stops working" after a camera change	114
D.4 asyncio ↔ ROS "everything just stops"	114
D.5 gRPC / _pose_stream crash	114
D.6 A* returns "no path"	115
D.7 Drone stuck on a waypoint / doesn't advance	115
D.8 Goal-on-obstacle after finding the auto-goal (Jetson bring-up)	115
D.9 Incorrect occupancy grid logic	115
D.10 MAVSDK connect() hangs on the bench	115
D.11 RealSense streams degraded	116
D.12 RealSense driver won't install (GPG key)	116
D.13 "Fence requires position" prearm blocks arming	116
D.14 Compass inconsistency, 128° between internal and external	116
D.15 "Vehicle stuck in Position mode; won't arm"	116
D.16 U-Net OOM: 1.04 GiB allocation fails on the Jetson	117
D.17 Keras 3 .keras file will not load under Keras 2.15	117
D.18 protobuf collision on the Jetson	117
D.19 4.5 Hz camera on distributed SITL	117
D.20 Duplicate ROS apt repository	118
D.21 setuptools too new for ROS 2 Humble	118
D.22 Gazebo gz.transport / gz.msgs import error	118
D.23 Empty road masks — three stacked faults	118
D.24 A* goal on the road edge — drone hugs the shoulder	119
Appendix E — Source Register	120
E.1 Datasets	120
E.2 Foundational papers and methods	120
E.3 Frameworks, middleware, and toolchains	121
E.4 Hardware and its documentation	121
E.5 The project's own artefacts	122
E.6 Intellectual sources of the original contribution	122
E.7 Acknowledgements	122
Appendix F — Development Chronology	123
F.1 Architecture and simulation	123
F.2 The bug sequence	123
F.3 The faculty presentation	123
F.4 Flight-controller bring-up	123
F.5 Jetson integration	124
F.6 Distributed SITL and the performance hunt	124
F.7 The segmentation rewrite	124
F.8 Centreline-guided planner (v7)	124
F.9 Mechanical design	125
F.10 Where that leaves Prototype 1-α	125
Appendix G — List of Figures	126
Appendix H — Verification Evidence Register	127
H.1 Flight controller and firmware	127
H.2 Bench-test findings (QGC session)	127
H.3 Companion computer ↔ flight controller	127
H.4 Perception — simulation	128
H.5 Perception — replacement network	128

Contents

H.5a Centreline-guided planner (v7)	128
H.6 Camera	129
H.7 Environment and dependencies	129
H.8 Distributed SITL	129
H.9 Safety	130
H.10 Items not yet verified (in-scope for next review)	130
H.11 Interpretation	130
Appendix I — Code Excerpts from v7	132
I.1 The safety wrapper: core/px4.py	132
I.2 The pilot handoff gate: nodes/control_node.py	133
I.3 Skeleton-based waypoint chain: core/skeleton_goal.py	134
I.4 Pure Pursuit and pose streaming: nodes/control_node.py (continued)	137
I.5 Segmentation deployment: core/segmentation.py	138
I.6 The segmentation architecture: core/mobilenet_unet.py	139
I.7 Costmap fixes: core/depth_costmap.py (selected functions)	140
I.8 The launch file: launch/drone.launch.py (selected)	141
Appendix J — Deployment and Reproducibility Guide	143
J.1 Base operating system	143
J.2 System package baseline	143
J.3 ROS 2 Humble	143
J.4 The pinned Python environment	144
J.5 Gazebo (workstation side, not Jetson)	144
J.6 Repository build	144
J.7 RealSense D455	145
J.8 Trained model weights	145
J.9 First run — the smoke test	145
J.10 What is deliberately not in this guide	145
Appendix K — Parameter Derivations and Design Rationale	146
K.1 Perception thresholds	146
K.2 Planner and continuous-replanning parameters	146
K.3 Segmentation-training parameters	147
K.4 Safety and PX4 parameters	147
K.5 v7 chain, planner, and controller parameters	147
K.6 Mechanical parameters	148
K.7 The general rule	148
Appendix L — Pre-flight Checklist and Test Procedures	149
L.1 Pre-first-power checks (with battery disconnected)	149
L.2 First powered bench test (props off)	149
L.3 Pipeline dry-run (no propellers, no takeoff)	150
L.4 Outdoor prearm verification (no motion, no propellers)	150
L.5 First propellers-on test (Stabilized, altitude only, tethered if possible)	150
L.6 First positioned flight (Altitude → Position, outdoors)	150
L.7 First Offboard test (companion in the loop)	151
L.8 First autonomous road-following flight	151
L.9 Post-flight	151
L.10 Standing rules	152

Autonomous Semantic Drone Road-Following System

Prototype 1- α — Progress Review

Author. Achyut Tiwari (B. Tech. Computer Engineering, 2nd year, NIAMT Ranchi) **Submitted to.** Dr. Anubhav Mishra, Namtech Research Park, IIT Gandhinagar **Repository.** [ACHYUT13807/semantic-drone](#) (private) **Reporting period.** Internship to date; state as of Prototype 1- α .

Purpose of this document

This is a technical progress report on the semantic drone project as it stands at **Prototype 1- α** — the configuration in which every subsystem exists in a working form, on the target hardware, with a verified path from camera photons to motor outputs, and no critical unknowns remaining. The report exists to give Dr. Mishra a complete account of what has been built, what has been verified against hardware, what remains, and — where relevant — why particular design choices were made or reversed. It also serves as the durable record needed to reproduce the working configuration on a fresh Jetson.

Structure

The document is organised into sixteen parts followed by twelve appendices (A–L). Parts I–XV cover the system as of Prototype 1- α prior to outdoor flight. Part XVI records post- α improvements, the first outdoor autonomous flight of 16 July 2026, the oscillation diagnosis, and the corrective patches that followed. Parts I–VIII cover the system as originally designed and demonstrated end-to-end in simulation on the cobot desktop. Parts IX–XIII cover the more recent work: the electrical and mechanical integration on the aircraft, the segmentation rewrite that superseded the original U-Net, the dependency-engineering work that let TensorFlow and MAVSDK coexist on the Jetson, the distributed-SITL performance diagnosis, and the safety architecture that gates the autonomy behind the pilot. Part XIV records the original road-occlusion recovery theory developed with Dr. Mishra. Part XV enumerates the remaining tasks with dependencies.

The appendices are: glossary, command reference, parameter table, troubleshooting log of every bug found and fixed, source register (Appendix E), development chronology, list of figures, verification evidence register (Appendix H), annotated code excerpts (Appendix I), a deployment and reproducibility guide for provisioning a fresh Jetson (Appendix J), a parameter-derivation appendix (Appendix K), and a pre-flight checklist and ordered test-flight procedure (Appendix L). Pending items are flagged (Remaining) in the running text and collected in Part XV; several were resolved by the first outdoor flight recorded in Part XVI.

Executive Summary — Prototype 1-α



Figure ES.1. The Prototype 1-α airframe: quad-X carbon frame, Pixhawk 6C Mini + external GPS/compass mast, 6S 12000 mAh pack, Simplify

Status at a glance

Prototype 1-α is the first configuration of the system in which every major subsystem exists in a working form, on the hardware it is intended to fly on, with a verified path from camera photons to motor outputs. The software stack runs end-to-end, the flight controller is fully brought up and bench-verified, the companion computer talks to it, and the perception network has been rebuilt to a usable standard. On 16 July 2026 the aircraft performed its first outdoor autonomous Offboard flight (Part XVI §63). That flight exercised the full pipeline under real GPS lock and produced a clear, log-diagnosed oscillation whose root cause (180° camera rotation) and corrective patches are recorded in §64. What remains is a short list of physical and software tasks required for a corrected, centreline-tracking flight, not unsolved architectural problems.

What is verified

Subsystem	State	Evidence
Flight controller	Brought up	Reflashed ArduPilot 4.6.3 → PX4 1.17.0; accel, gyro, compass, RC calibrated
Flight modes / kill switch	Assigned	RC switch mapping; kill switch as hardware backstop
Motor outputs	Verified	Individual motors driven from the QGC Actuators panel, props off
Companion ↔ flight controller	Verified	MAVSDK 2.8.4 connects to the real Pixhawk over <code>serial:///dev/ttyACM0:57600</code>
Perception (sim)	Working	HSV segmentation; road-following confirmed end-to-end on <code>cobot</code>
Perception	Rebuilt	MobileNetV2 U-Net + Gabor fusion, Dice+CE loss, <code>val_road_iou = 0.80</code>

(real)		
Camera (real)	Live	RealSense D455 publishing on the Jetson; real intrinsics measured
Planning	Working	A* over the semantic costmap, publishing waypoints against live frames

What remains

Several items that were open at the start of Prototype 1- α have been cleared by the 16 July 2026 flight (Part XVI §63); a short residual list remains.

1. RealSense on a true USB 3.x port, to lift the residual USB 2.1 bandwidth cap (Ch. 18).
2. Mount-plate geometry conflict between the 3×3 cm side aperture and the Jetson pocket — a CAD revision (Ch. 37).
3. Domain fine-tuning / validation of the rebuilt network on real D455 frames after the 180° camera-rotation fix (Ch. 43; Part XVI).
4. Flight-configuration serial link (Pixhawk TELEM2 → Jetson UART1) end-to-end verification (Ch. 35).
5. Post-fix centreline-tracking outdoor flight confirming the §64 patches (camera rotation, Pure Pursuit, deceleration).

2. **Outdoor GPS lock**, to clear the `Fence requires position` prearm and permit arming. (Ch. 32)
3. **RealSense on a true USB 3.x port**, to lift the USB 2.1 bandwidth cap that currently forces 640×480 @ 15 fps. (Ch. 18)
4. **Mount-plate geometry conflict** between the 3×3 cm side aperture and the Jetson pocket — a CAD revision, not a redesign. (Ch. 37)
5. Domain fine-tuning of the rebuilt network on real D455 frames. (Ch. 43; Part XV)

Principal results

Three pieces of diagnostic work account for most of the project's forward motion since the last review:

- **Segmentation rewrite** (Part X). The plain U-Net's near-empty masks were traced to three independent stacked bugs; the replacement network (MobileNetV2 encoder + Gabor texture fusion + Dice/CE loss) reaches `val_road_iou = 0.80`.
- **Centreline-guided planner** (§22.6–22.12). Edge-hugging behaviour of the original A*-based goal selection was diagnosed and replaced with a v7 pipeline that skeletonises the road mask (Zhang–Suen or `cv2.ximgproc.thinning`), prunes to the longest branch via double-BFS, and feeds A* an ordered chain of 12 forward waypoints advanced one at a time. Two costmap bugs that would have defeated the chain were also fixed: `largest_road_component` restricts goal-picking to the main connected road, and road-protected inflation stops obstacle dilation from pinching narrow corridors shut.
- **Distributed SITL diagnosis** (Part XII). A 4.5 Hz pipeline was localised past the bridge, network, USB, and protobuf to Gazebo software-rendering at ~20% real time.
- **Protobuf collision** (Part XI). TensorFlow 2.15 and MAVSDK's mutually exclusive protobuf requirements were resolved by downgrading MAVSDK to 2.8.4 and pinning `protobuf == 3.20.3`, verified against the real Pixhawk.

Sources are recorded in Appendix E; chronology in Appendix F.

Part I — The Project at a Glance

1. What the System Does and Why



Figure 1.1. The Prototype 1- α hardware stack at the bench: Jetson Orin Nano developer kit (bottom-left, the companion computer), Pixhawk 6C Mini flight controller (bottom-centre, blue-mounted), and Intel RealSense D455 depth camera on a tripod (top-right). The printed sheet in the background is the pre-flight checklist of Appendix L. The two-computer split — flight-critical control on the Pixhawk, perception and planning on the Jetson — is the architectural centrepiece of this project (Chapter 1).

1.1 The one-sentence description

The project is an aircraft that can look straight down at a road from the air, decide in real time where the drivable surface goes, and fly itself along that surface without any pre-loaded map or GPS waypoint list — using only what its downward-facing camera sees, frame by frame.

That sentence hides a surprising amount of engineering. To make it work you have to solve, simultaneously, a **perception** problem (where is the road in this image?), a **planning** problem (given where the road is, where should I go next?), and a **control** problem (given where I want to go, how do I command the motors to get there safely?). Each of those is a research field in its own right. The value of this project is that it stitches all three into a single closed loop running on a small computer bolted to a quadroter.

1.2 Why "semantic" is the important word

A lot of drones already follow roads — but most of them cheat. They either:

1. Follow a **pre-recorded GPS track**, so they are not really seeing the road at all, just replaying coordinates; or
2. Follow a **coloured line or marker** laid down specifically for them, which is line-following, not road-following; or
3. Use a **remote pilot** who is doing all the perception with their own eyes.

"Semantic" means the drone builds an *understanding of the scene* — it classifies pixels into meaningful categories ("this region is road, that region is not") rather than tracking a colour blob or a GPS breadcrumb. Semantic segmentation is the computer-vision task of assigning a class label to *every pixel* in an image. When we say the drone is semantic, we mean its

notion of "where can I fly" comes from per-pixel scene understanding, which is far more general: it can in principle follow an unmarked rural road it has never seen, in changing light, around curves, because it recognises *road-ness* rather than a specific track.

1.3 The concrete objective

Stated operationally, the system must:

- Take a continuous video stream from a **downward-facing (nadir) camera**.
- For each frame, produce a **binary mask**: road vs. not-road.
- Convert that mask into an **occupancy grid** the planner can reason over (free space = road, occupied = everything else).
- Run a **path-planning search** (A*) over that grid to pick the next stretch of road to fly toward.
- Emit a sequence of **waypoints** describing that path.
- Feed those waypoints to a **flight controller** that drives the motors to follow them, while respecting safety limits (altitude bounds, geofence, kill switch).
- Do all of the above **onboard and in real time**, closing the loop many times a second so that as the drone moves and the view changes, the plan is continuously refreshed.

This is a genuinely autonomous perception-to-actuation loop. Nothing about the road is known in advance.

1.4 The two-computer split

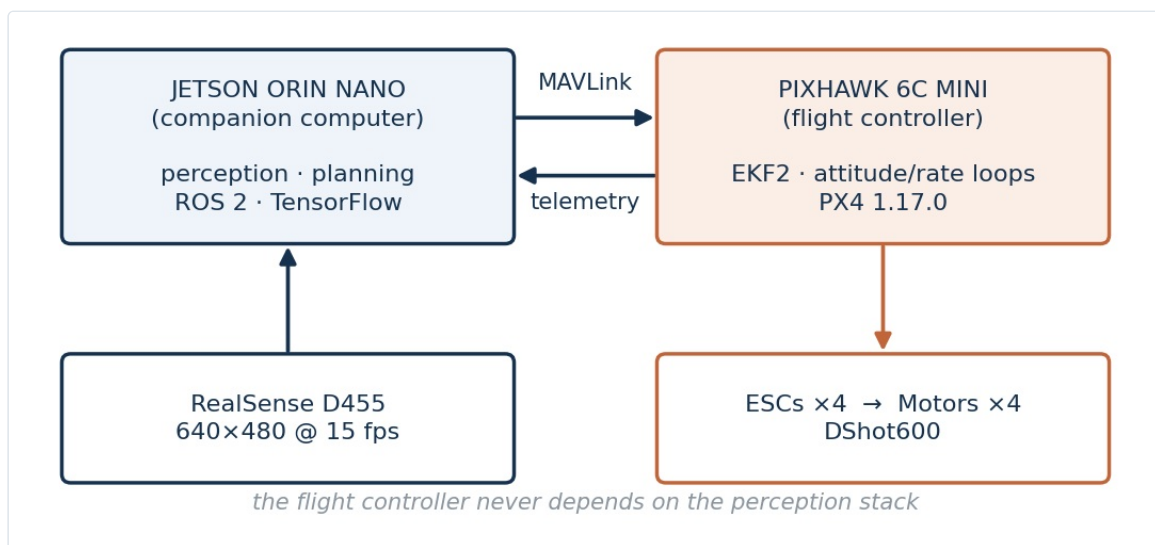


Figure 1. The two-computer split. The Jetson runs perception and planning; the Pixhawk owns attitude, rate control, and every safety decision. Nothing flight-critical depends on the perception stack.

A single fact shapes the entire architecture: the work is divided across **two computers** that talk to each other.

- The **companion computer** — a **Jetson Orin Nano 8GB** — does the heavy thinking. It runs the camera driver, the segmentation, the planner, and the high-level control logic. This is where all the ROS 2 code and all the machine learning live. It has a GPU, which is why the vision workload lives here.
- The **flight controller** — a **Pixhawk 6C Mini** running **PX4** — does the fast, safety-critical, low-level work: reading the IMU hundreds of times a second, running the attitude and position control loops, driving the ESCs, handling arming and failsafes. This is a real-time embedded system; it is *not* where you run a neural network.

The two are connected by a serial/USB link over which they speak **MAVLink** (via MAVSDK on the Jetson side). The Jetson says "go to this position" or "take off to this altitude"; the Pixhawk figures out the exact motor commands to make that happen and refuses anything unsafe. Keeping the split clean — smart-but-slow on the Jetson, dumb-but-fast-and-safe on

the Pixhawk — is the single most important architectural principle in the whole system, and most of the debugging in this project comes down to getting the handoff between these two right.

1.5 Where the project sits right now

At the time of writing, the **simulation pipeline is validated end-to-end**: in Gazebo with PX4 SITL, the drone segments a simulated road, plans over it, and flies the path. The project has crossed the hard bridge from "works in sim" to "bringing up real hardware." The Pixhawk 6C Mini is flashed and calibrated; the Jetson is running with the real RealSense camera feeding the planner against live sensor data. What is **not** yet done is the final hardware wiring (motors/ESCs) and outdoor positioning (GPS lock), which are the last blockers before a real flight. Part X lays this out precisely; every chapter also ends with a status note so the boundary is always visible.

2. The End-to-End Pipeline

2.1 The loop, in words

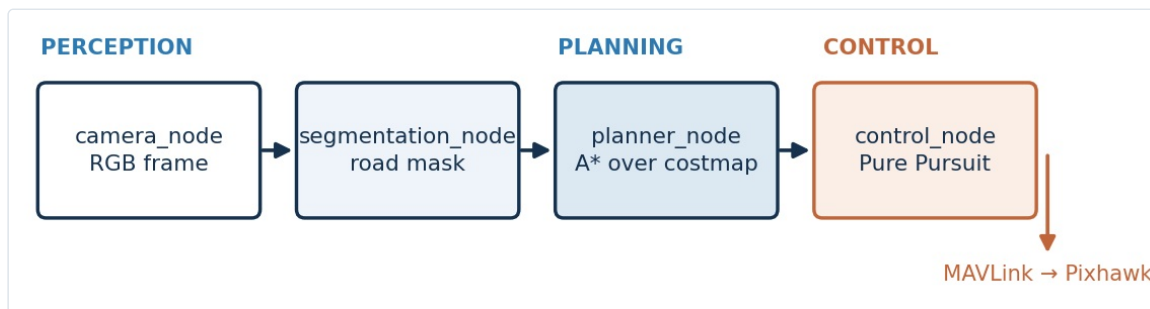


Figure 2. The perception → planning → control loop, one ROS 2 node per stage.

Think of the system as a conveyor belt that runs continuously. One trip along the belt looks like this:

1. **Capture.** The downward camera produces a frame — a grid of pixels, each with a colour.
2. **Segment.** The frame goes into a segmentation stage that outputs a mask the same size as the image, where each pixel is marked road (1) or not-road (0).
3. **Project to a grid.** The mask is turned into an occupancy grid: a 2-D array the planner treats as a little map, with road cells "free" and everything else "occupied."
4. **Choose a goal.** Because there is no external destination, the system picks its own goal automatically — a point far along the visible road in the direction of travel. This is the `_find_auto_goal` logic.
5. **Plan.** A* searches the grid from the drone's current position to that goal, returning the shortest free-space path.
6. **Emit waypoints.** The path is converted into a handful of waypoints in the world frame.
7. **Follow.** The control stage feeds waypoints to PX4 via MAVSDK; PX4 flies to them.
8. **Repeat.** As the drone moves, the camera sees a new stretch of road, and the whole belt runs again — continuous replanning.

Every stage is a ROS 2 node (Part II). The "conveyor belt" is the ROS 2 message bus: nodes publish their outputs on topics and subscribe to the inputs they need, so the stages are decoupled and run concurrently rather than strictly one-after-another.

2.2 The loop, as data types

It helps to track exactly what type of thing flows between stages, because most integration bugs are type/frame mismatches:

Stage	Input	Output	ROS message type (typical)
Camera	raw sensor	colour image	<code>sensor_msgs/Image</code>
Segmentation	colour image	binary mask image	<code>sensor_msgs/Image</code> (mono8)
Planner	mask + current pose	path / next waypoint	<code>nav_msgs/Path</code> or a point
Control	waypoint + telemetry	MAVLink setpoints	(MAVSDK offboard call)
Pose feedback	PX4 telemetry	current position	<code>geometry_msgs/PointStamped</code> on <code>/mavsdkt/pose</code>

The pose feedback arrow is what closes the loop: the planner cannot plan from "where am I" unless something is continuously telling it where the drone is. In this project that "something" is a **MAVSDK telemetry stream** that the control node subscribes to and republishes on `/mavsdkt/pose` — a design choice explained in detail in Chapters 6 and 25. Earlier the planner used simulation odometry (`/odom`); switching it to consume `/mavsdkt/pose` was a deliberate move to make the same code path work on real hardware, where there is no `/odom`.

2.3 Why decoupling the stages matters

If the pipeline were a single function that captured, segmented, planned, and commanded in strict sequence, it would run only as fast as its slowest stage, and any hiccup (a slow segmentation frame, a dropped telemetry packet) would stall the whole thing. By making each stage an independent node that publishes and subscribes, you get:

- **Concurrency.** The camera keeps capturing while the planner is still chewing on the previous mask.
 - **Rate independence.** Segmentation can run at 10 Hz while control commands go out at a different rate; each consumes the latest available message from the other.
 - **Isolation for debugging.** You can record and replay any topic, run one node in isolation, or swap the HSV segmenter for the U-Net without touching the planner. This modularity is precisely why so much of the project's progress has been "fix one node at a time."
-

3. System Architecture and Data Flow

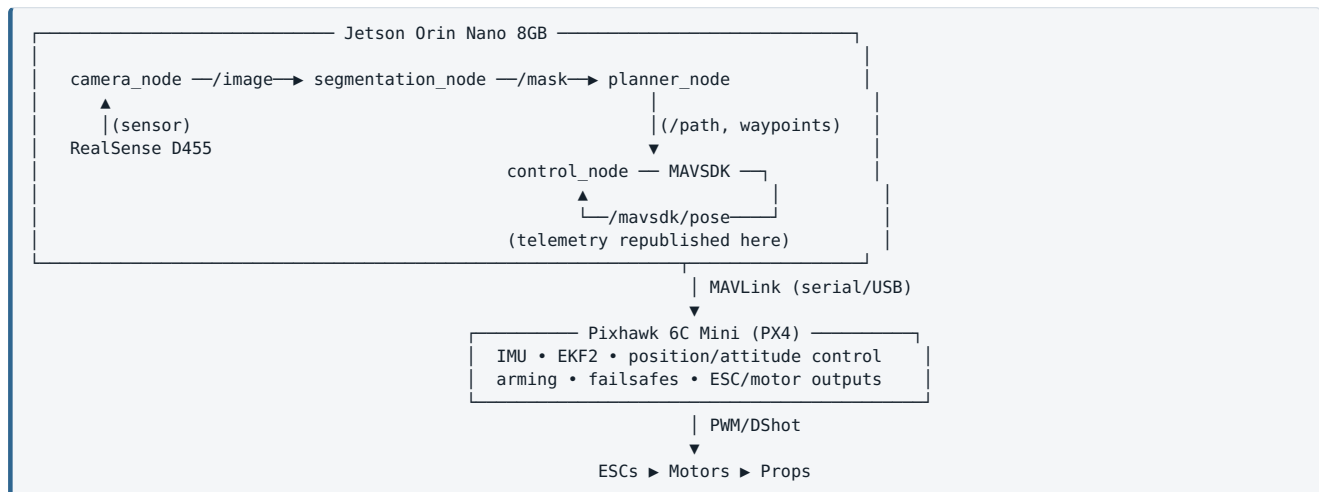
3.1 The three planes

A clean way to hold the whole system in your head is to separate three "planes":

- **The perception plane** turns pixels into a notion of free space. Nodes: camera, segmentation, and (in sim) the Gazebo camera bridge. Output: an occupancy grid / mask.
- **The decision plane** turns free space into intent. Node: planner. It picks a goal and computes a path. Output: waypoints.
- **The actuation plane** turns intent into motion. Node: control (on the Jetson) plus PX4 (on the Pixhawk). Output: physical movement, and a telemetry stream back up.

Data flows perception → decision → actuation, and a thin feedback wire (telemetry/pose) runs back from actuation to decision so the planner always knows where it is. This is a textbook **sense-plan-act** architecture, which is the dominant paradigm for this class of autonomous robot.

3.2 Where each piece physically runs



The dashed feedback path (`/mavsdk/pose`) is the loop-closing wire. Everything above the MAVLink line is ROS 2 + Python + ML on Linux; everything below it is hard real-time embedded firmware.

3.3 The frames of reference problem

A subtle but constant source of bugs in any drone project is **coordinate frames**. There are at least four you have to keep straight:

- **Image frame.** Pixel coordinates (u, v), origin top-left, u right, v down. The segmentation mask lives here.
- **Grid frame.** The occupancy grid the planner searches — array indices (row, col). Usually a direct reindexing of the image, but you must know the mapping.
- **Body frame.** Attached to the drone: x forward, y right, z down (this is the aerospace **FRD** convention PX4 uses onboard).
- **World / local frame.** A fixed frame the drone moves within. PX4 and MAVSDK commonly expose position in **NED** (North-East-Down) or a local ENU/NED depending on the interface.

Whenever the planner produces a path in image/grid coordinates and the control node has to turn it into a position setpoint, there is a transform between grid → body → world. Getting a sign wrong here sends the drone the opposite direction. A number of the tuning parameters (Chapter 24) exist partly to be robust to small frame/latency errors. The lesson learned repeatedly: **always know which frame a coordinate is in before you do arithmetic on it.**

4. The Hardware and Software Stack

4.1 The stack inventory

This is the complete list of technologies the project stands on, with a one-line statement of what each is *for*. Every one of these is expanded into its own chapter later; this table is the map.

Layer	Technology	Role in this project
Companion OS	Ubuntu 22.04.5 LTS (on Jetson, via JetPack 6.x / L4T R36.3.0)	The Linux the ROS code runs on
Middleware	ROS 2 Humble	Message bus + node framework tying the pipeline together
Build system	colcon + ament	Compiles/links the ROS 2 workspace
Language	Python 3 (+ some C++ under ROS)	Everything in the pipeline is Python
Vision (classical)	OpenCV, HSV colour segmentation	Fast road segmentation used in sim and bring-up
Vision (learned)	U-Net (TensorFlow, AerialScapes dataset)	Semantic road segmentation, trained but deferred
Inference acceleration	TensorRT	Converting the network to run fast on the Jetson GPU
Depth sensor	Intel RealSense D455	Real downward camera + depth on hardware
Planning	A* over an occupancy grid	Road-following path search
Flight comms	MAVLink via MAVSDK-Python	Jetson ↔ Pixhawk command/telemetry link
Flight stack	PX4 1.17.0	Real-time flight control firmware
Flight controller HW	Pixhawk 6C Mini	The board PX4 runs on
Simulator	Gazebo + PX4 SITL	Software-in-the-loop testing before hardware
Ground station	QGroundControl	Flashing, calibration, parameter tuning, monitoring
Compute HW	NVIDIA Jetson Orin Nano 8GB	Onboard GPU computer for the whole pipeline

4.2 Why each major choice was made

Why ROS 2 (not ROS 1, not a bespoke framework). ROS 2 gives node isolation, a typed message bus, quality-of-service controls, and an enormous ecosystem of drivers (including the RealSense driver). ROS 2 Humble specifically because it is the LTS that matches Ubuntu 22.04, which is what JetPack 6 ships. Building your own message bus would mean reinventing publish/subscribe, serialization, discovery, and QoS — all of which ROS gives for free.

Why PX4 (not ArduPilot, not custom firmware). PX4 is a mature, open, real-time flight stack with first-class **offboard control** and a clean **MAVSDK** API for exactly this "companion computer sends setpoints" use case. It handles the parts you must not get wrong — attitude stabilisation, arming logic, failsafes — so the project can focus on perception and planning. Writing flight-control firmware from scratch is a multi-year effort and a safety liability.

Why MAVSDK-Python (not raw MAVLink, not MAVROS). MAVSDK wraps the MAVLink protocol in a clean, async Python API: `drone.action.arm()`, `drone.offboard.set_position_ned(...)`, `drone.telemetry.position()`. It hides the byte-level protocol and gives you Python coroutines. The trade-off — that it is async and must be reconciled with ROS's own executor — is the subject of Chapter 7, and one of the more important lessons of the project.

Why HSV first, U-Net later. HSV colour thresholding is cheap, deterministic, needs no GPU, and is trivially debuggable — perfect for getting the *rest* of the pipeline (planning, control, hardware) working without waiting on a network. The U-Net is the principled long-term segmenter but suffers a **domain gap** (trained on the AerialScapes dataset, which does not match the exact look of the sim/real camera), and its latency needs TensorRT. So the pragmatic path is: prove the loop with HSV, deploy U-Net once the hardware loop is solid. This "make it work simply, then make it smart" ordering is a theme.

Why the Jetson Orin Nano. It has a CUDA-capable GPU in a form factor and power budget that can fly, and it runs full Ubuntu + ROS 2. That combination — GPU for the network, real Linux for the ecosystem, small/light enough to be airborne — is why it is the companion computer. The 8GB memory bounds how big a model can be loaded, which feeds back into the TensorRT/quantisation decisions.

Why the RealSense D455. It provides both an RGB image and a depth stream in one USB device with a good ROS 2 driver. Even though the current pipeline segments on colour, having depth available opens the door to obstacle awareness and altitude sanity-checking later. Its practical catch — needing genuine USB 3.x bandwidth or it silently caps at USB 2.1 rates — is a real hardware lesson (Chapter 18).

4.3 How the pieces fit as a dependency stack

Reading bottom-to-top, each layer depends on the ones below it:

Autonomous road-following behaviour	← the goal
A* planner + auto-goal + replanning	← decision logic
HSV / U-Net segmentation (OpenCV / TF / TRT)	← perception
ROS 2 Humble nodes, topics, QoS	← middleware glue
MAVSDK-Python ↔ MAVLink ↔ PX4 1.17.0	← flight interface + stack
Jetson Orin Nano (Ubuntu/JetPack) Pixhawk 6C	← hardware

A failure at any layer breaks everything above it, which is exactly why bring-up proceeds bottom-up: get the hardware and PX4 solid, then the MAVSDK link, then ROS, then perception, then planning, then behaviour. The project has climbed most of this stack in simulation and is now re-climbing the bottom layers on real hardware.

Part II — Robotics Software Foundations

5. ROS 2 Humble as Used in This Project

5.1 What ROS actually is (and is not)

ROS ("Robot Operating System") is a misnomer — it is not an operating system. It is a **middleware**: a set of libraries and conventions that let independent programs on one or more machines discover each other and exchange typed messages, plus tools to build, launch, record, and introspect those programs. You still run it on top of a real OS (Ubuntu here).

The mental model that matters: a ROS system is a **graph** of **nodes** (processes) connected by **topics** (named, typed message channels). A node that produces data **publishes** it on a topic; a node that needs that data **subscribes** to the topic. Neither node knows or cares who is on the other end — they are decoupled by the topic name and message type. This is the publish/subscribe pattern, and it is the backbone of the whole drone pipeline.

ROS 2 (as opposed to ROS 1) rebuilt this on top of **DDS** (Data Distribution Service), an industrial pub/sub standard, which gives it decentralised discovery (no central master to crash), real-time-friendly transport, and **Quality of Service** knobs. Humble is the 2022 LTS release, chosen because it targets Ubuntu 22.04 — the exact userland JetPack 6 provides on the Orin Nano.

5.2 Nodes

A **node** is a single participant in the graph — in practice, a Python class extending `rclpy.node.Node`. Each pipeline stage in this project is a node:

- `camera_node` — publishes images
- `segmentation_node` — subscribes to images, publishes masks
- `planner_node` — subscribes to masks + pose, publishes paths/waypoints
- `control_node` — subscribes to waypoints, talks to PX4, publishes pose
- plus bridge/helper nodes in simulation

Minimal node skeleton, which every one of the above follows:

```
import rclpy
from rclpy.node import Node

class CameraNode(Node):
    def __init__(self):
        super().__init__('camera_node')          # the node's graph name
        # create publishers, subscribers, timers here

def main():
    rclpy.init()
    node = CameraNode()
    rclpy.spin(node)          # hand control to ROS: process callbacks forever
    node.destroy_node()
    rclpy.shutdown()
```

`rclpy.spin(node)` is the key call: it blocks and repeatedly services the node's callbacks (incoming messages, timers). Understanding what `spin` does — and that it wants to own the thread — is central to the async problem in Chapter 7.

5.3 Topics, publishers, subscribers

A **topic** is just a string name (e.g. `/image`, `/mask`, `/mavsdm/pose`) with an associated **message type**. Publishing:

```
self.pub = self.create_publisher(Image, '/image', qos_profile)
# ...later, in a timer or callback:
self.pub.publish(msg)
```

Subscribing:

```
self.sub = self.create_subscription(
    Image, '/image', self.on_image, qos_profile)

def on_image(self, msg: Image):
    # called automatically whenever a new Image arrives on /image
    ...
```

The subscription registers a **callback**. You never call `on_image` yourself; ROS calls it when a message arrives *and* the executor gets a chance to run it. Messages arrive asynchronously; callbacks run when `spin` services them.

5.4 Messages and interfaces

Every topic carries a fixed **message type** — a struct-like definition of fields. The project mostly uses standard interface packages:

- `sensor_msgs/Image` — an image: height, width, encoding (e.g. `rgb8`, `mono8`), and the raw pixel bytes.
- `geometry_msgs/PointStamped` — a 3-D point (x, y, z) plus a header (timestamp + frame id). Used for `/mavsdk/pose`.
- `nav_msgs/Path` — an ordered list of stamped poses; the natural type for a planned path.
- `std_msgs/Header` — the timestamp + `frame_id` that stamps most messages, crucial for knowing *when* and *in which frame* data was produced.

Because images are raw byte arrays in ROS but OpenCV wants NumPy arrays, a bridge is needed — see `cv_bridge` in 5.9.

5.5 Quality of Service (QoS) — and why it bit this project

DDS lets each publisher and subscriber declare a **QoS profile**, and — critically — a publisher and subscriber only connect if their QoS is **compatible**. The knobs that matter here:

- **Reliability**: `RELIABLE` (retransmit until delivered, like TCP) vs `BEST_EFFORT` (fire and forget, like UDP).
- **Durability**: `VOLATILE` (only future messages) vs `TRANSIENT_LOCAL` (late subscribers get the last message).
- **History / depth**: how many messages to buffer.

The trap: **sensor data** in ROS is conventionally `BEST_EFFORT` (you want the newest frame, not a guaranteed-but-stale one), while control/command data is usually `RELIABLE`. If a publisher is `BEST_EFFORT` and a subscriber demands `RELIABLE`, **they silently do not connect** — no error, just no messages. In this project the fix logged in the camera node was to add an explicit `RELIABLE` QoS so that the image producer and consumer matched, and the segmentation side likewise pinned reliable delivery. This is exactly the kind of failure that looks like "my callback never fires" and is actually a QoS mismatch.

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy

qos = QoSProfile(depth=1)
qos.reliability = ReliabilityPolicy.RELIABLE
qos.history = HistoryPolicy.KEEP_LAST
```

5.6 Executors and callbacks

When a message arrives, its callback does not run instantly — it is queued and run by an **executor** when it gets a turn. `rclpy.spin(node)` uses a **single-threaded executor** by default: callbacks run one at a time, in the same thread, never overlapping. This is simple and safe (no data races between callbacks) but means a **slow callback blocks all others**. If segmentation takes 300 ms in a callback, no other callback on that node runs during those 300 ms.

There is also a **multi-threaded executor** and the notion of **callback groups** (reentrant vs mutually-exclusive) for when you need concurrency. The relevance to this project: the interaction between "ROS wants to own the thread and run callbacks" and "MAVSDK wants to own an asyncio event loop and run coroutines" is precisely the executor-conflict problem of Chapter 7.

5.7 Timers

A node often needs to *do something at a fixed rate* rather than only reacting to messages — e.g. publish a camera frame at 30 Hz, or push a control setpoint at a steady cadence. That is a **timer**:

```
self.timer = self.create_timer(0.1, self.tick) # tick() every 100 ms

def tick(self):
    ...
```

The timer callback is serviced by the same executor as message callbacks, so the same "one at a time" rule applies. Setpoint-style logic and periodic diagnostics (the "log road% every 30 frames" style counters) hang off timers or per-frame counters.

5.8 Workspaces, packages, colcon, and `setup.py`

ROS code lives in a **workspace** — a directory with a `src/` folder containing one or more **packages**. A package is the unit of build and distribution; `semantic_drone` is a package. You build with **colcon**:

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash # makes the built package importable/runnable
```

For a Python package, the `setup.py` (and `setup.cfg` / `package.xml`) declares metadata, dependencies, entry points (which functions become `ros2 run` executables), and — importantly — which sub-packages and data files to install. A concrete, painful lesson from this project: the **core library package was missing from `setup.py`**, so although the code existed in the source tree, it was **not installed** into the ROS environment and imports failed at runtime. Adding it to the packages list in `setup.py` fixed it. The general rule: *if a module runs when you execute it directly but fails under `ros2 run`, suspect that it was never installed because `setup.py` does not list it.*

```
# setup.py (essential shape)
from setuptools import setup, find_packages
setup(
    name='semantic_drone',
    packages=find_packages(), # must actually pick up core/ and nodes/
    entry_points={
        'console_scripts': [
            'camera_node = semantic_drone.nodes.camera_node:main',
            'segmentation_node = semantic_drone.nodes.segmentation_node:main',
            'planner_node = semantic_drone.nodes.planner_node:main',
            'control_node = semantic_drone.nodes.control_node:main',
        ],
    },
)
```

5.9 `cv_bridge` — the OpenCV ↔ ROS image adapter

ROS carries images as `sensor_msgs/Image` (raw bytes + metadata); OpenCV/NumPy want an `ndarray`. `cv_bridge` converts between them:

```
from cv_bridge import CvBridge
bridge = CvBridge()

# incoming ROS Image -> OpenCV/NumPy
frame = bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')

# OpenCV/NumPy mask -> outgoing ROS Image
mask_msg = bridge.cv2_to_imgmsg(mask, encoding='mono8')
```

Two things to watch: the **encoding** must match what you actually have (`bgr8` vs `rgb8` swaps red and blue — a classic silent bug that makes HSV thresholds mysteriously fail), and mono/grayscale masks use `mono8`. Getting the encoding right is part of why the HSV thresholds had to be measured against a *real* frame rather than assumed.

5.10 Launch files and running the system

You can start each node by hand with `ros2 run semantic_drone camera_node`, but a real system starts many nodes together via a **launch file** (Python-based in ROS 2), which also sets parameters and remaps topics. Introspection tools used constantly during bring-up:

- `ros2 node list` / `ros2 node info <node>` — what is running, what it pub/subs.

- `ros2 topic list` / `ros2 topic echo <topic>` — see the actual messages.
 - `ros2 topic hz <topic>` — measure the real publish rate (is the camera actually at 30 Hz?).
 - `ros2 topic info <topic> -v` — shows QoS on each end (this is how you *catch* the QoS mismatch of 5.5).
-

6. The Codebase, Node by Node

This chapter walks through the actual `semantic_drone` package as it stands after the April–June 2026 work, node by node, stating each node's job, its inputs/outputs, and the specific changes and bug-fixes that were made. This is the map of the code.

6.1 `core/` — the shared library (and `px4.py`)

`core/` is a plain Python sub-package of shared code used by the nodes — most importantly `px4.py`, which wraps the MAVSDK connection and the high-level flight actions (connect, arm, take off, go to position). It is *library* code, not a node.

Key facts and changes about `core/px4.py`:

- The `connect()` routine had its health-check removed. MAVSDK's default connect flow can block waiting for the vehicle to report "all health checks passed" (GPS, home position, calibration, etc.). During bench/sim bring-up — no GPS lock, no home position — that check never passes and `connect()` hangs forever. Removing the health gate lets the code connect and proceed in exactly the situations bring-up needs. (The safety those checks provided is instead enforced by PX4's own prearm logic and by the operator; see the geofence discussion in Chapter 32.)
- `takeoff()` default altitude is **20.0 m** and the call is wrapped in `asyncio.wait_for(..., timeout=60)` so a takeoff that never completes fails cleanly after 60 s instead of hanging. Bounding every "wait for the vehicle to do X" call with a timeout is a lesson learned the hard way: without it, one stuck await freezes the whole control coroutine.
- Because `core/` was, at one point, **missing from `setup.py`**, it was not installed and imports failed under `ros2 run` (see 5.8). That was fixed by listing it in the packages.

Sketch of the wrapped connection/actions:

```
# core/px4.py (essential shape)
from mavsdk import System
import asyncio

class PX4:
    def __init__(self, url='udp://:14540'):
        self.url = url
        self.system = System()

    async def connect(self):
        await self.system.connect(system_address=self.url)
        # NOTE: no "wait until health_all_ok" gate here – intentional,
        # so bring-up without GPS/home does not hang.
        async for state in self.system.core.connection_state():
            if state.is_connected:
                break

    async def arm(self):
        await self.system.action.arm()

    async def takeoff(self, altitude=20.0):
        await self.system.action.set_takeoff_altitude(altitude)
        await self.system.action.arm()
        await asyncio.wait_for(self.system.action.takeoff(), timeout=60)
```

6.2 `nodes/camera_node.py` — the image source

Job: provide the downward image stream to the rest of the pipeline. In simulation it is fed from the Gazebo camera (via the bridge, 6.6); on hardware the RealSense driver provides frames.

Changes/facts:

- The camera **topic was updated to `x500_mono_cam_down_0`** to match the actual Gazebo camera sensor name for the downward mono camera on the x500 airframe. Topic-name drift between the simulator model and the code is a recurring cause of "no images," and this rename aligned them.
- **RELIABLE QoS was added** so the image publisher matches consumers that require reliable delivery (the QoS-compatibility issue of 5.5). Without this, the segmentation node's subscription would silently receive nothing.

6.3 `nodes/segmentation_node.py` — pixels to a road mask

Job: take each colour frame and output a binary mask marking road pixels. The current, deployed segmenter is **HSV colour thresholding** (fast, no GPU); the U-Net is the future replacement (Chapter 16).

The core operation: convert the frame to **HSV** colour space and keep only pixels whose (H, S, V) fall inside a calibrated range:

```
import cv2, numpy as np

LOWER = np.array([100, 0, 50])
UPPER = np.array([170, 50, 180])

def segment(frame_bgr):
    hsv = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, LOWER, UPPER) # 255 where road, else 0
    return mask
```

Changes/facts:

- **The HSV thresholds were calibrated against a real nadir frame**, not guessed. Measured pixel statistics on the road region of an actual downward frame were **H = 136–140, S = 21–24, V = 109–135**, and the thresholds were set to bracket that: **lower** `[100, 0, 50]` → **upper** `[170, 50, 180]`. This is the difference between a segmenter that works on *this* camera and one that works only in a tutorial. (Full theory of *why* those numbers, and why HSV rather than RGB, is in Chapter 15.)
- **Diagnostic logging of road-percentage every 30 frames** was added — i.e. what fraction of the image the mask marks as road. This single number is the fastest health check for the whole perception front-end: if road% suddenly drops to near zero, either the lighting/colour changed or the encoding flipped (bgr/rgb), and you catch it immediately instead of wondering why the planner has nothing to plan on.

6.4 `nodes/planner_node.py` — mask to a path

Job: turn the road mask into an occupancy grid, choose a goal automatically, run A* to that goal, and publish the resulting path/waypoints. This is the decision plane (Part VI covers the algorithms in full).

Changes/facts:

- **It now subscribes to `/mavsdk/pose` instead of `/odom`**. This is a pivotal change: `/odom` is simulation odometry that does not exist on real hardware, whereas `/mavsdk/pose` is the vehicle's position as reported by PX4 telemetry (republished by the control node). Making the planner consume the MAVSDK pose means the *same planner code* runs in sim and on the real drone — a deliberate step to erase a sim-only dependency.
- **A mask-zero diagnostic** was added: if the incoming mask is entirely empty (no road pixels), that is logged, because A* over an all-occupied grid can never find a path and you want to know the cause is "no road detected" rather than "planner broken."
- **`_find_auto_goal` ignores the top 20% of the image**. The far top of a downward frame is the most distorted/uncertain region (shallow viewing angle, horizon effects, most motion blur), so goals are chosen from the more reliable lower 80%. This makes the auto-goal both safer and more stable.
- **The goal-on-obstacle bug** — where `_find_auto_goal` could return a goal cell that sits on a non-road (occupied) pixel, which A* then cannot reach — was fixed with `_nearest_free_goal()`, a **radial spiral search** that, given a desired goal, walks outward in a spiral until it lands on the nearest free (road) cell and uses that instead. Details and code in Chapter 22.

6.5 `nodes/control_node.py` — path to motion, and the pose feedback

Job: consume the planner's waypoints and drive PX4 to fly them via MAVSDK, and — just as important — **stream the vehicle's telemetry pose back into ROS** so the planner can close the loop.

Changes/facts:

- The `/odom` subscription was removed (matching the planner change): the control node no longer depends on simulation odometry.
- A MAVSDK telemetry pose stream (`_pose_stream`) was added, which asynchronously reads position from `drone.telemetry` and publishes it on `/mavsdk/pose` as a `PointStamped`. This is the wire that closes the loop (see 2.2 and 3.2). It is also the piece implicated in the gRPC crash of 7.4.
- Startup logging was added so that when the node comes up you can see the connection/telemetry state, rather than staring at a silent process wondering whether it connected.

Sketch of the pose stream and the offboard follow:

```
# control_node.py (essential shape)
async def _pose_stream(self):
    async for pos in self.drone.telemetry.position_velocity_ned():
        msg = PointStamped()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.point.x = pos.position.north_m
        msg.point.y = pos.position.east_m
        msg.point.z = pos.position.down_m
        self.pose_pub.publish(msg)      # -> /mavsdk/pose

async def _follow(self, waypoint):
    await self.drone.offboard.set_position_ned(
        PositionNedYaw(waypoint.n, waypoint.e, waypoint.d, waypoint.yaw))
```

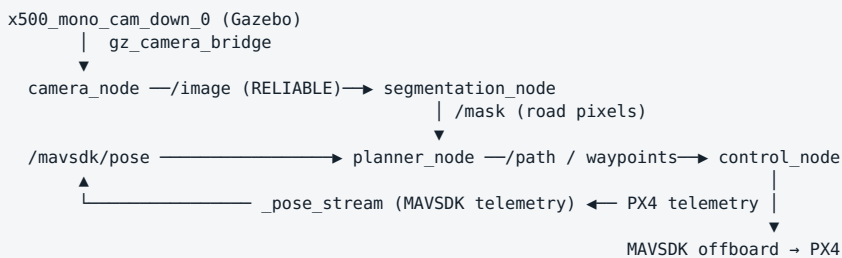
6.6 `nodes/gz_camera_bridge.py` — simulation image plumbing

Job (sim only): bridge the Gazebo camera topic into ROS 2 so `camera_node`/`segmentation_node` see simulated frames exactly as they would see real ones.

Changes/facts:

- The bridged topic was updated from `x500_mono_cam_0` to `x500_mono_cam_down_0` to match the downward camera sensor name in the Gazebo model. This is the mirror of the camera-node rename in 6.2; both had to agree on the same topic string. This kind of "two files must name the same topic" coupling is a frequent bring-up snag, and keeping the bridge and the consumer in sync is the fix.

6.7 How the nodes are wired together (recap)



7. The asyncio ↔ ROS 2 Integration Problem

This chapter is about the single most conceptually important software problem in the project: **two different concurrency systems both want to own the main thread**. Getting them to cooperate is what makes MAVSDK usable inside a ROS node, and misunderstanding it is behind the gRPC/pose-stream crash.

7.1 Two event loops, one thread

- **ROS 2** (`rclpy`) processes callbacks with an **executor**. `rclpy.spin(node)` blocks the thread and runs callbacks as messages/timers fire. It expects to own that thread.
- **MAVSDK-Python** is built on **asyncio**: its API is `async def` coroutines you `await`, run inside an **asyncio event loop** (`asyncio.run(...)` / `loop.run_until_complete(...)`). It, too, wants an event loop owning the thread.

You cannot naively call `rclpy.spin()` (blocks forever) and also `asyncio.run(my_coro())` (blocks forever) in the same thread — whichever you call first never returns, so the other never runs. This is the crux: **two blocking "run forever" loops, one thread**.

7.2 The general shapes of a solution

There are a few standard ways to reconcile them; the project's control node embodies one of them:

1. **Async-first, ROS spun cooperatively**. Make the program an asyncio program (`asyncio.run(main())`), and inside it drive ROS by repeatedly calling the *non-blocking* `rclpy.spin_once(node, timeout_sec=0)` from an async task, interleaving it with the MAVSDK coroutines. ROS callbacks and MAVSDK awaits then share the one event loop cooperatively.
2. **Two threads**. Run `rclpy.spin(node)` in one thread and the asyncio loop in another, marshalling data between them with thread-safe queues. Simpler conceptually but you must handle cross-thread hand-off carefully (and ROS publish calls need to be thread-safe).
3. **A MultiThreadedExecutor with an asyncio bridge**. More elaborate; overkill here.

The control node uses the async-first style: it establishes the MAVSDK connection and telemetry coroutines (like `pose_stream`) as asyncio tasks and services ROS alongside them, so that publishing `/mavsdk/pose` (a ROS action) and awaiting `drone.telemetry...` (an asyncio action) happen in the same cooperative loop.

Skeleton of the async-first pattern:

```
import asyncio, rclpy

async def spin_ros(node):
    while rclpy.ok():
        rclpy.spin_once(node, timeout_sec=0.0) # non-blocking
        await asyncio.sleep(0.005)           # yield to the event loop

async def main():
    rclpy.init()
    node = ControlNode()
    await node.connect()                      # MAVSDK connect (awaitable)
    await asyncio.gather(
        spin_ros(node),                      # ROS callbacks
        node._pose_stream(),                 # MAVSDK telemetry -> /mavsdk/pose
        node._control_loop(),                # consume waypoints -> offboard setpoints
    )

asyncio.run(main())
```

The `await asyncio.sleep(...)` in `spin_ros` is not a delay for its own sake — it is the **yield point** that lets the event loop hand time to the MAVSDK coroutines. Without a yield, one `while` loop would starve everything else. This is the cooperative-multitasking discipline asyncio requires: every long-running task must periodically `await` something to give others a turn.

7.3 Why this is easy to get wrong

Cooperative concurrency has no preemption: if any coroutine does blocking work (a synchronous sleep, a heavy CPU computation, a blocking I/O call) without `await` ing, it freezes *every other coroutine*, including the ROS callbacks and the telemetry stream. Symptoms are "the pose stops updating," "callbacks stop firing," "the drone stops responding" — all of which look like different bugs but share the root cause of a starved event loop. The timeouts added around MAVSDK calls (`asyncio.wait_for(..., 60)`, 6.1) are partly a defence against one stuck await taking the whole loop down.

7.4 The gRPC / `_pose_stream` crash and its true cause

A recurring, alarming symptom during simulation was a **gRPC crash tied to `_pose_stream`** — the telemetry stream would die and take the run with it. The important diagnosis: **this was traced to VM resource starvation, not a logic bug in the code**. MAVSDK-Python talks to the MAVSDK server over **gRPC**; when the simulation VM was starved of CPU/memory, the gRPC channel could not keep up with the telemetry stream and the connection collapsed, surfacing as a crash in `_pose_stream` because that is where the stream is consumed.

Two consequences followed:

1. **Do not "fix" the code that is merely the victim.** The stream code was correct; the environment was the problem. Recognising that a crash's *location* is not necessarily its *cause* is a genuine engineering lesson — the pose stream was where it manifested, resource starvation was why.
2. **A reconnect patch with exponential backoff was designed** (though not yet pushed) so that if the gRPC channel drops, the code re-establishes it rather than dying — waiting 1 s, then 2 s, then 4 s, etc., between attempts, so a transient starvation event self-heals instead of ending the flight. Exponential backoff is the standard way to reconnect to a flaky service without hammering it.

Shape of the intended reconnect wrapper:

```
async def _pose_stream_with_backoff(self):
    delay = 1.0
    while rclpy.ok():
        try:
            async for pos in self.drone.telemetry.position_velocity_ned():
                self._publish_pose(pos)
                delay = 1.0          # reset backoff on success
        except grpc.aio.AioRpcError:
            self.get_logger().warn(f'telemetry dropped; retry in {delay}s')
            await asyncio.sleep(delay)
            delay = min(delay * 2, 30.0)  # exponential, capped
```

Part III — The Flight Stack

8. PX4 Architecture

8.1 What PX4 is

PX4 is the **flight control firmware** — the real-time software that actually flies the aircraft. It runs on the Pixhawk (an embedded flight-controller board) and is responsible for everything that has to happen reliably and fast: reading the inertial sensors, estimating the vehicle's attitude and position, running the stabilisation and position control loops, arming and disarming safely, executing failsafes, and translating a desired motion into individual motor commands. When the project's Jetson says "go here," PX4 is what makes the four motors spin at the right speeds to get there without the aircraft tumbling out of the sky.

PX4 is a **hard real-time** system with a strict scheduler, used here for deterministic control at high rates. Non-real-time work (neural network inference, planning) runs on the companion computer.

8.2 The internal module + uORB architecture

Internally, PX4 is organised much like ROS in spirit: a set of **modules** (small programs) that communicate through a **publish/subscribe bus called uORB** (micro Object Request Broker). Each module publishes and subscribes to **topics** (e.g. `vehicle_attitude`, `sensor_combined`, `vehicle_local_position`, `actuator_outputs`). This decoupling is why PX4 is modular and portable across airframes.

The pipeline of modules that matters conceptually:

```
sensors (IMU, mag, baro, GPS)
  | publish raw measurements
  v
EKF2 (state estimator) — vehicle_attitude, vehicle_local_position
  |
  v
commander (arming / mode / failsafe state machine)
  |
  v
position controller → attitude controller → rate controller
  | (each turns a higher-level setpoint into a lower-level one)
  v
control allocation / mixer → actuator_outputs → ESC/motor PWM or DShot
```

- **Sensors** feed raw data in.
- **EKF2**, the Extended Kalman Filter estimator, fuses IMU + magnetometer + barometer + GPS into an estimate of attitude and position. All downstream control depends on it; sensor calibration therefore governs control accuracy.
- **Commander** is the state machine deciding whether the vehicle is armed, what flight mode it is in, and whether a failsafe should trigger.
- The **cascaded controllers** turn a position setpoint into an attitude setpoint into a body-rate setpoint into a torque/thrust demand.
- **Control allocation / mixing** turns that demand into per-motor outputs given the airframe geometry — the step that needs the actuator mapping to be correct (Chapter 31).

8.3 Flight modes

A **flight mode** determines where the setpoints come from. The ones that matter here:

- **Manual / Stabilized / Acro** — the pilot's sticks drive the vehicle directly (with varying amounts of stabilisation). Used to test that the aircraft flies at all.
- **Position / Altitude** — the vehicle holds position/altitude using GPS/baro; sticks command velocity. Needs a good position estimate.
- **Offboard** — **setpoints come from an external computer over MAVLink**. This is the mode the whole project depends

on: the Jetson streams position/velocity/attitude setpoints and PX4 obeys them. Offboard has strict rules (8.6).

- **Hold / Return / Mission** — autonomous modes driven by PX4's own navigator; not central here since planning is done off-board on the Jetson.

Mode-switch and kill-switch assignments were completed as part of bring-up (Chapter 30). The kill switch is a hardware motor cut-off, independent of software state.

8.4 Arming and the safety chain

Arming is PX4 permitting the motors to spin. Before it will arm, PX4 runs a chain of **prearm checks**: sensors healthy and calibrated, estimator converged, RC present (unless configured otherwise), no active failsafe, and — if a geofence needing position is configured — a valid position estimate. Any failed check blocks arming and reports a reason in `QGroundControl`.

Two prearm-related realities shaped this project:

- The **"Fence requires position" prearm warning** appears because a geofence is configured but the vehicle has no position lock (no GPS fix indoors). Until GPS lock is obtained outdoors, this blocks arming. This is one of the two remaining hardware blockers (Chapter 32).
- On the software side, the MAVSDK `connect()` health gate was removed (6.1) precisely so bring-up code does not hang waiting for these same checks to pass in an environment where they cannot (bench, no GPS). The checks still exist and still protect a real flight; the code just does not *block on the client side* waiting for them.

8.5 Failsafes

A **failsafe** is an automatic protective action when something goes wrong: loss of RC link, loss of the offboard command stream, low battery, loss of position estimate, geofence breach. Depending on configuration, PX4 will hold, return-to-launch, or land. Two of the changed parameters (Chapter 11) directly tune failsafe behaviour so that *the specific things that happen during off-board autonomy bring-up do not trip an unwanted failsafe* — e.g. so that not having an RC transmitter connected, or a momentary gap in the offboard stream, does not immediately abort. Configuring failsafes is a balance: too twitchy and you cannot test; too lax and you lose the aircraft.

8.6 The offboard-mode rules (why setpoints must stream continuously)

Offboard mode has a rule that trips everyone once: **PX4 requires a continuous stream of setpoints (roughly ≥ 2 Hz) before it will enter offboard and to stay in it**. If the stream stops, PX4 assumes the companion computer died and triggers a failsafe. Practically:

1. You must **start streaming setpoints before commanding offboard mode**, or the switch to offboard is rejected.
2. You must **keep streaming** for the whole duration, even while "holding," or the mode drops out.

This is why the control node is built around a steady setpoint cadence and why a starved event loop (7.3) is dangerous — if the loop stalls, the setpoint stream gaps and PX4 bails out of offboard. MAVSDK's `offboard` plugin has an explicit `start()`/`stop()` and expects you to have primed at least one setpoint before `start()`.

```
# priming offboard before starting it (MAVSDK pattern)
await drone.offboard.set_position_ned(PositionNedYaw(0, 0, 0, 0)) # prime
await drone.offboard.start()                                     # now legal
# ...then keep pushing setpoints continuously...
```

9. MAVLink and MAVSDK-Python

9.1 MAVLink — the protocol

MAVLink is the lightweight messaging protocol drones use to talk to ground stations and companion computers. It defines a compact binary packet format and a large dictionary of message types: heartbeats, attitude, GPS, battery, command-long (do X), position-target (go here), etc. It is the *lingua franca* between PX4 and everything off-board — QGroundControl speaks it, and so does the Jetson.

You rarely touch raw MAVLink in this project because MAVSDK wraps it, but it is worth knowing that underneath the clean Python calls, discrete MAVLink packets are flowing over the serial/USB link, and that the **heartbeat** message is what each side uses to know the other is alive. Loss of heartbeat is a failsafe trigger.

9.2 MAVSDK — the clean API over MAVLink

MAVSDK is a set of libraries (with a Python binding, MAVSDK-Python) that presents MAVLink as an ergonomic, high-level API organised into **plugins**, each covering an area:

- **action** — discrete commands: `arm()`, `takeoff()`, `land()`, `set_takeoff_altitude()`.
- **offboard** — streaming setpoints: `set_position_ned()`, `set_velocity_ned()`, `start()`, `stop()`.
- **telemetry** — subscribe to state streams: `position()`, `position_velocity_ned()`, `attitude_euler()`, `flight_mode()`, `armed()`, `health()`.
- **mission**, **param**, etc. — missions and parameter access.

Architecturally, MAVSDK-Python is a **thin client that talks over gRPC to a MAVSDK server**, which in turn speaks MAVLink to the vehicle. That gRPC hop is exactly the channel whose collapse under VM starvation caused the `_pose_stream` crash (7.4). Knowing the architecture is what let that crash be correctly attributed to the environment rather than the code.

```
your Python —gRPC→ mavsdk_server —MAVLink→ PX4
(asynchronous API) (protocol engine) (serial/UDP)
```

9.3 Everything is a coroutine

Because MAVSDK-Python is async, **every call is awaited** and every telemetry stream is an **async iterator**:

```
from mavsdk import System
drone = System()
await drone.connect(system_address='udp://:14540')

await drone.action.arm()
await drone.action.takeoff()

async for pos in drone.telemetry.position():
    print(pos.latitude_deg, pos.longitude_deg, pos.relative_altitude_m)
```

This async nature is the reason for the entire Chapter 7 discussion: to use MAVSDK inside a ROS node, its event loop has to coexist with ROS's executor. The project's `core/px4.py` wraps the common actions (connect/arm/takeoff) so the nodes call clean methods, and the control node runs the telemetry iterators as asyncio tasks that publish into ROS.

9.4 The connection address

MAVSDK connects via a URL that encodes the transport:

- `udp://:14540` — UDP on port 14540, the default PX4 SITL offboard port (used in simulation).
- `serial:///dev/ttyACM0:57600` (or similar) — a serial link, used on real hardware where the Jetson is wired to the Pixhawk.

Getting this address right (and the matching PX4-side configuration/ baud) is part of hardware bring-up: in sim it is UDP to the simulator; on the drone it is the physical serial/USB connection to the flight controller.

10. Offboard Control in Practice

10.1 The control node's contract with PX4

Chapter 6 introduced the control node; this chapter states precisely what it must do to fly the planner's path safely. It has three jobs running concurrently as asyncio tasks:

1. **Maintain the offboard setpoint stream.** Continuously push position (or velocity) setpoints at a steady rate so PX4 stays in offboard (8.6). Even when "holding" over the current waypoint, it must keep sending the same setpoint.
2. **Consume waypoints from the planner.** As the planner publishes new paths/waypoints, update the current target the setpoint stream is driving toward.
3. **Stream telemetry pose back to ROS.** Run `_pose_stream` to publish `/mavsdk/pose` so the planner can close the loop.

The delicate part is that all three share one cooperative event loop, so none may block (7.3), and the setpoint cadence must never gap.

10.2 Position setpoints and the NED frame

Offboard position setpoints are given in **NED** (North-East-Down) local coordinates plus a yaw:

```
from mavsdk.offboard import PositionNedYaw
await drone.offboard.set_position_ned(
    PositionNedYaw(north_m, east_m, down_m, yaw_deg))
```

Note **Down is positive** — to fly *up* to 20 m altitude you command `down = -20`. This sign convention is a classic source of "the drone dove into the ground" bugs and is one more instance of the frames problem from 3.3. The planner produces a path in grid/image space; the control node must transform that into NED position targets with the correct signs before streaming them.

10.3 The takeoff sequence and altitude gating

Takeoff is orchestrated (via `core/px4.py`) roughly as: set takeoff altitude (default **20 m**, 6.1), arm, `takeoff()` bounded by a **60 s** `asyncio.wait_for` so it fails cleanly if it stalls. Once airborne and stable, the node primes an offboard setpoint and switches to offboard to begin following the planner.

A related simulation-side idea is **altitude-gated takeoff**: the pipeline does not begin the road-following behaviour until the drone has actually reached a safe operating altitude. This prevents the planner from trying to fly a path while the aircraft is still on the ground or climbing through the first metre — the segmentation and geometry only make sense from altitude, where the downward camera sees a useful swath of road. Gating on altitude is a small but important guard that sequences "get airborne" strictly before "start following."

10.4 Velocity vs position control

Two ways to drive the vehicle offboard:

- **Position setpoints** (`set_position_ned`) — "be at this point." Simple; PX4 handles the trajectory. Good for waypoint following.
- **Velocity setpoints** (`set_velocity_ned` / body velocity) — "move at this velocity." More direct control of motion; useful for smooth continuous following where you command a heading/speed derived from the road direction rather than discrete points.

For road-following, position-waypoint control is the straightforward choice and matches the planner's waypoint output; velocity control is an option for smoother continuous tracking (and connects to the "hold heading across a gap" idea in the occlusion-recovery theory, Chapter 34).

11. PX4 Parameters That Were Changed (and Why)

PX4's behaviour is governed by hundreds of named **parameters** you set in QGroundControl or via MAVSDK's `param` plugin. Changing the right few is often what turns "PX4 fights my off-board autonomy" into "PX4 cooperates." Here are the parameters this project deliberately set, each with the reason.

11.1 `COM_RCL_EXCEPT = 4`

`COM_RCL_EXCEPT` ("RC loss exceptions") controls **which modes are exempt from the RC-loss failsafe**. By default, losing the RC transmitter link triggers a failsafe — sensible for a manually piloted drone, fatal for autonomy bring-up on a bench with no transmitter bound. Setting `COM_RCL_EXCEPT = 4` marks **offboard** as exempt, so the vehicle does **not** abort offboard control just because there is no RC link. This is essential when the whole point is that a companion computer, not a human with a transmitter, is in command. (The bit value 4 corresponds to the offboard exception in PX4's bitmask for this parameter.)

11.2 `NAV_DLL_ACT = 0`

`NAV_DLL_ACT` is the **data-link-loss failsafe action** — what PX4 does if it loses its connection to the ground control station / companion data link. `0` sets the action to **disabled / no action**, so a momentary loss of the GCS link does not trigger an automatic return or land during bring-up and testing. Like `COM_RCL_EXCEPT`, this is a *bring-up* setting that removes an over-eager safety reaction so you can actually develop the autonomy; on a real deployed flight you would reconsider these to restore appropriate protections.

11.3 Why these two together

Both parameters address the same underlying friction: **PX4's defaults assume a piloted vehicle with a human and a transmitter, and they treat the absence of RC/GCS as an emergency**. In this project the "pilot" is the Jetson over offboard, so the defaults fire failsafes at exactly the wrong times. `COM_RCL_EXCEPT = 4` says "offboard is allowed to run without RC," and `NAV_DLL_ACT = 0` says "don't panic on data-link loss." Together they let the off-board autonomy operate without being constantly interrupted by protections meant for a different operating model.

Safety note. These are bring-up/test settings that intentionally relax protections. They are appropriate on a bench and in controlled testing where a **kill switch** is the real backstop, but each one you relax is a protection you have removed — so they are exactly the kind of thing to revisit before any flight where losing the aircraft matters.

11.4 The calibration "parameters" (context)

Beyond these behavioural parameters, the **sensor calibrations** (accelerometer, gyro, compass, RC) are also stored as parameters, written during the QGroundControl calibration flows (Chapter 29). They are what make EKF2's state estimate trustworthy. So "the parameters that were set" really fall into two families: **calibration data** (making the estimate correct) and **behavioural flags like the two above** (making the failsafe logic compatible with off-board autonomy).

11.5 Reading and setting parameters programmatically

Although QGroundControl is the usual tool, MAVSDK can read/write parameters too, which is useful for scripting a known-good configuration onto a fresh board:

```
await drone.param.set_param_int('COM_RCL_EXCEPT', 4)
await drone.param.set_param_int('NAV_DLL_ACT', 0)
val = await drone.param.get_param_int('COM_RCL_EXCEPT')
```

Part IV — Simulation

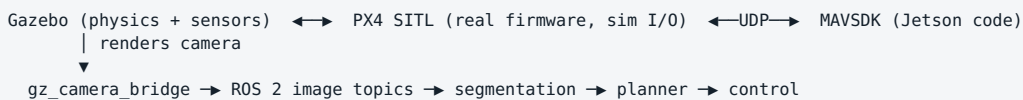
12. Gazebo + PX4 SITL

12.1 Why simulate at all

You do not debug a perception-planning-control loop by crashing real quadrotors. Simulation lets you run the exact same ROS 2 nodes against a synthetic world where mistakes cost nothing, iterate in seconds, and reproduce a scenario perfectly. The project's entire pipeline was **validated end-to-end in simulation** before hardware bring-up, which is why the transition to real hardware is a matter of re-plumbing inputs (real camera, real serial link, real GPS) rather than re-writing logic. The simulation is the safety net and the development speed-up in one.

12.2 SITL — Software In The Loop

PX4 SITL runs the *real PX4 firmware as a normal program on your PC*, with the sensors and motors replaced by simulated ones fed from a physics simulator. The flight code is genuinely the same code that runs on the Pixhawk — only the hardware I/O is faked. This is what makes SITL trustworthy: you are testing the actual flight stack, its EKF2, its controllers, its arming logic, driven by simulated IMU/GPS/baro. MAVSDK connects to SITL over `udp://:14540` exactly as it would connect to a real vehicle, so the companion-side code is unchanged between sim and hardware.



12.3 Gazebo — the physics and rendering

Gazebo is the 3-D simulator providing rigid-body physics, sensor simulation, and rendering. It flies a model of the airframe, computes how it responds to motor forces, and renders what the drone's camera would see. For this project the crucial output is the **simulated downward camera image** — Gazebo renders the scene from the drone's nadir camera and that image is what the perception pipeline consumes, standing in for the real RealSense feed.

12.4 The x500 airframe and the downward camera

The simulated vehicle is the **x500** quadrotor (a standard PX4 SITL airframe). The project uses a variant carrying a **downward-facing mono camera**, whose Gazebo sensor is named `x500_mono_cam_down_0`. Two files had to agree on that exact sensor/topic name — the **camera node** and the **gz_camera_bridge** — and getting them aligned (renaming from the earlier `x500_mono_cam_0` to `x500_mono_cam_down_0`) was the fix for images not flowing (6.2, 6.6). The "mono" and "down" in the name encode two real properties: it is a single-channel-style downward camera, and it looks straight down (nadir), which is the viewing geometry the whole road-following approach assumes.

12.5 The camera bridge

Gazebo publishes its camera on a Gazebo-transport topic, not a ROS 2 topic. The **gz_camera_bridge** node bridges that Gazebo image into a ROS 2 `sensor_msgs/Image` so the rest of the pipeline sees frames identically to how it would see real camera frames. This is the seam that makes "sim vs real" invisible to the perception nodes: whether the bytes came from Gazebo's renderer or a RealSense, **segmentation_node** gets a ROS image and does not care. Preserving that indifference — same message type, same topic contract — is what lets the identical code run in both worlds.

12.6 Where the pipeline was validated

The whole loop — camera → HSV segmentation → occupancy grid → A* → waypoints → MAVSDK offboard → PX4 SITL flying the path — was confirmed working in this Gazebo+SITL setup, including on the **native cobot desktop** (running natively rather than in a resource-starved VM, which is precisely what avoided the gRPC starvation crash of 7.4). "Validated end-to-end in sim" is the milestone that justified moving to hardware.

13. The Simulation-to-Reality Gap

13.1 What transfers cleanly

Because SITL is the real firmware and the ROS nodes are literally the same code, a lot transfers with no change:

- **All the ROS 2 logic** — nodes, topics, QoS, the planner, the auto-goal, the waypoint handling.
- **The MAVSDK interface** — connect/arm/takeoff/offboard/telemetry calls are identical; only the connection URL changes from UDP to serial.
- **The control-loop structure** — the offboard setpoint streaming, the pose feedback, the async concurrency.
- **PX4's behaviour** — arming logic, failsafes, the parameters (`COM_RCL_EXCEPT` , `NAV_DLL_ACT`) mean the same thing.

This is the payoff of the architecture: the sim-to-real change is mostly at the edges (inputs and the physical link), not the core.

13.2 What does not transfer — the domain gap in perception

The biggest gap is in **perception**, and it has two parts:

- **The image itself looks different.** Gazebo's rendered road and a real road (via the RealSense) differ in colour, texture, noise, lighting, shadows, and dynamic range. A segmenter tuned to one is not automatically right for the other. This is why the **HSV thresholds were re-measured on a real nadir frame** (H = 136–140, S = 21–24, V = 109–135) rather than reused from sim — the colours are simply not the same, and Chapter 15 explains why HSV makes this re-tuning tractable.
- **The learned model's domain gap.** The U-Net was trained on the **Aeroscapes** aerial dataset, whose imagery matches neither the sim renderer nor the exact real camera perfectly. A network only performs on data that resembles its training distribution; feed it out-of-distribution frames and it degrades. This **domain gap is the stated reason the U-Net is deferred** in favour of HSV until it can be adapted (fine-tuned/retrained on representative frames) — see Chapter 16.

13.3 What does not transfer — physics and sensing realities

- **Real IMUs drift and are noisy; real GPS has error and needs a lock.** SITL's sensors are cleaner and its "GPS" always has a fix. On hardware you meet the **"Fence requires position" prearm** because there is no indoor GPS lock — a problem that literally cannot occur in the always-fixed simulator (Chapter 32).
- **Real motors/ESCs must be wired and their spin direction verified.** In sim the mixing is defined by the model; on hardware you must do the **actuator mapping** and confirm each motor spins the correct way (Chapter 31). A reversed motor that "works" in sim will flip a real aircraft instantly.
- **Real USB has bandwidth limits.** The RealSense stalling at USB 2.1 rates (Chapter 18) is a pure hardware phenomenon with no simulation analogue.
- **Real compute is finite and shared.** The gRPC starvation crash (7.4) was a *resource* problem; running natively on capable hardware avoided it, but it is a reminder that on the actual Jetson, the network, the planner, and the telemetry stream all compete for a fixed GPU/CPU budget in a way the desktop sim did not stress identically.

13.4 The strategy the gap dictates

The sim-to-real gap is exactly why the bring-up is sequenced the way it is: prove the *logic* in sim (done), then re-establish the *bottom of the stack* on hardware (flash, calibrate, wire, GPS), then re-tune *perception* on real frames (HSV re-measured; U-Net adaptation pending), rather than trying to change everything at once. Each gap item maps to a specific remaining task in Part X. Treating the gap as a checklist — "which of my assumptions was only true in sim?" — is the discipline that makes hardware bring-up finite instead of open-ended.

Part V — Perception

14. The Camera Pipeline

14.1 What "the camera pipeline" has to deliver

Everything downstream depends on one thing: a steady stream of usable downward images arriving on a ROS topic at a predictable rate and in a known encoding. The camera pipeline's whole job is to make that true, whether the source is Gazebo (sim) or the RealSense D455 (hardware), so that `segmentation_node` never has to know or care which. The perception result is only ever as good as this input; a wrong colour encoding, a stalled frame rate, or a mismatched QoS here poisons segmentation, which poisons the grid, which poisons the plan.

14.2 The nadir viewing geometry

The camera looks **straight down (nadir)**. This choice is fundamental to the road-following approach and has consequences:

- A downward view turns "follow the road" into a clean 2-D image problem: the road appears as a band across the frame, and its direction in the image maps directly to a direction to fly. A forward-tilted camera would mix road appearance with perspective and horizon, making the geometry far messier.
- **Altitude sets the ground footprint.** Higher altitude = wider swath of road in view = more context for the planner and more robustness to the road briefly leaving frame; lower = finer detail but a narrow view. This is why the occlusion-recovery theory's first move is to *climb* to widen the field of view (Chapter 34), and why altitude-gated takeoff matters (10.3).
- The **most reliable image region is the centre-to-lower part of the frame**; the far top is the most distorted and motion-blurred, which is exactly why `_find_auto_goal` ignores the top 20% (6.4, 22.3).

14.3 Frame rate, encoding, and QoS — the three things that must be right

- **Rate.** The planner replans continuously, so it needs frames often enough to keep the plan fresh as the drone moves. `ros2 topic hz /image` is the tool to confirm the camera is actually publishing at the expected rate rather than stalling.
- **Encoding.** OpenCV works in **BGR** by default; ROS images can be `rgb8` or `bgr8`. If the segmenter converts assuming the wrong order, red and blue swap and the HSV thresholds silently fail. `cv_bridge`'s `desired_encoding` must match reality (5.9). This is a top suspect whenever segmentation "stops working" after a camera change.
- **QoS.** Sensor streams and their consumers must have compatible QoS or no frames arrive (5.5). The camera node was pinned to `RELIABLE` to match the segmentation subscription.

14.4 Sim source vs hardware source

- **In simulation**, frames come from Gazebo's rendered `x500_mono_cam_down_0` sensor, bridged to ROS by `gz_camera_bridge` (Chapter 12).
- **On hardware**, frames come from the **RealSense D455** via its ROS 2 driver (Chapter 18). The D455 also provides depth, which the current pipeline does not yet use for segmentation but which is available for future obstacle/altitude reasoning.

Because both deliver a `sensor_msgs/Image` on the agreed topic, the swap from sim to hardware camera is, from the pipeline's point of view, invisible — the whole reason the bridge and topic-naming discipline mattered.

15. HSV Colour Segmentation

15.1 The idea

The deployed road segmenter is **colour thresholding in HSV space**: classify a pixel as "road" if its colour falls inside a measured range, else "not road." It is classical computer vision — no learning, no GPU — and it is the workhorse that let the *rest* of the pipeline (planning, control, hardware) be built and tested without waiting on a neural network. Simple, deterministic, fast, and trivially debuggable: exactly what bring-up needs.

15.2 Why HSV and not RGB — the crucial insight

In **RGB**, a colour is a mix of red, green, and blue. The problem: when lighting changes — a cloud passes, a shadow falls, the sun brightens — **all three RGB channels change together**, so the "same" road under different light lands at very different RGB values. Thresholding in RGB is therefore fragile to lighting.

HSV re-parameterises colour into three more meaningful axes:

- **Hue (H)**: the *kind* of colour (where it sits on the colour wheel) — red vs blue vs grey-ish. Largely independent of how bright or washed-out the colour is.
- **Saturation (S)**: how *vivid* vs *washed-out/grey* the colour is. Road surfaces (asphalt/concrete) are typically **low-saturation** (greyish), which is itself a useful discriminator.
- **Value (V)**: how *bright* the pixel is. This is the axis that moves most when lighting changes.

The payoff: by putting brightness on its own axis (V), HSV lets you be **loose on brightness but tight on the actual colour character (H, S)**, so a threshold survives lighting changes far better than RGB. That is the entire reason the project segments in HSV. It also matches intuition about roads: "a not-very-colourful (low S), mid-brightness (broad V) surface of a particular greyish hue" is naturally expressed as a box in HSV and clumsily expressed in RGB.

15.3 The measured thresholds — and why they were measured, not guessed

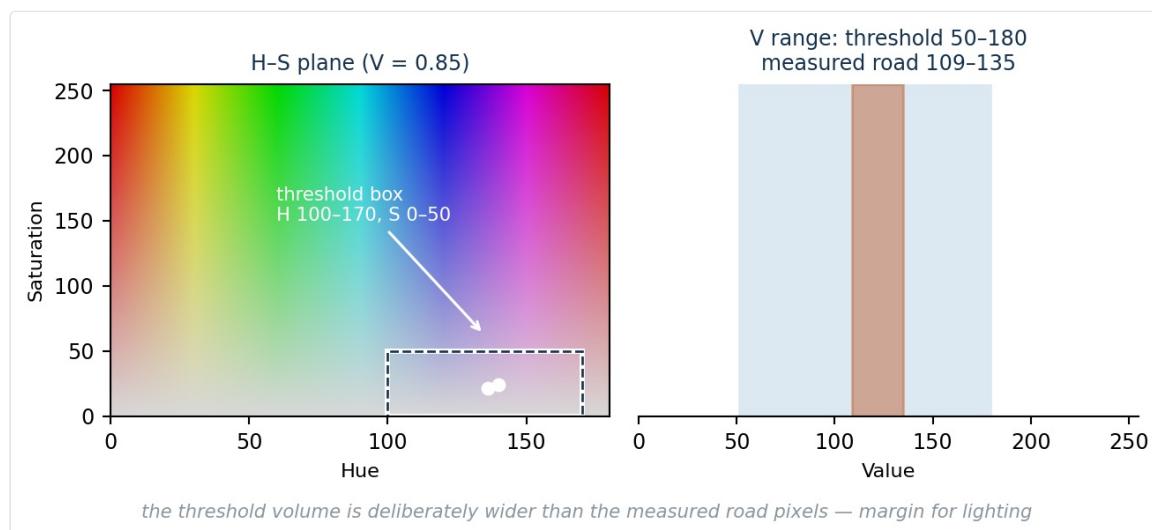


Figure 3. The HSV road-threshold volume, against the pixel values actually measured off a real nadir frame (H 136–140, S 21–24, V 109–135).

The thresholds in use were **calibrated from an actual nadir frame**, not copied from a tutorial. The road region of a real downward image measured:

- **H $\approx$ 136–140**
- **S $\approx$ 21–24** (low — confirming the road is greyish/washed-out)
- **V $\approx$ 109–135** (mid brightness)

and the operating thresholds were set to comfortably bracket that measurement:

- **Lower bound** [100, 0, 50]
- **Upper bound** [170, 50, 180]

Read what those brackets *mean*: a **wide hue window (100–170)** around the measured 136–140 to tolerate colour variation and camera differences; a **saturation ceiling of 50** to admit only low-saturation (greyish) pixels — i.e. "roads are not vivid" made into a rule; and a **broad value window (50–180)** to be forgiving about brightness, which is the whole point of using V as a separate axis. The bounds are looser than the raw measurement on purpose: tight enough to reject vivid, off-hue regions, loose enough to keep the road as light and viewing angle vary across the frame.

```
import cv2, numpy as np

LOWER = np.array([100, 0, 50]) # H_lo, S_lo, V_lo
UPPER = np.array([170, 50, 180]) # H_hi, S_hi, V_hi

def road_mask(bgr):
    hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV) # note: BGR in, or R/B swap!
    mask = cv2.inRange(hsv, LOWER, UPPER) # 255=road, 0=not road
    return mask
```

The lesson embedded here is a general one about applied CV: **you measure thresholds on your own data**. The reason the earlier sim thresholds could not simply be reused on hardware is the domain gap (13.2) — real and rendered roads occupy different HSV boxes — and HSV's channel separation is exactly what made re-measuring a quick, well-understood adjustment rather than a mystery.

15.4 The road-percentage diagnostic

A single logged number — the **fraction of the frame the mask calls road, logged every 30 frames** — is the health monitor for this whole stage (6.3). Interpreting it:

- **Reasonable, stable road%:** segmentation is working; the planner has free space to plan on.
- **Suddenly near-zero:** the road left the box — likely a lighting/colour shift beyond the thresholds, or a **BGR/RGB swap** flipping the colours, or the camera actually not seeing road. Because you log it periodically, you notice within a second instead of debugging a silent downstream failure.
- **Suddenly near-100%:** thresholds too loose or the whole frame is road-coloured; the grid becomes all-free and the plan degenerates.

This diagnostic pairs with the planner's **mask-zero diagnostic** (6.4): together they pin exactly where "the drone won't plan" comes from — no road detected vs. planner fault.

15.5 Strengths, limits, and why it is a stepping stone

Strengths: fast (a colour conversion and a threshold per frame), deterministic (same input → same mask, great for debugging), no GPU, no training data, and — done in HSV — reasonably lighting-robust.

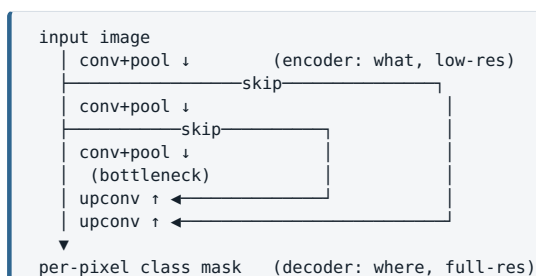
Limits: it only knows *colour*. It cannot tell a grey road from a grey rooftop or a grey parking lot of the same hue and saturation; it has no notion of *shape*, *context*, or *texture*; and a road that is off-colour (wet, sun-bleached, different material) can fall outside the box. These limits are precisely what a **learned semantic segmenter (U-Net)** overcomes by using spatial context and texture, and what the proposed **texture feature** (Chapter 19) begins to address even within a classical pipeline. HSV is the right tool to get flying; it is not the final perception system.

16. Semantic Segmentation with U-Net

Superseded — read with Part X. This chapter describes the *first* U-Net and the reasoning that led to deferring it in favour of HSV. That reasoning was partly wrong: the network's near-empty masks were later traced to three stacked bugs, not to the domain gap alone. The architecture, loss, and results were subsequently rebuilt. **Part X (Chapters 39–43) is the current state.** This chapter is retained because the concepts it explains — encoder–decoder structure, skip connections, why segmentation is a per-pixel classification — remain correct and are assumed by Part X.

16.1 What a U-Net is

U-Net is a convolutional neural network architecture designed for **semantic segmentation** — labelling every pixel with a class. Its name comes from its shape: a **contracting path** (an encoder that repeatedly downsamples the image, capturing *what* is present at ever-coarser scales) followed by an **expanding path** (a decoder that upsamples back to full resolution, recovering *where* things are), drawn as a "U." The signature feature is the **skip connections** that copy high-resolution features from the encoder directly across to the matching decoder level, so the network combines *coarse* "what" with *fine* "where" and can draw sharp, correctly-placed boundaries.



For this project the classes reduce to the binary question that matters: **road vs not-road** — the same output shape as the HSV mask, but produced by a network that has *learned* what road looks like from context and texture rather than from a hand-set colour box.

16.2 Why a learned segmenter is the principled goal

The HSV segmenter's blindness (15.5) is the motivation: a U-Net can, in principle, distinguish a road from a same-coloured rooftop because it uses **spatial context** (roads are long connected ribbons; rooftops are blobs), **texture** (asphalt vs tiles vs grass have different fine structure), and **shape**, none of which a per-pixel colour threshold can see. That generality is the whole reason "semantic" is in the project's name — the intended long-term perception is learned scene understanding, with HSV as the bootstrap.

16.3 The training setup

The network was trained in **TensorFlow** on the **Aeroscapes** dataset — an aerial semantic-segmentation dataset (drone imagery with per-pixel class labels including road-like classes). Training a segmenter means showing it many (image, ground-truth-mask) pairs and adjusting its weights so its predicted masks match the labels, using a segmentation loss (cross-entropy and/or a region-overlap loss such as Dice/IoU). The result is a set of weights that map an input frame to a predicted road mask.

16.4 The domain gap — why it is trained but deferred

The decisive practical fact: **the U-Net is trained but not deployed, because of a domain gap.** A neural network only performs well on inputs that resemble its **training distribution**. Aeroscapes imagery does not match the exact appearance of either the Gazebo renderer or the project's real downward camera — different altitude, optics, colour rendition, resolution, and scene statistics. Fed those out-of-distribution frames, the network's masks degrade, sometimes badly. So rather than fly an unreliable segmenter, the project uses the **deterministic, tunable HSV** segmenter for now and holds the U-Net until it can be **adapted** to the real domain (fine-tuning or retraining on frames that actually look like what the

drone sees, and/or augmentation that closes the gap). This is a mature engineering judgement: a simple method that *reliably* works beats a sophisticated one that *unreliably* works, until the sophisticated one is made reliable.

16.5 Latency and the road to deployment

Two hurdles stand between the trained U-Net and deployment on the drone:

1. **The domain gap** (above) — a *correctness* problem, solved by domain-appropriate retraining/fine-tuning.
 2. **Latency on the Jetson** — a *speed* problem. A full U-Net run per frame can be too slow for a real-time loop on an 8 GB Orin Nano. Two levers address this: **TensorRT** conversion for GPU-optimised inference (Chapter 17), and architectural/feature changes to reduce the work — which is exactly the context of **Dr. Mishra's proposal to add texture as a second segmentation feature to reduce U-Net latency** (Chapter 19): give the network a stronger, cheaper cue so it can be smaller/faster while staying accurate.
-

17. TensorRT and Inference on the Jetson

17.1 The problem TensorRT solves

Training happens in a framework (TensorFlow) optimised for flexibility, not for the fastest possible inference on one specific GPU. On the drone you do not train — you only run the trained network, repeatedly, as fast as possible, on a fixed device (the Jetson's GPU). **TensorRT** is NVIDIA's inference optimiser and runtime that takes a trained network and compiles it into a highly optimised engine for *that* GPU, typically giving a large speed-up over running the original framework. On a power- and compute-constrained flying computer, that speed-up can be the difference between a segmenter that keeps up with the control loop and one that cannot.

17.2 How it speeds things up

TensorRT applies several optimisations when it builds an engine:

- **Layer/kernel fusion** — combining several operations (e.g. convolution + bias + activation) into one GPU kernel, cutting memory traffic and launch overhead.
- **Precision reduction** — running in **FP16** (half precision) or **INT8** (8-bit integer) instead of FP32, which on the Jetson's tensor cores is much faster and uses less memory. INT8 requires a calibration step to keep accuracy acceptable; FP16 is often a near-free win. The **8 GB memory budget** of the Orin Nano makes this precision/size trade-off matter — a smaller, lower-precision engine fits and runs where a full-precision one might not.
- **Kernel auto-tuning** — picking the fastest implementation of each layer for the specific GPU.
- **Memory planning** — reusing buffers to minimise footprint.

17.3 The workflow

The recommended path for this project is to take the trained TensorFlow U-Net and convert it for the Jetson:

```
TensorFlow model → (ONNX export) → TensorRT build (FP16/INT8) → .engine → run on Jetson
```

Typically the model is exported to **ONNX** (an interchange format), then TensorRT parses the ONNX and builds an optimised engine (`.plan`/`.engine`) for the Jetson, which is loaded and run per frame. **TensorRT conversion was recommended for the Jetson deployment path** because the raw framework model is too slow for the real-time loop. This is a deployment-time transformation; it does not change what the network computes, only its runtime on this hardware.

17.4 Why this is a separate concern from the domain gap

It is worth keeping straight: **TensorRT fixes speed, not correctness**. A U-Net accelerated with TensorRT but still suffering the domain gap will run fast and be wrong. A domain-adapted U-Net without TensorRT will be right but possibly too slow. Deploying the learned segmenter needs *both*: retrain/fine-tune to close the domain gap (correctness) **and** convert with TensorRT (speed). That is why both appear on the "further required" list independently (Part X).

Status update (Part XVI, §62).

TensorRT FP16 conversion described above has since been completed. Perception latency fell from ≈ 309 ms (3.2 Hz, plain TensorFlow) to ≈ 31.2 ms (≈ 32 FPS) end-to-end; inference itself dropped to 6.86 ms. Domain fine-tuning on real D455 frames (Ch. 43; Part XV) remains the one open item above.

18. The Intel RealSense D455



Figure 18.1. The RealSense D455 (on tripod) publishing live to the Jetson Orin Nano (left) — the configuration used for the intrinsics measurement and end-to-end perception smoke tests. On this bench the camera enumerated at USB 2.1, capping the stream at 640×480 @ 15 fps; the flight-configuration mount will use a USB 3.x port.

18.1 What it is and why it was chosen

The **Intel RealSense D455** is the real downward sensor on the drone: a single USB device that provides an **RGB image** and a **depth stream** (a per-pixel distance map) together, with a well-supported ROS 2 driver. It was chosen because it delivers, in one unit, both the colour image the segmentation needs *now* and the depth information that opens up obstacle-awareness and altitude sanity-checking *later* — and because its ROS driver makes it publish frames as standard `sensor_msgs/Image`, so it drops into the pipeline in place of the simulated camera with no change to the perception nodes.

18.2 How the depth works (stereo)

The D455 is a **stereo depth camera**: it has two infrared cameras a fixed distance apart, and it computes depth by **triangulation** — finding where each scene point appears in the left vs right image (the *disparity*) and using the known camera separation (baseline) and geometry to solve for distance. A wider baseline (the D455 has a relatively large one for its class) gives better depth accuracy at range, which is well-suited to looking down from altitude. An onboard projector adds infrared texture to help stereo matching on plain surfaces. The result is a depth image aligned to the colour image, so each colour pixel can carry a distance.

18.3 The USB bandwidth reality — the live blocker

A concrete hardware lesson: **the D455 is live on the Jetson, but constrained by a USB bandwidth cap at USB 2.1 rates**, and moving it to a genuine **USB 3.x** port is flagged to resolve this. Why it happens and why it matters:

- Streaming colour **and** depth at full resolution and frame rate is a lot of data — enough to need **USB 3.x** bandwidth. If the device negotiates or is plugged into a link that only runs at **USB 2.x** speeds (a wrong port, a 2.0-only cable, a hub, or an enumeration quirk), the camera silently falls back to reduced resolution/frame rate to fit the smaller pipe.
- The symptom is *degraded* streams, not an outright failure — which is insidious, because perception "works" but at lower quality/rate than intended, quietly capping how good and how timely the road mask can be.

- The fix is physical: ensure the camera is on a true USB 3.x port with an adequate cable so it gets the bandwidth to stream at full spec. This is why the USB 3.x port move is an explicit pending item (Part X) — until it is done, the real camera is running under its potential.

18.4 Where the D455 sits in the pipeline today and tomorrow

Today: it provides the colour frames that the HSV segmenter (and, later, the U-Net) turns into a road mask — it is the real-world stand-in for the Gazebo camera, and it is already feeding the planner against live sensor data (the RealSense-on-Jetson bring-up with A* publishing waypoints on real frames).

Tomorrow: the depth stream is available for (a) **obstacle awareness** — marking cells that are too close/too high as occupied in the grid; (b) **altitude/ground-plane sanity checks** — cross-checking the barometric/EKF altitude against measured ground distance; and (c) potentially aiding the occlusion-recovery logic (Chapter 34) by giving geometric information across a gap. None of this is wired yet, but choosing a depth camera now is what makes it possible later without a hardware change.

19. Texture as a Second Segmentation Feature

19.1 The proposal and its purpose

During a faculty discussion, **Dr. Mishra proposed adding *texture* as a second segmentation feature to reduce U-Net latency**. The idea sits at the intersection of the perception chapters: colour alone (HSV) is fast but blind to structure; a full U-Net is context-aware but heavy. Texture is a **cheap, strong cue** that can be handed to the network so it can be smaller/faster while remaining accurate — reducing the latency that is one of the two things holding the U-Net back (16.5).

19.2 What "texture" means here

Texture is the fine-grained spatial pattern of a surface — the characteristic roughness/regularity of asphalt vs the smoothness of a rooftop vs the busy stipple of grass or foliage. Two regions can share a colour (both grey) yet differ sharply in texture, which is exactly the case HSV cannot resolve (15.5). Texture can be captured with classical operators computed quickly on the CPU/GPU, for example:

- **Local variance / standard deviation** of intensity in a small window (rough surfaces have higher local variance).
- **Gradient magnitude** (edge density) — how much fine edge structure a region has.
- **Local Binary Patterns (LBP)** or Gabor-filter responses — classic texture descriptors encoding local micro-patterns.

Any of these turns the raw image into an extra channel that says, per pixel, "how textured is it here," alongside the colour channels.

19.3 Two ways it reduces U-Net latency

1. **As an extra input channel.** Instead of feeding the U-Net only RGB, feed it RGB **plus** a precomputed texture channel. Because the texture cue is already extracted and highly informative for road-vs-not-road, the network has to *learn less from scratch* and can be **shallower/narrower** (fewer layers/filters) while reaching the same accuracy — and a smaller network is a faster network. You are trading a cheap hand-computed feature for expensive learned capacity.
2. **As a pre-filter / region proposer.** Use texture (and colour) classically to quickly mask out obviously-not-road regions, and run the expensive U-Net only on the uncertain remainder or at lower resolution, cutting the per-frame inference cost.

Either way the principle is the same: **inject a strong prior cheaply so the learned part can be smaller and run faster** — directly attacking the latency hurdle to U-Net deployment.

19.4 How it connects to the rest of the perception story

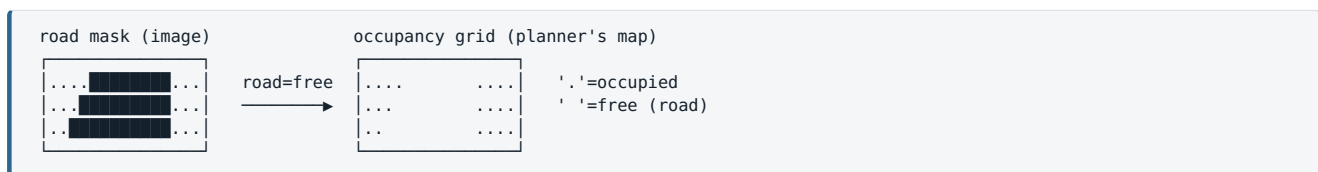
This proposal is the bridge between the classical and learned segmenters. It keeps the *reliability and speed* virtues that made HSV the bootstrap, while giving the U-Net the extra discriminative signal it needs to be both accurate *and* real-time on the Jetson. Combined with TensorRT (speed) and domain-appropriate retraining (correctness), texture-augmentation is a third lever making the semantic segmenter deployable — and it is a concrete, tractable next experiment rather than a vague aspiration.

Part VI — Planning

20. Occupancy Grids and Costmaps

20.1 From a mask to a map the planner can search

The segmenter produces a **binary mask** (road / not-road) the size of the image. The planner cannot search a picture; it searches a **map**. The bridge between them is the **occupancy grid**: a 2-D array of cells where each cell is either **free** (the drone may fly over it) or **occupied** (it may not). The conversion is direct — **road pixels become free cells, everything else becomes occupied cells** — so the grid is essentially the mask reinterpreted as traversable/blocked space. This is the fundamental representational move of the decision plane: perception's "where is road" becomes planning's "where can I go."



20.2 Free vs occupied vs unknown

A binary free/occupied grid is the baseline, but a subtlety that becomes important later (Chapter 34) is a **third state: unknown**. In a strict binary grid, any cell that is not road is an obstacle A* must avoid. But some "not road" cells are genuinely obstacles (a building, the grass verge) while others are merely **not-currently-visible road** — for instance a damaged stretch or a gap where the road momentarily leaves frame. Treating those as hard obstacles makes the planner refuse to path across a gap it should fly straight over. The occlusion-recovery theory's key implementation insight is exactly to mark a gap as **unknown rather than occupied**, so A* is *allowed* to path through it. For the current baseline, though, the grid is free-vs-occupied, and this refinement is a documented next step.

20.3 Costmaps — beyond binary

A **costmap** generalises the occupancy grid: instead of just free/blocked, each cell carries a **cost** to traverse. This lets you express "you *can* fly here but it is undesirable" — e.g. cells near the road edge cost more than cells down the centre, so the planner naturally prefers to stay centred. The A* search (Chapter 21) minimises total path cost, so a costmap steers the *style* of the path, not just its feasibility. The project's baseline uses the grid as free/occupied; costmap shaping (edge-avoidance, centreline preference) is a natural enhancement that the same A* machinery supports without change.

20.4 The occupancy-grid bug that was fixed

A real defect found and fixed early in the project was **incorrect occupancy grid logic** — the mapping from mask to grid was not marking free/occupied cells correctly, which meant A* was searching a map that did not match what the camera saw, producing wrong or impossible paths. Fixing the grid construction so that road → free and non-road → occupied *correctly and consistently* was foundational: a planner is only as good as the map it is handed, and a subtly wrong grid produces confidently wrong plans. This is a recurring theme — **most planner "bugs" are really map bugs**.

21. A* Pathfinding

21.1 The problem A* solves

Given the occupancy grid, a **start cell** (where the drone is) and a **goal cell** (where it wants to go), A* finds the **shortest free-space path** from start to goal, or reports that none exists. It is the classical, optimal, and efficient graph-search algorithm for exactly this kind of grid navigation, which is why it is the project's planner.

21.2 The grid as a graph

A* searches a **graph**. The occupancy grid becomes one by treating each **free cell as a node** and connecting each node to its free **neighbours** (the 4 orthogonal neighbours, or 8 if diagonal moves are allowed). Moving between adjacent cells has a **cost** (1 for orthogonal, $\sqrt{2} \approx 1.414$ for diagonal, or the costmap value). Occupied cells are simply not nodes — they have no edges, so paths cannot go through them. A* then finds the least-cost route through this graph.

21.3 The core idea: $g + h = f$

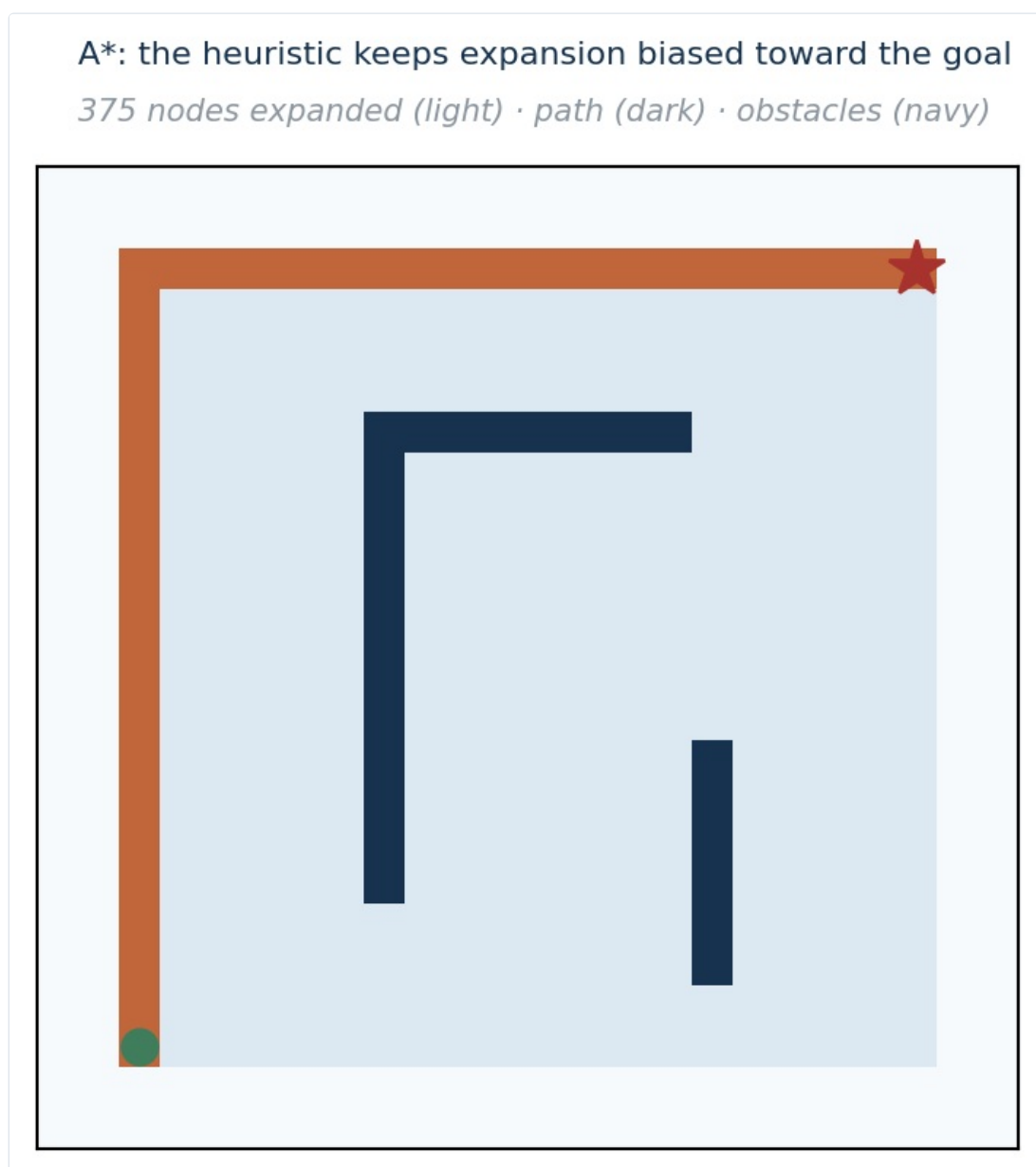


Figure 4. A* expansion over a semantic costmap. The heuristic keeps the frontier biased toward the goal, so far fewer nodes are expanded than a uniform-cost search would touch.

A* is **Dijkstra's algorithm made smart with a heuristic**. For each cell it tracks:

- **$g(n)$** : the known cost of the *best path found so far from the start to n* .
- **$h(n)$** : a **heuristic** estimate of the remaining cost from n to the *goal* — a guess at how far is left.
- **$f(n) = g(n) + h(n)$** : the estimated total cost of a path that goes through n .

A^* always **expands next the frontier cell with the smallest f** — the one that looks most promising because it balances "how far I've already come" (g , the certainty) against "how far I probably have left" (h , the guidance). This is what makes it far faster than blind search: the heuristic pulls the search *toward the goal* instead of spreading uniformly in all directions.

21.4 The heuristic and admissibility

The heuristic h is what steers A^* . For grid navigation it is a distance-to-goal estimate:

- **Manhattan distance** ($|\Delta row| + |\Delta col|$) for 4-connected grids.
- **Euclidean distance** (straight-line) or **Octile distance** for 8-connected grids.

The crucial property is **admissibility**: h must **never overestimate** the true remaining cost. If h never overestimates, A^* is guaranteed to find the **optimal** (shortest) path. If h is exactly zero everywhere, A^* degenerates into Dijkstra (still correct, just slower because it has no guidance). If h *overestimates*, A^* becomes faster but may return a non-optimal path. Straight-line distance is admissible on a grid because you can never do better than a straight line, which is why it is the natural choice.

21.5 The algorithm, concretely

A^* maintains an **open set** (frontier cells discovered but not yet expanded, ordered by f — a priority queue) and a **closed set** (cells already expanded). The loop:

```
open = priority queue ordered by f; push start with g=0, f=h(start)
came_from = {} # to reconstruct the path
g_score[start] = 0

while open not empty:
    current = pop cell with lowest f from open
    if current == goal:
        return reconstruct_path(came_from, current) # done
    for each free neighbour of current:
        tentative_g = g_score[current] + cost(current, neighbour)
        if tentative_g < g_score.get(neighbour, +inf):
            came_from[neighbour] = current
            g_score[neighbour] = tentative_g
            f = tentative_g + h(neighbour, goal)
            push/update neighbour in open with priority f
    return failure # goal unreachable (no free path)
```

reconstruct_path walks **came_from** backwards from goal to start and reverses it. The priority queue (a min-heap) is what makes "expand the lowest- f cell" efficient. The **tentative_g < existing** check is the relaxation step: if we found a cheaper way to a cell, we update it.

A minimal Python core (the shape the planner uses):

```
import heapq

def astar(grid, start, goal, h):
    open_pq = [(h(start, goal), 0, start)] # (f, g, cell)
    came_from = {}
    g_score = {start: 0}
    closed = set()
    while open_pq:
        f, g, current = heapq.heappop(open_pq)
        if current == goal:
            return _reconstruct(came_from, current)
        if current in closed:
            continue
        closed.add(current)
        for nb, step in neighbours(grid, current): # only free cells
            ng = g + step
            if ng < g_score.get(nb, float('inf')):
                g_score[nb] = ng
```

```
came_from[nb] = current
heapq.heappush(open_pq, (ng + h(nb, goal), ng, nb))
return None    # unreachable
```

21.6 Why A* is the right choice here

- **Optimal** with an admissible heuristic — it returns the genuinely shortest road path, not a good-enough guess.
- **Efficient** — the heuristic focuses the search toward the goal, so it explores far fewer cells than Dijkstra/BFS on the same grid, which matters for a real-time loop.
- **Simple and grid-native** — an occupancy grid *is* the natural input to A*; no extra machinery is needed.
- **Handles "no path" cleanly** — if the goal is unreachable (e.g. the grid is all-occupied because the mask was empty), A* terminates and reports failure, which the planner surfaces via its **mask-zero diagnostic** (6.4) so you know *why*.

21.7 The failure modes A* exposes (and how the project handles them)

A* is only as good as its inputs, and two input problems caused real bugs:

- **Empty/wrong grid → no path.** If segmentation returns no road, the grid is all-occupied and A* correctly finds nothing. The **mask-zero diagnostic** distinguishes this "no road detected" case from a genuine planner fault.
- **Goal on an occupied cell → no path.** If the chosen goal cell is not a free (road) cell, A* can never reach it. This is the **goal-on-obstacle bug**, fixed by snapping the goal to the nearest free cell via a spiral search before running A* (Chapter 22). This one is subtle because the *planner* looks broken when really the *goal* was invalid.

22. Automatic Goal Selection

22.1 Why the drone must invent its own goal

Normal path planning is given a destination. Here there is **none** — the drone is not going to a coordinate; it is *following a road* it discovers frame by frame. So the planner must **manufacture a goal every cycle**: a point far along the visible road, in the direction of travel, that A* can head toward. Reach it (or nearly), see new road, pick a new goal, repeat — that is what continuous road-following is. This is the job of `_find_auto_goal`.

22.2 The logic of `_find_auto_goal`

Conceptually, the auto-goal is chosen to pull the drone *forward along the road*. It looks at the road mask and selects a goal cell that is (a) **on the road** (free), (b) as **far forward** as is reliable, and (c) consistent with the drone's heading so the path keeps advancing rather than doubling back. In practice this means examining the free (road) region and picking a target deep into it in the forward direction — the far end of the visible road becomes the goal, so A* plans a path that carries the drone along the road toward the next stretch.

22.3 Ignoring the top 20% of the image

A specific, important rule: `_find_auto_goal` **ignores the top 20% of the image**. The reasoning is geometric and was covered from the camera side (14.2): in a downward frame the far top is the least reliable region — shallowest effective viewing angle, most lens/perspective distortion, most motion blur, and the road there is thinnest and noisiest. Choosing goals from that band would put the goal on the *least trustworthy* pixels, making the path jittery and sometimes wrong. Restricting goal selection to the more reliable lower **80%** yields goals that are both safer (on solid road detections) and more stable (less frame-to-frame jump), which in turn makes the flown path smoother. It is a small rule with an outsized effect on stability.

22.4 The goal-on-obstacle bug and `_nearest_free_goal()`

The subtle failure: the naive forward-goal might land on a **non-road (occupied) cell** — e.g. the road curves and the "straight ahead, far forward" point falls on the verge, or a segmentation hole punches a not-road pixel right where the goal would go. A* then has an **impossible goal** (an occupied cell has no node) and returns *no path*, and the whole planner appears broken when in fact only the goal was invalid.

The fix is `_nearest_free_goal()`, a **radial spiral search**: given the desired-but-possibly-invalid goal, it walks **outward in a spiral** from that cell, checking cells in rings of increasing radius, and returns the **first free (road) cell it finds** — the nearest valid goal to where you wanted to aim. That snapped goal is then handed to A*, which can now reach it. The spiral order guarantees you get the *closest* free cell to the intended goal, so the correction is minimal and the goal stays as close as possible to the true forward target.

```
def nearest_free_goal(grid, desired):
    r, c = desired
    if is_free(grid, r, c):
        return desired
    radius = 1
    while radius < MAX_SEARCH:
        # expand outward in rings
        for dr in range(-radius, radius + 1):
            for dc in range(-radius, radius + 1):
                if max(abs(dr), abs(dc)) != radius:
                    # only the ring's edge
                    continue
                rr, cc = r + dr, c + dc
                if in_bounds(grid, rr, cc) and is_free(grid, rr, cc):
                    return (rr, cc)
            # nearest free cell to the desired goal
        radius += 1
    return None
    # no free cell anywhere nearby
```

This pairs with the same "snap invalid target to nearest valid" idea used for the goal-on-obstacle fix on the Jetson bring-up, and it is a clean example of *defensive planning*: never hand A* a goal it cannot possibly reach.

22.5 Why auto-goal + spiral-snap together make road-following robust

The two mechanisms complement each other: `_find_auto_goal` (restricted to the reliable lower 80%) proposes a sensible forward target, and `_nearest_free_goal` guarantees that target is actually a road cell A* can reach. Between them, the planner *always* has a valid, forward, on-road goal every cycle, so A* *always* has a solvable problem (given any road is visible at all), so the drone *always* gets a sensible next path. That reliability — a valid goal every single frame — is what turns the raw A* search into dependable continuous road-following.

22.6 The edge-hugging failure and its diagnosis

Even with both mechanisms in place, empirical observation of the planner on real masks exposed a further failure mode. The auto-goal on a straight road section consistently landed at coordinates such as `goal = (86, 9)` — at the **left edge of the road strip**, not on its centre. `_find_auto_goal` selects the *topmost reachable road pixel* in the ignored-top-20%-clipped frame, and on a road that runs vertically through the frame, the topmost reachable road pixel is a road-edge pixel: the boundary of the reachable region has road cells at its top-left or top-right corner rather than at its horizontal midpoint. A* then returns a path that hugs that edge, and the drone follows the road **shoulder** rather than its centreline.

This is not a bug in A* — A* correctly returns the shortest path to the requested goal. It is a bug in the *choice of goal*. And for a real drone flying over a real road, hugging the shoulder is a flight-safety issue: a small tracking error carries the aircraft off the road entirely. The correct fix is to pick a target that lies on the road's **centreline**, so any tracking error stays within the road width.

A second class of failure was observed at the same time: the auto-goal could **flip between branches frame-to-frame** on masks with texture-induced noise, producing "goal jitter" where the aircraft was constantly told to head to a slightly different pixel and never committed to a stable direction. Any centreline-based fix has to address this stability problem as well as the edge-hugging one.

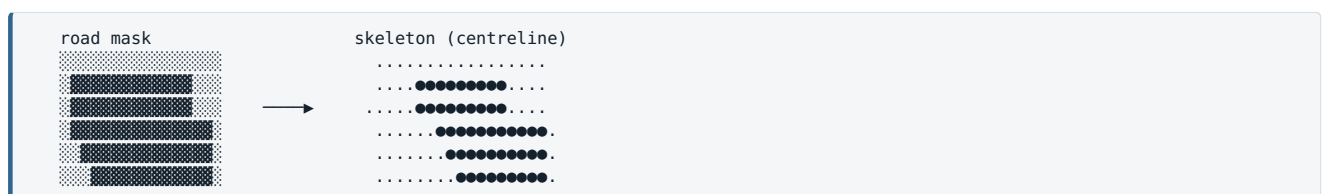
The v7 answer to both is a fundamental change in what the planner *asks A* for*: not a single goal pixel per cycle, but a **stable, ordered chain of waypoints along the road's centreline**, advanced one at a time.

22.7 The centreline via skeletonisation, not distance-transform argmax

There are two natural ways to formalise "centreline" on a binary road mask: the argmax of the Euclidean distance transform (a single pixel on the medial axis) and morphological **skeletonisation** (the entire medial axis as a one-pixel-wide connected curve). v7 uses the latter, for two reasons.

First, the distance-transform argmax returns a *single* pixel per cycle. That pixel is centred and correct, but it does not constitute a stable path: on the next frame the argmax may shift slightly along the road due to segmentation noise, and the drone is once again reacting to a different goal. Second, a single-pixel target throws away all the geometric information the perception has actually delivered — the *shape* of the road ahead — and forces A* to rediscover it.

Skeletonisation instead reduces the road mask to its complete medial axis as a **connected curve**. A road strip of any shape becomes a 1-pixel-wide line running down its middle. Standard implementations (Zhang–Suen thinning, and `cv2.ximgproc.thinning` in opencv-contrib) proceed by iterative boundary removal: repeatedly delete pixels whose removal does not disconnect the shape or change its topology, until only the skeleton remains.



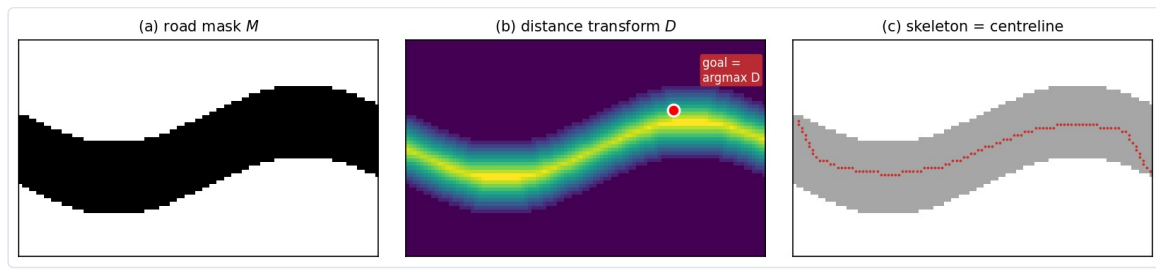


Figure 22. Centreline extraction on a curved road strip. (a) The binary road mask. (b) Its distance transform (shown for comparison; not the algorithm used) — intensity encodes distance to the nearest boundary, and the ridge of maximum distance follows the medial axis. (c) The morphological skeleton actually used, which extracts the full centreline as a one-pixel-wide connected curve. The v7 planner samples an ordered chain of N waypoints along (c), not the argmax of (b).

22.8 Pruning to the main branch: the double-BFS trick

Skeletonising a real road mask does not produce a clean single curve. Segmentation noise produces small holes and finger-like intrusions from the boundary; morphological thinning turns each of these into a **spur** — a short dead-end branch off the main skeleton. If we sample waypoints across the whole skeleton without pruning, some of them land on spurs and the drone is asked to follow a twig rather than the road.

The correct object to keep is the **longest simple path through the skeleton**, which for a road-shaped region is the road itself. Every spur is by construction shorter than the main line it branches off, so a longest-path pruner discards them by definition.

Finding the longest path in a general graph is NP-hard, but the skeleton of a connected shape is a **tree** (or near-tree), and the diameter of a tree can be found by a classic two-BFS trick:

1. From any pixel S in the skeleton, BFS to find the farthest pixel A .
2. From A , BFS to find the farthest pixel B .
3. The path $A \rightarrow B$ is the longest path in the tree.

The correctness argument is short. In step 1, A is the farthest pixel from an arbitrary S . In a tree, the farthest pixel from *any* node is guaranteed to be an endpoint of the diameter — if it were not, one could construct a longer path through S , contradicting A 's farthest-ness. So A is a diameter endpoint. Step 2 finds the pixel farthest from A , which is the *other* diameter endpoint B . The A – B path is the diameter itself.

Two BFS passes over an image-sized skeleton run in tens of milliseconds; the total cost of pruning is negligible next to the network inference that produced the mask.

22.9 The waypoint chain: `build_waypoint_chain`

Having a pruned centreline path, the planner samples a **fixed number of evenly-spaced waypoints** along it, orients them so they run away from the drone, and drops any that lie behind:



The output is a list of pixel coordinates, ordered forward. `_chain_goal()` in the planner then **advances through them one at a time**: the current waypoint is the goal fed to A^* , and the planner only moves to the next waypoint when the aircraft

has reached the current one, when it becomes an obstacle, or when the chain is exhausted and a new one must be built. The chain is *not* rebuilt every frame; it is a small piece of internal state that gives the planner temporal consistency.

This directly addresses both failure modes of §22.6. Every waypoint sits on the pruned skeleton — the road's medial axis — so **edge-hugging is impossible by construction**. And because waypoints are held across frames rather than recomputed per cycle, mask jitter no longer produces goal jitter — the aircraft commits to the current waypoint until it is genuinely reached, and only then advances.

The tunables set for v7 are `N_CHAIN_POINTS = 12`, `REACH_PX = 12` (a waypoint is considered reached at 12 pixels' distance), `MIN_GOAL_DIST_PX = 30`, and horizon-drop `horizon_frac = 0.10` (the top 10% of the frame is masked out before skeletonisation to avoid noisy far-field pixels producing short wobbly branches).

22.10 Costmap protections that make the chain usable

Two v6 costmap bugs would have defeated the chain approach if they had been left in place, and they were fixed alongside the planner change. Both are documented in `core/depth_costmap.py` and reproduced in Appendix I.7.

largest_road_component. A segmenter running on real imagery occasionally produces isolated road fragments — a person standing on the road segments as a separate blob, or a distant road patch across a gap is disconnected from the main road. The planner's goal-picker would happily place a goal in that fragment even though there is no path to it. `largest_road_component` restricts the road mask to its largest connected component before skeletonisation, so a fragment cannot become a waypoint.

Road-protected inflation. v6 dilated the obstacle mask across the whole grid, including over road cells:

```
grid[inflated == 1] = 255          # v6: inflates over road
```

On narrow roads with an obstacle at the edge (a car parked on the shoulder), this dilation "ate inward" across the road until the corridor was pinched shut and A* correctly reported that no path existed. v7 inflates only into non-road cells:

```
grid[(inflated == 1) & (~road_cells)] = 255  # v7: road cells protected
```

A single boolean-and, and the aircraft can now plan through a road that has a parked car on it.

reachable_free_mask. A connected-components pass from the drone's current cell, used to restrict goal-picking to cells A* can actually reach. Together with `largest_road_component`, this makes the "picked a valid-looking pixel that A* can't reach" failure impossible.

22.11 Smart replanning gate

The final v7 planner change is not about the goal but about *when to plan*. v6 ran A* on every 0.2-second timer tick regardless of whether anything had actually moved. v7 keeps track of the last-planned start and last-planned goal pixel, and skips the A* call if both have moved less than `REPLAN_MOVE_PX = 8` cells:

```
if start_moved < REPLAN_MOVE_PX and goal_moved < REPLAN_GOAL_PX:
    return                                # nothing changed enough
```

The effect is that A* runs only when the world has changed enough to justify a new plan, and the aircraft is not told to switch paths every 200 ms on the basis of sub-pixel noise. This is the third of the three v7 stability fixes to the planner: `largest_road_component` gives it a clean input, the waypoint chain gives it a stable target, and the replanning gate keeps it from thrashing.

22.12 Status and relation to A*

At time of writing:

Component	State
Skeletonisation of the road mask (<code>skeleton_goal.py</code>)	Implemented — Zhang–Suen with an opencv-contrib fast path
Longest-branch pruning (<code>main_branch</code>)	Implemented — double-BFS

Waypoint chain (<code>build_waypoint_chain</code>)	Implemented — N = 12 forward-oriented waypoints
<code>_chain_goal()</code> in the planner	Implemented — advances one waypoint at a time
<code>largest_road_component</code> , road-protected inflation, <code>reachable_free_mask</code>	Implemented in <code>core/depth_costmap.py</code>
Smart replanning gate	Implemented — <code>REPLAN_MOVE_PX = REPLAN_GOAL_PX = 8</code>
A* over the semantic costmap (existing)	Retained — the chain provides goals, A* provides the path to each
End-to-end run against real D455 imagery	Pending — awaits the outdoor GPS lock and USB 3.x port swap (Chapters 32, 18)

A* is not replaced. The chain provides the *goals*; A* still provides the *paths* between them. The two systems compose: skeletonisation delivers a semantically-informed, temporally-stable sequence of on-road waypoints, and A* handles the local obstacle avoidance and grid-level path search between them. The combination is what makes the aircraft follow the *centre* of the road, robustly, rather than the topmost reachable pixel on it.

23. Waypoint Management

23.1 From a path to waypoints to motion

A* returns a **path** — an ordered list of grid cells from start to goal. But you do not command a drone cell-by-cell; you give the controller a manageable sequence of **waypoints** (points in the world frame) to fly through. So the path is converted into waypoints, and the control node flies to them one after another: fly toward the current waypoint, and when it is **reached**, advance to the next. Managing that "advance when reached" bookkeeping correctly is what makes the drone progress along the path instead of stalling on the first point or skipping ahead.

23.2 The missing-waypoint-popping bug

A real, instructive bug found early was **missing waypoint popping**: the logic that is supposed to **remove ("pop") a waypoint once it has been reached** and move on to the next was not doing so. The consequence of *not* popping a reached waypoint is that the controller keeps trying to fly to a point it is already at (or has passed) and never advances — the drone gets stuck, or behaves erratically, because its notion of "current target" never updates. The fix was to correctly pop reached waypoints so the target advances through the list. Conceptually:

```
if distance(current_position, waypoints[0]) < REACHED:
    waypoints.pop(0)          # THIS was missing: retire the reached waypoint
# then command flight toward the new waypoints[0]
```

The lesson is that a navigation system needs an explicit, correct notion of "**I have arrived at this target, so switch to the next one,**" and getting that state transition wrong strands the vehicle. It is the discrete-progress counterpart to the continuous control loop.

23.3 The "reached" threshold

You never reach a waypoint *exactly* — position is continuous and estimated, so you declare a waypoint reached when the drone is **within some radius** of it. That radius is the **REACHED** parameter (Chapter 24). Too small and the drone fusses trying to hit an impossible-to-hit exact point and never advances (the same stuck symptom as the popping bug, from a different cause); too large and it cuts corners, advancing before it has really arrived. Tuning REACHED is part of making waypoint progression smooth.

23.4 How waypoint following meets continuous replanning

There is an interplay: A* produces a fresh path (and thus fresh waypoints) *continuously* as new frames arrive, while the controller is *simultaneously* consuming waypoints from the current path. The system must decide how aggressively to replace the old waypoints with new ones — replan too eagerly and the target jitters as each frame nudges the path; replan too lazily and the drone follows a stale plan as the road curves away. This tension is exactly what the **GOAL_HYSTERESIS** and **LOOKAHEAD** parameters (Chapter 24) exist to manage. Waypoint management is therefore not just "pop when reached"; it is the meeting point of discrete progress and continuous replanning.

24. Continuous Replanning and the Tuning Parameters

24.1 Why replan continuously

The drone plans on what it currently sees. As it flies, it sees new road, the old plan goes stale, and the far part of any earlier plan was always a guess about pixels that have since become clearer (or changed). So the planner **replans continuously** — every cycle it takes the latest mask/grid and current pose and computes a fresh path. Continuous replanning is what lets the drone follow a road it is *discovering*, handle curves, and recover from a momentary bad frame (the next frame simply produces a better plan). The cost is instability if done naively (the target jittering frame to frame), which is what the tuning parameters tame.

24.2 The parameters and what each one does

The road-following behaviour is shaped by a handful of tuned constants. Each addresses a specific tension between *responsiveness* (react fast to new information) and *stability* (don't twitch on noise).

LOOKAHEAD — *how far ahead along the road to aim / how far forward to look when picking the goal and the point to steer toward.*

- **Larger LOOKAHEAD:** the drone aims further down the road, giving **smoother, more anticipatory** motion (it "sees the curve coming"), but it can **cut corners** on sharp bends and is less precise about the immediate road.
- **Smaller LOOKAHEAD:** the drone hugs the road closely and turns tightly, but motion is **twitchier** and more reactive to every wiggle in the near mask.
- This is the classic pure-pursuit look-ahead trade-off: it sets how far in front the "carrot" the drone chases sits.

REACHED — *the radius within which a waypoint counts as reached* (introduced in 23.3).

- **Larger REACHED:** waypoints retire easily, so progression is smooth and never stalls, but the drone **cuts corners**, advancing before truly arriving.
- **Smaller REACHED:** the drone tracks each waypoint precisely, but risks **fussing/stalling** if it can't get inside a tight radius (position noise keeps it just outside).
- Tuned to be small enough for accurate following but large enough to guarantee steady advance.

GOAL_HYSTERESIS — *resistance to changing the goal/plan when the new one is only marginally different from the current one.*

- **Hysteresis** means "don't switch unless the change is big enough to be worth it." Without it, tiny frame-to-frame differences in the auto-goal would make the target **flicker**, and the drone would chase a jittering point, wasting motion and looking unstable.
- **Higher GOAL_HYSTERESIS:** the goal/plan is **sticky** — it only changes when the new goal is meaningfully different, giving **stable, committed** motion, at the cost of being slower to adopt a genuinely better plan.
- **Lower GOAL_HYSTERESIS:** the goal updates readily, more responsive but **jittery**.
- This parameter is specifically the antidote to the "replan every frame → target jitters" problem of continuous replanning (24.1) and the waypoint-vs-replan tension (23.4). It lets the system replan continuously *for safety and freshness* while not *acting* on trivial changes.

24.3 How the three parameters interact

They collectively place the system on the **responsiveness ↔ stability** axis:

- **LOOKAHEAD** governs *spatial* smoothness (how far ahead you aim).
- **REACHED** governs *progression* smoothness (when you advance to the next point).
- **GOAL_HYSTERESIS** governs *temporal* smoothness (how readily the plan itself changes).

Tuned together, they produce a drone that follows the road **smoothly and stably** — aiming far enough ahead to anticipate, advancing steadily through waypoints, and holding a committed plan rather than chasing noise — while still replanning continuously underneath so it stays correct as the road unfolds. Getting these three into balance was a core part of making the simulation loop behave, and they carry directly to hardware because they encode *behaviour*, not sim-specific values.

24.4 The other behavioural pieces tuned alongside

Two related settings from the same tuning effort:

- **Altitude-gated takeoff** (10.3): don't start following until safely at altitude — sequencing, so the planner isn't handed a ground-level frame.
- **Continuous replanning** itself (24.1): the loop that makes all the above necessary.

Together with the PX4 parameters (Chapter 11), these constitute the "knobs" that turn a technically-correct pipeline into a *well-behaved* one. The distinction is worth stating: correctness (does A* find the path?) and behaviour (is the flight smooth and stable?) are different problems, and the tuning parameters are entirely about the second.

Part VII — Control

25. The Control Node and Loop Closure

25.1 The control node as the system's keystone

Chapters 6 and 10 introduced the control node's parts; this chapter assembles them into the single most integration-heavy component in the project. The control node is where **four different worlds meet**: ROS (it is a ROS node subscribing to the planner), asyncio (it drives MAVSDK coroutines), MAVLink/PX4 (it commands the flight controller), and the feedback loop (it republishes telemetry so the planner can see where the drone is). If any one of those seams is wrong, the drone does not fly correctly — which is why so much of the project's careful engineering (the asyncio reconciliation of Chapter 7, the offboard contract of Chapter 8, the sim-to-real pose change of Chapter 6) converges here.

25.2 The three concurrent duties, restated as a system

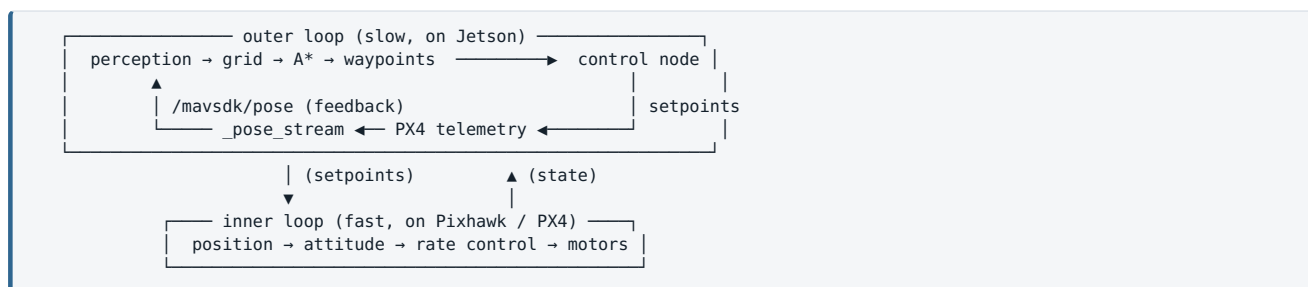
The node runs three cooperating asyncio tasks (7.2), and it is worth seeing them as a *control system* rather than three functions:

1. **Setpoint streaming (the output)**. A steady cadence of position setpoints to PX4 in offboard mode. This is the actuation signal — "fly to here." It must never gap, or PX4 drops offboard (8.6).
2. **Waypoint intake (the reference)**. New waypoints from the planner continuously update *what* "here" is. This is the reference the control system tracks; it changes as the road unfolds.
3. **Pose feedback (the sensor)**. `_pose_stream` reads PX4 telemetry and republishes `/mavsdk/pose`, which the planner consumes to compute the next reference. This is the feedback that closes the loop.

Put together, the control node is a **closed-loop tracker**: reference (waypoints) in, actuation (setpoints) out, feedback (pose) around — with PX4 as the fast inner loop actually stabilising the aircraft, and the perception-planning stack as the slow outer loop deciding where to go.

25.3 Closing the loop: why `/mavsdk/pose` is the linchpin

The planner cannot plan a path from "where I am" to "where I want to go" unless something continuously tells it where it is. In simulation that was `/odom`; on hardware there is no `/odom`. The control node's `_pose_stream` solves this by turning **PX4's own telemetry** (which exists identically in sim and on hardware) into a ROS topic `/mavsdk/pose`, and the planner was rewired to consume it (6.4). This single design decision is what makes the loop close *the same way in both worlds* — the outer loop's "sensor" is always the flight controller's state estimate, surfaced into ROS. It is the concrete embodiment of "the logic transfers; only the edges change" (13.1).



25.4 Startup logging and observability

Startup logging was added to the control node (6.5) so that when it launches you can see it connect, see telemetry begin, and see it enter offboard — rather than watching a silent process and guessing. On a component this integration-dense, observability is not a luxury: when something in the loop stalls, the logs are what tell you *which* of the four seams broke (did it connect? is telemetry flowing? did offboard start? are waypoints arriving?). This is the same philosophy as the perception road-% log and the planner mask-zero log — instrument the boundaries so failures announce themselves.

26. Takeoff, the Flight Sequence, and Safety Integration

26.1 The flight as a state sequence

A flight is not one action but an ordered sequence of states, each of which must complete before the next begins. Laid out:

```
CONNECT → CONFIGURE → ARM → TAKEOFF → REACH ALTITUDE → OFFBOARD/FOLLOW → (LAND / KILL)
```

- **CONNECT.** MAVSDK connects to PX4 (UDP in sim, serial on hardware). The health-check gate was removed so this does not hang on a bench without GPS/home (6.1).
- **CONFIGURE.** The behavioural parameters are in place (`COM_RCL_EXCEPT = 4` , `NAV_DLL_ACT = 0`) so offboard will be tolerated (Chapter 11).
- **ARM.** PX4's prearm checks must pass; on hardware the "Fence requires position" check blocks until GPS lock (Chapter 32).
- **TAKEOFF.** Set altitude (default **20 m**), take off, bounded by a **60 s** `asyncio.wait_for` so a stuck takeoff fails cleanly (6.1).
- **REACH ALTITUDE.** **Altitude-gated** — following does not begin until the drone is safely up (10.3), so the planner is never handed a ground-level frame.
- **OFFBOARD/FOLLOW.** Prime a setpoint, start offboard, then stream setpoints toward planner waypoints while feeding back pose (Chapter 25). Maintain the setpoint cadence for the whole duration.
- **LAND / KILL.** End the flight deliberately, or cut motors immediately with the kill switch if anything goes wrong.

Sequencing these strictly — never starting a stage until the previous truly completed — is what keeps the flight orderly. The bounded waits and the altitude gate are the guards that enforce the ordering even when something stalls.

26.2 The kill switch as the ultimate backstop

The **kill switch** (assigned during bring-up, Chapter 30) is an independent, immediate motor cut-off that works regardless of software state. Its importance is amplified by exactly the relaxations this project made for autonomy: because `COM_RCL_EXCEPT` and `NAV_DLL_ACT` deliberately *disable* some automatic protections (Chapter 11), the human's hard kill becomes the real safety net. On any experimental autonomous flight, the operating discipline is: relaxed software failsafes, but a finger on a reliable kill switch. The kill switch is the reason the parameter relaxations are acceptable during testing.

26.3 Why the timeouts and gates matter as safety, not just tidiness

The `asyncio.wait_for(..., 60)` on takeoff and the altitude gate are usually described as robustness, but they are also safety features. A control coroutine that hangs on a never-completing await (7.3) stops streaming setpoints, which drops offboard, which triggers a failsafe (or, if failsafes are relaxed, leaves the aircraft in an undefined state). Bounding the wait means a stuck operation fails *fast and visibly* instead of silently freezing the loop. The altitude gate means the aircraft is committed to following only once it is in a regime where following makes sense. Both convert "undefined hang" into "defined, recoverable failure" — which on a flying machine is the whole game.

26.4 Landing and ending a flight

Ending cleanly mirrors takeoff: stop the offboard stream, command land (or return), disarm once on the ground. During bring-up and testing, though, the most-used "end" is the deliberate one — bring it down under control, or kill if needed. A tidy end matters because leaving the vehicle armed or in offboard with no setpoints is exactly the condition that trips failsafes or, with them relaxed, leaves motors live.

26.5 The control plane, complete

Stepping back, the control plane is: a strict flight-state sequence (connect → configure → arm → takeoff → altitude → follow → end), executed over a non-blocking asyncio loop that honours PX4's offboard streaming contract, closing the outer loop through `/mavsdk/pose`, guarded by bounded waits and an altitude gate, and backstopped by a hardware kill

switch. Every one of those elements exists because of a specific lesson learned — hanging connects, stuck takeoffs, dropped offboard, sim-only odometry, over-eager failsafes — and together they are what will let the perception-planning stack's decisions become safe real flight.

Part VIII — Hardware Bring-Up

27. The Pixhawk 6C Mini

27.1 What it is and its role

The **Pixhawk 6C Mini** is the **flight controller board** — the physical computer that runs PX4 and is wired to every flight-critical device: the IMUs and barometer (onboard), the magnetometer/compass and GPS (usually external), the RC receiver, the ESCs/motors, the battery monitoring, and the companion computer (the Jetson) over a serial/USB link. It is a compact member of the Pixhawk family, standardised hardware designed to run PX4 (or ArduPilot). In the two-computer architecture (Chapter 1) it is the "spinal cord": fast, deterministic, safety-owning, and *not* where any perception or planning runs.

27.2 What lives on the board vs off it

- **On-board:** the microcontroller running PX4, redundant **IMUs** (accelerometers + gyroscopes), a **barometer** (altitude from air pressure), and the I/O to drive motor outputs.
- **Off-board (wired to it):** the **GPS module** (usually with an integrated **magnetometer/compass**), the **RC receiver**, the **ESCs** to the motors, **battery** voltage/current sensing, and the **companion link** to the Jetson.

Understanding this split is what makes calibration and wiring make sense: you calibrate the *sensors* (some on-board, some external) so EKF2 gets good data, and you wire the *actuators* (ESCs/motors) so control allocation can drive them.

27.3 Its place in the bring-up

The Pixhawk is the bottom of the dependency ladder (4.3), so bring-up starts here: flash the firmware, calibrate the sensors, assign the flight modes and kill switch, wire and map the actuators, and get positioning working. The project has done the first three (firmware, calibration, modes/kill) and has the last two (actuator mapping, GPS lock) remaining. This chapter and the next few walk that exact sequence.

28. Firmware: Flashing PX4 1.17.0

28.1 What flashing is and why the version matters

Flashing firmware means writing the PX4 flight-stack binary onto the Pixhawk's microcontroller. The board did whatever its factory firmware did; flashing puts *your chosen version of PX4* on it. The project **reflashed the Pixhawk 6C Mini to PX4 1.17.0** — pinning a specific, known PX4 version so that the firmware's behaviour, parameter set, and features are exactly the ones the rest of the setup was built and calibrated against. Version pinning matters because parameter names/defaults and offboard/failsafe behaviour can differ between PX4 releases; matching the firmware to the version your procedures and parameters assume avoids a class of "it behaves differently than the docs/notes say" problems.

28.2 How flashing is done (QGroundControl)

Flashing is done through **QGroundControl (QGC)**, the ground-station application, over USB:

1. Connect the Pixhawk to the computer by USB.
2. In QGC's Firmware setup, select PX4 and the desired version (here **1.17.0**).
3. QGC downloads the firmware and writes it to the board, then the board reboots into the new firmware.

After flashing, the board comes up as a "blank" PX4 vehicle that must be told its airframe and then calibrated — which is the next step. QGroundControl is the single tool used across firmware, calibration, mode assignment, and parameter tuning, so it is worth being fluent in it.

28.3 Airframe selection

After flashing, PX4 needs to know the **airframe** — the physical configuration (a quadrotor in the standard "X" layout, in this case) — because that determines the control allocation (which motor does what) and sensible default parameters. Selecting the correct airframe is the prerequisite for the actuator mapping in Chapter 31 to make sense: the airframe defines the *expected* motor layout, and mapping/verification confirms the *physical wiring* matches it.

29. Sensor Calibration

29.1 Role of calibration in the estimator

PX4's state estimator (**EKF2**) fuses the IMU, magnetometer, barometer, and GPS into the attitude and position estimate that *every* controller trusts (8.2). If the raw sensors are biased or misaligned, EKF2's estimate is wrong, and the aircraft will fly badly or dangerously *no matter how good the control code is*. **Calibration** measures and stores each sensor's biases/scale so EKF2 gets corrected data. The project **completed accelerometer, gyroscope, compass, and RC calibration** — this is the foundation that makes the state estimate trustworthy, and it is stored as parameters (11.4). Skipping or botching it is the root of a huge fraction of "my drone won't fly level / drifts / toilet-bowls" problems.

29.2 Accelerometer calibration

The **accelerometer** measures linear acceleration, including the constant 1 g of gravity — which is how the vehicle knows which way is down when static. Calibration corrects the accelerometer's bias and axis scaling by holding the vehicle in a set of known orientations (level, on each side, nose up, nose down, etc.) so PX4 sees gravity along each axis and solves for the corrections. Done through QGC's sensor calibration, following the on-screen orientation prompts. A good accel calibration is what lets the vehicle hold a true level attitude.

29.3 Gyroscope calibration

The **gyroscope** measures angular rate (how fast the vehicle is rotating), which the rate controller — the innermost, fastest control loop — depends on. Gyros have a small **bias** (a non-zero reading even when perfectly still) that must be measured and subtracted, or the vehicle thinks it is slowly rotating when it is not, and drifts. Gyro calibration is simple: hold the vehicle **completely still** while PX4 measures the resting output and stores the bias. It is quick but essential; an uncalibrated gyro bias corrupts attitude estimation and stabilisation.

29.4 Compass / magnetometer calibration

The **magnetometer (compass)** measures the Earth's magnetic field to determine **heading** (yaw), which GPS-based position control needs to know which way the vehicle is pointing. The problem is that the drone's own motors, ESCs, wiring, and battery currents create magnetic fields that distort the reading — **hard-iron and soft-iron distortions**. Compass calibration corrects these by having you **rotate the vehicle through many orientations** (the characteristic "spin it around all axes" dance) while PX4 maps the distortion and computes corrections. A bad compass calibration causes heading errors that make GPS position modes misbehave (drifting, circling), so it is especially important for the outdoor, GPS-based flying the project is heading toward.

29.5 RC calibration

RC calibration maps the **radio transmitter's stick and switch positions** to the values PX4 expects: it records the endpoints and centres of each stick channel (so full-left really reads as full-left, centre as centre) and establishes the channel ranges. This ensures manual control is correct and — critically — that **switches assigned to flight modes and the kill switch** register their positions properly. RC calibration is the prerequisite for the mode/kill-switch assignment in Chapter 30: you cannot reliably assign a switch to "kill" if the RC channels are not calibrated to read that switch cleanly.

29.6 The calibration set as a whole

Together, these four calibrations give EKF2 trustworthy inputs (accel + gyro for attitude, compass for heading) and provide a correctly-mapped radio. With sensing calibrated, the remaining bring-up tasks (actuator mapping, GPS lock) concern actuators and positioning rather than state estimation.

30. Flight Modes and the Kill Switch

30.1 Assigning modes to RC switches

With RC calibrated (29.5), the transmitter's switches were **assigned to flight modes** so a human can select what the vehicle does — e.g. a switch position for a manual/stabilised mode (to check basic flight), for a position/altitude mode, and importantly to hand control to / take it back from **offboard** (so the autonomy can be engaged and, crucially, *overridden*). Having a physical, reliable way to switch modes is what makes autonomy testing safe: if the offboard behaviour misbehaves, the operator flicks back to a manual/stabilised mode and takes over.

30.2 The kill switch — what it is and why it is special

The **kill switch** was assigned as part of this step. It is not a flight mode; it is a **hard, immediate motor cut-off** — flip it and the motors stop *now*, regardless of what the software is doing. Its independence from the normal control path is the point: it works even if the autonomy code, the offboard stream, or the mode logic has gone wrong. For a project flying experimental perception-driven autonomy — with some automatic failsafes deliberately relaxed (`COM_RCL_EXCEPT`, `NAV_DLL_ACT`, Chapter 11) — the kill switch is the **real backstop** (26.2). The operating rule is simple and absolute: never fly the autonomy without a finger ready on the kill.

30.3 Why modes + kill are assigned *before* actuators are wired

The ordering is deliberate: modes and the kill switch were set up as part of getting the flight controller ready, *before* the ESCs and motors are physically wired and mapped (Chapter 31). This ensures that mode selection and motor cut-off are established and tested prior to any motor spin — so when the ESCs are wired and direction verification runs for the first time, the kill switch and mode overrides are already available.

31. Actuator Mapping and ESC Direction (*Remaining*)

31.1 The one remaining flight-controller blocker

This is the **actuator mapping** step: **motor-to-output wiring** and **ESC spin-direction verification**, and it is the **one remaining blocker** on the flight-controller side, deliberately **deferred until the ESCs are physically wired to the Pixhawk outputs**. Everything before it (firmware, calibration, modes, kill) is done; this is the next hardware action once the ESCs are connected. It matters intensely because getting it wrong makes the aircraft uncontrollable or worse the instant it tries to fly.

31.2 What actuator mapping is

Control allocation / mixing (8.2) turns the controller's demand (roll/pitch/yaw torques + thrust) into individual motor commands, assuming a *specific* correspondence between each **PX4 output channel** and each **physical motor position** on the airframe. **Actuator mapping** is establishing that correspondence: output 1 → front-right motor, output 2 → back-left, and so on, matching the airframe's expected layout (28.3). If the wiring does not match the mapping — say output 1 is actually connected to the front-left motor — then when PX4 tries to roll right it commands the wrong motor and the aircraft rolls the wrong way or tumbles. So mapping is verifying/assigning which output drives which motor so that the physical vehicle matches PX4's model of it.

31.3 ESC spin-direction verification

Each motor, driven by its **ESC** (Electronic Speed Controller), must spin in the **correct direction** — a quadrotor's motors alternate clockwise and counter-clockwise, and this pattern is what lets it generate yaw torque and stay balanced. **ESC spin-direction verification** is confirming each motor turns the right way (and its propeller is the matching CW/CCW type). A motor spinning the wrong direction produces thrust the wrong way / breaks the yaw balance, and the aircraft flips immediately on takeoff. If a motor spins backward, the fix is to **reverse it** — either by swapping any two of the three motor-to-ESC wires, or via a DShot/ESC configuration command — and re-verify. This is done with props *off* first (verify direction safely), then confirmed.

31.4 Why it is correctly deferred

The task is **deferred until the ESCs are physically wired to the Pixhawk outputs** for the obvious reason that you cannot map outputs to motors or verify spin direction until the motors are actually connected to the outputs. This is not procrastination; it is correct sequencing — the step is *defined and understood*, waiting only on the physical wiring to exist. When the ESCs are wired, the procedure is: assign/verify the output-to-motor mapping for the airframe, then with props off spin each motor and confirm direction, reversing any that are wrong, then re-verify — all with the kill switch (Chapter 30) ready.

31.5 The order of operations once ESCs are wired

1. Wire each ESC to a Pixhawk output.
2. Confirm airframe = quad-X (control allocation expects a known layout).
3. Map/verify: output N ↔ physical motor position N.
4. Props OFF: spin each motor via test; confirm CW/CCW direction matches layout.
5. Reverse any wrong-direction motor (swap two wires or ESC config); re-verify.
6. Confirm the yaw pattern (alternating CW/CCW) is correct across all four.
7. Only then, with kill switch ready, progress toward a first spin-up with props.

32. GPS, Positioning, and the Geofence Prearm (*Remaining*)

32.1 The second remaining blocker

The other outstanding hardware item is **GPS lock outdoors**, needed to clear the "Fence requires position" prearm warning. This cannot be solved indoors — it is inherently an outdoor task — and it is what stands between the calibrated, configured aircraft and being *allowed to arm* for a positioned flight.

32.2 Why GPS matters for this project

The autonomy is heading toward **outdoor, GPS-referenced flight**: PX4's position modes and the position feedback the planner uses (`/mavsdm/pose`, sourced from EKF2's position estimate) rely on a **position solution**, which outdoors comes primarily from **GPS** fused with the IMU/baro/compass in EKF2. Without a GPS fix, EKF2 has no absolute position, so position-dependent features are unavailable and certain safety checks refuse to pass. In simulation this never came up because SITL's "GPS" always has a fix (13.3); on real hardware, obtaining a genuine outdoor lock is a required step.

32.3 What "GPS lock" means

GPS lock (a fix) means the GPS module has acquired enough satellites with good geometry to compute a reliable position, and EKF2 has converged on a position estimate it trusts. Getting a good lock requires being **outdoors with a clear view of the sky** (buildings and roofs block/reflect signals), allowing time for the module to acquire satellites, and having the GPS/compass mounted and calibrated (Chapter 29) so heading and position are consistent. Once locked, the "no position" condition clears.

32.4 The geofence and the prearm check

A **geofence** is a configured boundary the vehicle must stay within; PX4 will refuse to arm (or will act) to enforce it. The "Fence requires position" prearm warning means: *a geofence is configured, but the vehicle has no valid position estimate to know whether it is inside the fence, so it will not arm*. It is PX4 correctly refusing to arm a position-limited vehicle that does not know where it is. The warning is not a bug — it is the safety chain (8.4) doing its job. It clears once a GPS lock gives EKF2 a valid position, at which point PX4 can confirm the vehicle is inside the fence and allow arming.

32.5 How this resolves

The resolution is procedural, not a code change: take the aircraft **outdoors, obtain a solid GPS lock**, confirm EKF2 reports a valid position, and the "Fence requires position" prearm clears, permitting arming for a positioned flight. (If, for a specific indoor/bench test, one needed to arm without position, one would reconfigure the fence/position requirement — but for the intended outdoor autonomy, getting the real GPS lock is the right path.) This item, like actuator mapping, is *understood and defined*, awaiting the physical circumstance (being outdoors) to complete.

33. The Jetson Orin Nano Platform

33.1 The companion computer, set up

The **NVIDIA Jetson Orin Nano 8GB** is the companion computer running the entire ROS 2 pipeline and all the machine learning — the "brain" of the two-computer architecture (Chapter 1). It was brought up **pre-flashed** with **JetPack 6.x**, which provides **Ubuntu 22.04.5 LTS** with **L4T (Linux for Tegra) R36.3.0** — the NVIDIA-tuned Linux for Jetson hardware. This exact software base is why ROS 2 **Humble** is the right ROS version (it targets Ubuntu 22.04) and why everything downstream (drivers, CUDA, TensorRT) lines up. Reading that stack top to bottom: JetPack 6.x → Ubuntu 22.04.5 → L4T R36.3.0 → CUDA/TensorRT + ROS 2 Humble → the pipeline.

33.2 Why the Orin Nano specifically

Three properties make it the choice (4.2): it has a **CUDA-capable GPU** (for running the U-Net/TensorRT inference), it runs **full Ubuntu + ROS 2** (so the whole ecosystem — the RealSense driver, rclpy, the nodes — just works), and it fits a drone's **size/weight/power** budget. The **8 GB memory** is the binding constraint that feeds back into model-size and precision decisions (FP16/INT8 via TensorRT, Chapter 17) — you must fit the network *and* the rest of the pipeline in that budget.

33.3 Installing and building the software

Bring-up on the Jetson involved:

- **ROS 2 Humble installed natively** on the Ubuntu 22.04 base (not in a container) — the middleware the whole pipeline runs on.
- **The repository cloned and built successfully on ARM64.** The Jetson is an **ARM64/aarch64** machine, not x86, so building the workspace there confirms the code compiles and runs on the target architecture — an important checkpoint, since ARM builds can surface dependency/wheel issues that never appear on an x86 desktop. `colcon build` completing on ARM64 means the package is genuinely deployable on the drone's actual computer.

33.4 The RealSense integration on the Jetson

The **RealSense D455** was **integrated** on the Jetson: its ROS 2 driver installed and its **topics confirmed live**, so the real camera publishes frames into the pipeline. A concrete hiccup solved along the way was **GPG key issues** during installation — package repositories are signed with GPG keys, and if the key is missing/expired the package manager refuses the repo; resolving the key (importing the correct one) let the RealSense packages install. With the driver working and topics live, the Jetson is running the perception front-end on **real sensor data**, which is the state the project is in: **A* publishing waypoints against live RealSense frames**. The one open camera item is the **USB 3.x bandwidth** move (Chapter 18) so the D455 streams at full spec rather than the USB 2.1 cap.

33.5 What running on the Jetson proves

Having the pipeline built and running on the Orin Nano with the real camera is the hardware-side counterpart to the sim validation: it proves the *software* stack is deployable on the *actual* onboard computer with the *actual* sensor. What remains for a full flight is on the *flight-controller* side (actuator mapping, GPS lock) and the *perception-quality* side (USB 3.x bandwidth; U-Net domain adaptation + TensorRT). The Jetson platform itself — OS, ROS, build, RealSense — is up.

Part IX — Electrical Integration and Mechanical Design

The chapters up to this point described a system that exists as software running on two computers. This part describes how those two computers, a battery, four motors, and a camera become one physical aircraft. It is the least glamorous and most unforgiving part of the project: a wiring error here does not throw an exception, it destroys hardware.

34. Power Architecture

34.1 The single energy source

Everything on the aircraft ultimately draws from one **6S 12000 mAh lithium-polymer battery**. "6S" means six cells in series, giving a nominal 22.2 V (and roughly 25.2 V fully charged, sagging toward 19.8 V at the low end of safe discharge). "12000 mAh" is the capacity — a large pack, chosen because the Jetson is a meaningful continuous load on top of the motors, and endurance matters for a road-following mission that must actually follow a road for some distance.

The important consequence of a 6S pack is that **its voltage is wrong for almost everything on board**. Motors want it raw through the ESCs. The Pixhawk wants a regulated ~ 5 V. The Jetson Orin Nano wants 19 V at its barrel jack. Nothing can simply be connected to the battery.

34.2 Three independent rails

The power design therefore splits into three paths from one pack:

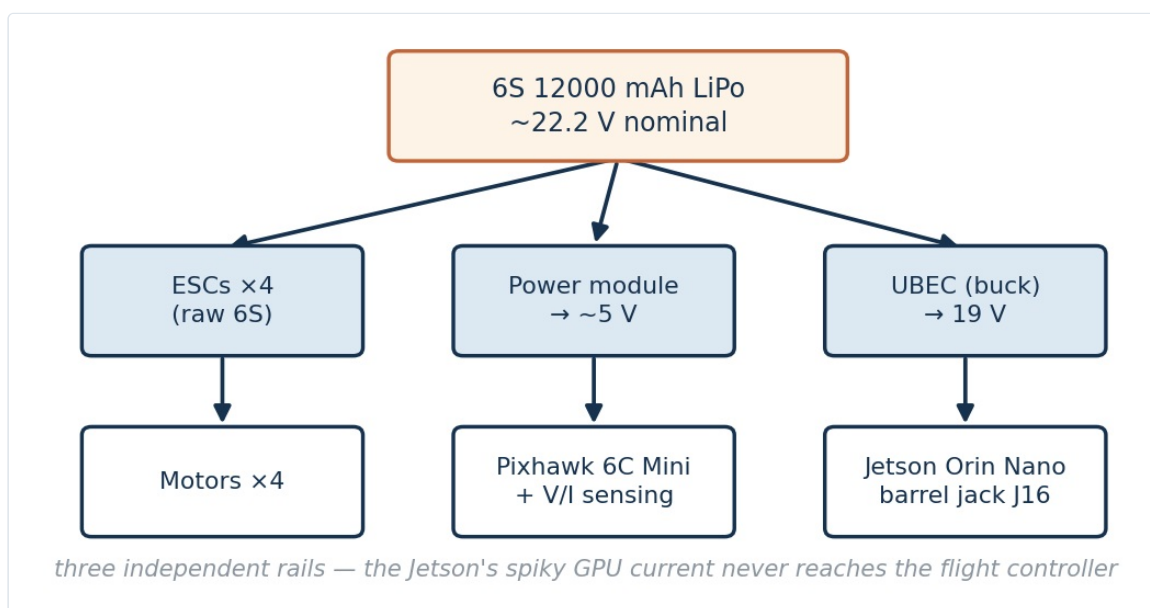


Figure 5. One 6S 12000 mAh pack, three independent rails. The Jetson's dedicated UBEC keeps spiky GPU current off the flight controller's supply.

- **Motors** are fed raw pack voltage through their **ESCs**, which are the devices that actually modulate power to the motor windings. This is the high-current path; it is where nearly all the energy goes.
- **The Pixhawk** is fed through a **power module**, which both regulates down to the ~ 5 V the board needs and provides the **battery voltage and current sensing** that PX4 uses for its battery failsafe and for reporting state of charge.
- **The Jetson** is fed through a **UBEC** — a DC-DC buck (step-down) converter — configured to output a regulated **19 V**, delivered to the Jetson developer kit's **barrel jack (J16)**. 19 V is what the Orin Nano developer kit expects at that input.

34.3 Why the Jetson gets its own converter

It would be simpler to run the Jetson from the Pixhawk's 5 V rail. It would also be a mistake. The Jetson under GPU load draws far more current than a flight-controller power module is designed to source, and — more dangerously — its load is *spiky*. Neural-network inference causes sudden, large current draws. If the Jetson shared a rail with the Pixhawk, those spikes would appear as voltage dips on the flight controller's supply, and a flight controller that browns out mid-flight is a falling object.

Giving the Jetson **its own dedicated UBEC** from the battery isolates the flight-critical rail from the compute rail. The Pixhawk's supply is then insensitive to whatever the neural network is doing. This is the electrical expression of the same

architectural principle that separates the two computers in the first place (Chapter 1): *flight safety must not depend on the behaviour of the perception stack*.

34.4 The consequence for the serial link

Because the two boards now have **independent power supplies**, the serial cable between them must **not** carry power. This is the reason for a wiring detail that would otherwise look like an omission, and it is the subject of the next chapter.

35. The Pixhawk ↔ Jetson Serial Link

35.1 What the link is for

The Jetson must send MAVLink to the Pixhawk (offboard setpoints, arm/takeoff/land commands) and receive MAVLink from it (attitude, position, flight mode, battery). During bench work this ran over **USB** (`serial:///dev/ttyACM0:57600`), which is convenient because it is one cable and it also powers nothing critical. For flight, the link moves to a **dedicated serial (UART) connection**, because USB connectors are mechanically unreliable under vibration and because the flight-ready wiring should not depend on a USB host stack coming up in the right order.

The flight link is Pixhawk **TELEM2** → Jetson **UART1**.

35.2 The pinout, and the one wire that is deliberately absent

Serial is a three-wire protocol in practice: transmit, receive, and a shared ground reference. The connection is:

Signal	Pixhawk TELEM2 pin	→	Jetson 40-pin header (J12) pin	Jetson signal
TX	2	→	10	RXD
RX	3	→	8	TXD
GND	6	→	GND	GND
VCC	1	✗	— not connected —	

TX crosses to RX. The Pixhawk's transmit line lands on the Jetson's receive pin and vice versa. Wiring TX→TX leaves two transmitters and two receivers with no established data path.

VCC (pin 1) is deliberately left unconnected. TELEM2's pin 1 carries power for peripherals such as telemetry radios; the Jetson is independently powered from its own 19 V UBEC (Chapter 34). Connecting VCC would tie two independently-regulated supplies together, back-feeding current between two systems that disagree on the target voltage. Ground must be shared so signal voltages have a common reference; power must not be.

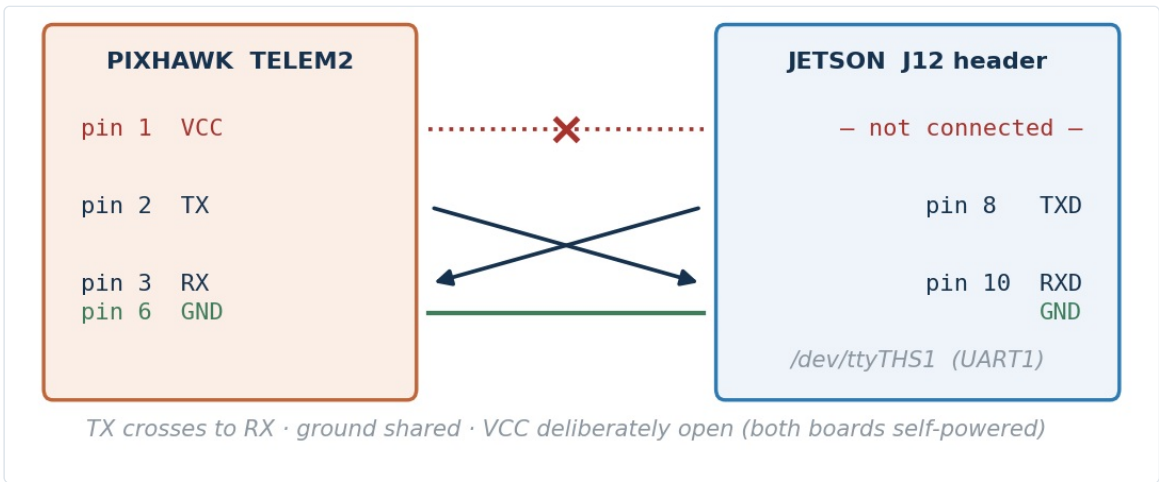


Figure 6. Pixhawk TELEM2 → Jetson UART1 (/dev/ttyTHS1). TX crosses to RX, ground is shared as a common reference, and VCC is deliberately left unconnected because both boards are independently powered.

35.3 Where UART1 actually is on the board

On the **Jetson Orin Nano developer kit** (the official carrier board), **UART1 is exposed on pins 8 and 10 of the 40-pin GPIO header (J12)**, and appears to Linux as `/dev/ttyTHS1`. That device path is what MAVSDK is pointed at for flight, in place of the bench-time `/dev/ttyACM0`.

A practical note recorded because it cost time: **the TXD/RXD labels are silkscreened on the underside of the carrier board**, not the top. Looking for them on the visible face and not finding them is not evidence that you have the wrong header. Separately, the **J14 header is the button/debug header**, not a second UART — it is easy to mistake for one, and doing so wastes an afternoon.

35.4 Bench link vs flight link

	Bench	Flight
Physical	USB cable	TELEM2 → UART1, 3 wires
Device	<code>/dev/ttyACM0</code>	<code>/dev/ttyTHS1</code>
MAVSDK address	<code>serial:///dev/ttyACM0:57600</code>	<code>serial:///dev/ttyTHS1:<baud></code>
Powers the Pixhawk?	Yes (from USB)	No (own power module)

The bench configuration is verified: **MAVSDK 2.8.4 connects to the real Pixhawk over `/dev/ttyACM0`** (Chapter 45). The flight configuration is wired and awaits the same verification once the aircraft is assembled.

36. ESCs, DShot, and Motor Outputs

36.1 The ESCs on this aircraft

The aircraft uses **Simplify 40A ESCs** — four of them, one per motor, rated to 40 A continuous. Critically for the configuration choices below, **these ESCs are DShot-capable**.

36.2 PWM versus DShot

The traditional way a flight controller talks to an ESC is **PWM**: an analogue-in-spirit signal where the *width* of a pulse encodes the throttle demand. Because it is analogue in spirit, both ends must agree on what pulse width means "zero" and what means "full" — and they do not agree by default. Hence **ESC calibration**: a ritual in which you tell each ESC, "this pulse width is minimum, this one is maximum," so it maps the range correctly. It is fiddly, it must be repeated per ESC, and getting it wrong means one motor spins up before the others.

DShot is a **digital** protocol. The throttle command is transmitted as a *number* — a bit pattern with a checksum — not as a pulse duration. There is no analogue range to agree on, because 1046 means 1046 on both ends.

Consequences:

- **ESC calibration is not required.** DShot sends digital values rather than pulse widths, so no endpoint range needs to be taught. This project runs DShot600 (600 kbit/s), and the calibration step is not performed.
- **It is noise-resistant.** The checksum means a corrupted command is rejected rather than acted upon, whereas a PWM pulse distorted by electrical noise is silently interpreted as a different throttle value.
- **Direction can be set in software.** DShot defines special commands, including *reverse motor direction*, which can be sent to an ESC to flip its spin direction without rewiring. This is a substantial convenience for the spin-direction verification of Chapter 31 — a motor turning the wrong way can be fixed with a command rather than by swapping two of the three motor wires.

36.3 What has been verified, and what has not

These two facts are easy to conflate and must be kept apart:

Verified. During the QGroundControl bench test (Chapter 38), **individual motors were driven directly from the QGC Actuators panel sliders** and responded. This proves the Pixhawk's outputs, the ESCs, and the motors themselves are functional and correctly powered. It isolates any arming problem to the arm/mode/output-permission chain rather than to the ESCs or the wiring — a genuinely useful piece of information obtained cheaply.

Not verified. The **mapping** of PX4 output channels to physical motor positions, and each motor's **spin direction**, are still open (Chapter 31). Driving a slider labelled "Motor 1" and seeing *a* motor turn does not establish that PX4's "Motor 1" is the front-right motor of a quad-X, nor that it turns clockwise. Control allocation depends on both. This is the remaining ESC-side task, and it is deferred until the ESCs are wired into their final positions on the airframe.

36.4 The order once ESCs are in place

1. Wire each ESC to its Pixhawk output, in the airframe's motor order.
2. Confirm the airframe is set to quad-X (control allocation assumes a layout).
3. PROPS OFF. Spin each output in turn; identify which physical motor moves.
4. PROPS OFF. Observe each motor's direction; compare to the quad-X CW/CCW pattern.
5. Reverse any wrong-direction motor — with DShot, by command; otherwise swap two wires.
6. Re-verify the full alternating pattern.
7. Only then consider props, with the kill switch in hand.

37. The Airframe Mount Plate

37.1 The mechanical problem

The Jetson, the Pixhawk, and the RealSense must be held rigidly, in known orientations, on a frame that vibrates. The Pixhawk in particular must be mounted with a defined heading and level attitude, because its IMU measures the aircraft's motion and its compass measures heading — a board mounted crooked produces a state estimate that is crooked in exactly the same way. The camera must point at nadir (straight down) and stay there, because the entire perception geometry (Chapter 14) assumes it does.

The solution is a **parametric mount plate**, designed in **OpenSCAD** — a CAD system where a model is *code* rather than a drawing. Parametric design is the right choice here because almost every dimension is provisional: hole spacings get re-measured, components get repositioned, and a plate defined by named variables can be regenerated rather than redrawn.

37.2 The plate geometry

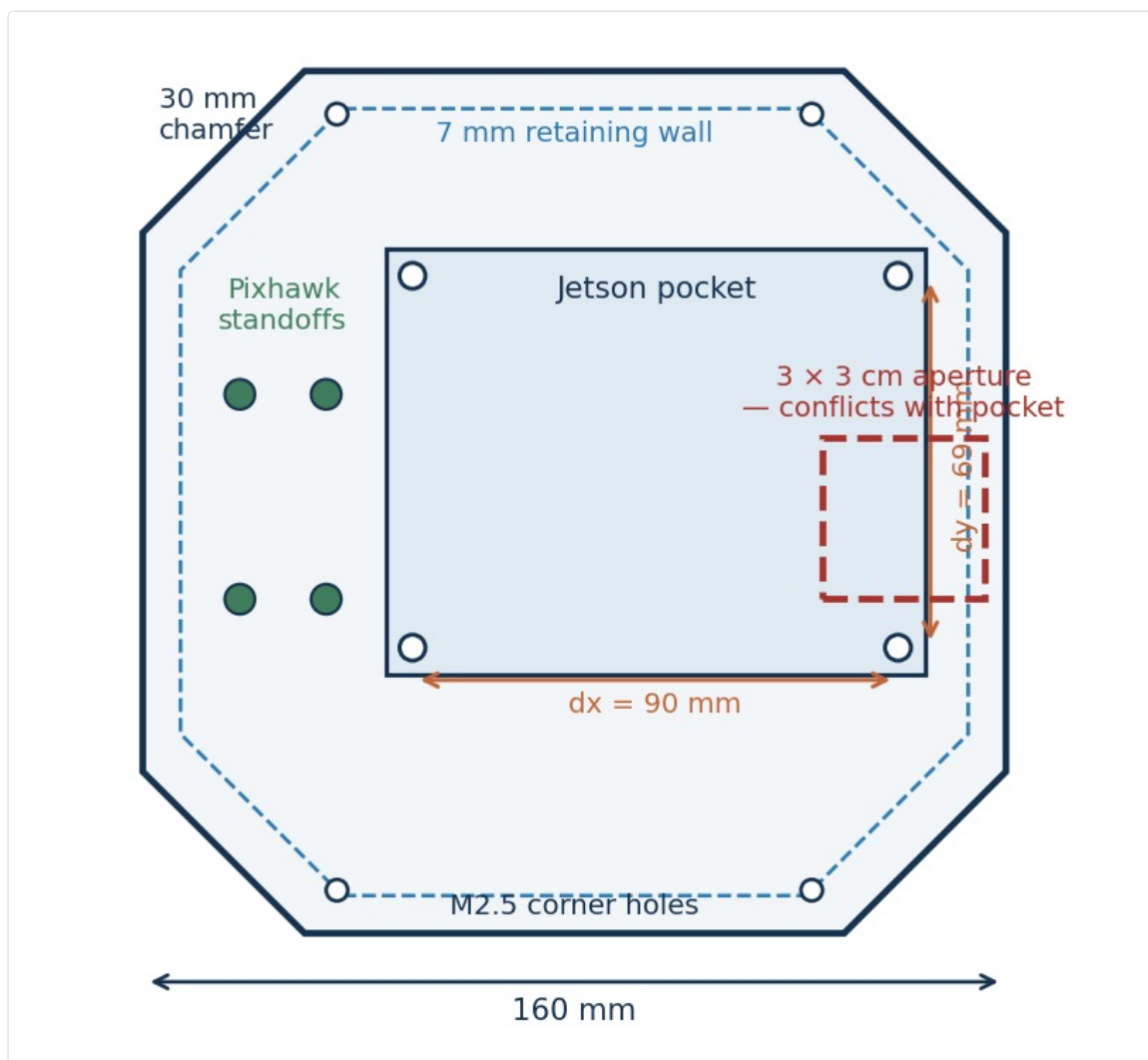


Figure 7. The parametric OpenSCAD mount plate: 160 × 160 mm octagonal, 30 mm chamfers, 7 mm retaining wall, Jetson hole grid dx = 90 / dy = 69 mm. The dashed red square is the unresolved 3 × 3 cm aperture conflicting with the Jetson pocket.

- **160 × 160 mm octagonal** — a square with the corners cut, which is what an octagon is here. The chamfers remove material and mass where it does nothing structurally, and clear the arms of the frame.
- **30 mm corner chamfers** — the size of those cuts.
- **7 mm retaining wall** around the perimeter — a raised lip that stops components sliding off laterally and stiffens the plate against bending. A flat plate of this span would flex; a lip turns it into a shallow tray, which is far stiffer for the same mass.

- **M2.5 corner screw holes** — the fasteners that attach the plate to the airframe.
- **Jetson mounting hole grid:** `jetson_hole_dx = 90 mm`, `jetson_hole_dy = 69 mm` — the rectangular spacing of the Orin Nano developer kit's mounting holes. These are the numbers the plate must match exactly.
- **Pixhawk standoffs repositioned to the left margin** — standoffs raise the flight controller off the plate (for connector clearance and to decouple it slightly from plate-borne vibration). They were moved to the left margin to resolve interference with the Jetson's footprint.

37.3 The unresolved geometric conflict (*Remaining*)

There is one known, open problem in the current design: **a 3 × 3 cm side hole conflicts geometrically with the Jetson pocket**. The aperture — intended for cable routing or airflow through the side of the plate — intersects the volume the Jetson occupies. The two cannot both exist as currently specified.

This is a **CAD revision, not a redesign**. The resolutions available, in rough order of preference:

1. **Move the aperture** along the side wall to a region clear of the Jetson footprint.
2. **Shrink the aperture** to whatever clearance actually remains, if it only needs to pass a cable bundle.
3. **Shift the Jetson pocket** — feasible, since the plate is parametric, but it cascades into the `dx/dy` hole positions relative to the plate origin and the Pixhawk standoffs that were already moved once for clearance.
4. **Split the aperture** into two smaller openings straddling the conflict.

Because the model is parametric, testing these is a matter of changing a variable and re-rendering, which is precisely the payoff of having written the plate as code. It is listed as remaining because a mount plate that cannot be printed is a hard blocker on assembly, and assembly gates everything physical.

37.4 Why this matters more than it looks

It is tempting to treat the plate as an afterthought — a piece of plastic. It is not. The plate determines the Pixhawk's alignment (hence the accuracy of every attitude and heading estimate), the camera's nadir orientation (hence the validity of the projection from image to ground), and the vibration environment the IMU sees (hence the noise floor of the state estimate). A poor mount produces symptoms — drift, toilet-bowling, noisy altitude — that will be misdiagnosed for days as software problems.

38. Bench Validation in QGroundControl

A bench test was conducted with the **real Pixhawk**, **no Jetson**, and **no propellers**. Three distinct problems surfaced and were resolved or characterised.

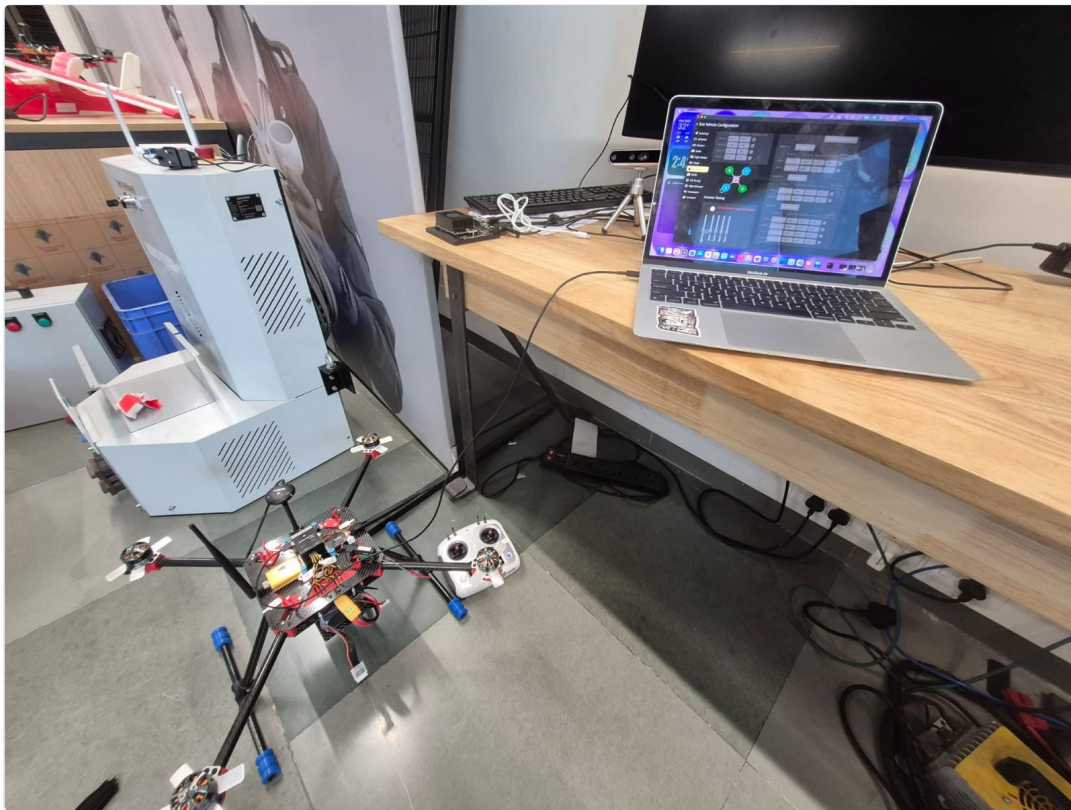


Figure 38.1. The bench configuration used for the QGroundControl validation session — Jetson Orin Nano developer kit (left), Pixhawk 6C Mini (bottom-centre), RealSense D455 on a tripod (right), and the printed pre-flight checklist (Appendix L). No propellers, no ESCs wired; motors were driven directly from the QGC Actuators panel to isolate any arming issue from the ESC/wiring chain.

38.1 Compass inconsistency

Symptom. A prearm error reporting a **compass inconsistency: roughly a 128° mismatch between the internal and external magnetometers**.

What it means. The Pixhawk 6C Mini has an internal compass; the GPS module carries an external one. PX4 cross-checks them: if two compasses disagree about where north is, at least one is wrong, and PX4 refuses to arm rather than fly on a heading it cannot trust. A 128° disagreement is enormous — not a calibration nuance but a gross discrepancy, consistent with the internal compass sitting inside a metal-and-current-filled airframe while the external one is out on a mast, or with an orientation mismatch between the two.

What was done. The check was **bypassed by setting** `COM_ARM_MAG_ANG = -1`, which disables the maximum-allowed-angle prearm check between compasses.

The honest caveat. This is a *bench* expedient, and it must be recorded as one. Disabling the consistency check does not make the compasses agree; it makes PX4 stop objecting. For indoor bench work with no GPS and no flight, that is acceptable — heading accuracy is irrelevant when nothing moves. **Before any outdoor positioned flight, the underlying disagreement must be resolved** (re-calibrate the compasses in situ, confirm the external compass's orientation setting, and consider disabling the internal compass entirely in favour of the external one, which is standard practice on airframes with high current near the flight controller). Flying a GPS position mode on inconsistent compasses produces exactly the drifting, circling behaviour that is so often misattributed to tuning.

38.2 GPS prearm blocks

Symptom. Prearm refusals related to the absence of a GPS fix — expected indoors.

What was found. `COM_ARM_WO_GPS` was **already set to "warning only"**, meaning PX4 was configured to permit arming without GPS and merely complain. So this parameter was not the obstacle. The real obstacle was the flight mode, which is the next item — and this is a good example of a check whose *message* points at GPS while the *cause* is elsewhere.

38.3 The flight-mode trap

Symptom. The aircraft would not arm, and the flight mode appeared stuck in **Position** — a mode which *requires* a valid GPS position and therefore cannot possibly work indoors.

Diagnosis. The three-position mode switch had its slots assigned as: **Flight Mode 1 = Position, Flight Mode 4 = Altitude, one position = Return. Stabilized was never assigned to any reachable switch position.** Consequently there was no physical way to select the one mode that needs neither GPS nor altitude sensing to arm and spin motors. The vehicle was not "stuck" in Position; Position was simply the only thing the switch could ask for in that slot, and Position is unusable without GPS.

Fix. Replace Flight Mode 1 (Position) with Stabilized, so the physical switch can reliably select a mode that works indoors and on the bench.

This is a configuration bug that presented as a hardware fault. The vehicle correctly refused Position mode without GPS; the mode switch simply offered no alternative that could arm indoors.

38.4 What the bench test established

- **Motors work.** Driving the **QGC Actuators panel sliders** directly spun individual motors. This bypasses the arming and mode logic entirely, and thereby isolates the fault: the ESCs, motor wiring, and Pixhawk outputs are healthy, and any arming failure lies in the arm/mode/permission chain above them.
- **ESCs are already calibrated** — and per Chapter 36, with DShot600 this is moot anyway.
- **The Pixhawk 6C Mini has no physical safety button.** Some Pixhawk variants require a physical button press before arming is permitted; this one does not, so its absence is not a fault to hunt for.

38.5 The parameters this chapter introduced

Parameter	Value	Meaning	Status
<code>COM_ARM_MAG_ANG</code>	-1	Disable the compass-inconsistency prearm check	Bench expedient — revisit before outdoor flight
<code>COM_ARM_WO_GPS</code>	warning only	Permit arming without a GPS fix	Already set; not the blocker
Flight Mode 1	Position → Stabilized	Make an indoor-usable mode reachable on the switch	Fix identified

Part X — The Segmentation Rewrite

Part V described the perception stack as it stood: HSV thresholding in simulation, with a U-Net trained on AeroScapes **deferred** because of a domain gap. That description is now obsolete, and the reason it is obsolete is the most instructive episode in the project. The U-Net was not merely suffering a domain gap. It was broken in three independent ways at once, and only one of them was a modelling problem.

This part supersedes Chapter 16.

39. The Failure: Near-Empty Masks

39.1 The symptom

The trained plain U-Net, run on validation imagery, produced **near-empty road masks**. Not *wrong* masks — not roads misplaced or boundaries blurred — but masks in which almost no pixel was labelled road at all. Downstream, this is fatal: an empty mask means an all-occupied occupancy grid, which means A* correctly reports that no path exists (Appendix D.6), which means the drone does not move.

39.2 Why the obvious explanation was wrong

The obvious explanation — and the one initially accepted — was the **domain gap**: the network trained on AeroScapes, a real aerial dataset, and was being asked to segment Gazebo's rendered imagery, which differs in colour, texture, lighting, and shadow. Domain gap is real (Chapter 13), it does degrade accuracy, and it was the honest reason the U-Net was set aside in favour of HSV.

But domain gap does not usually produce a mask that is *empty*. It produces a mask that is *noisy*, or *confidently wrong*, or *right in the middle and wrong at the edges*. A network that has learned what road looks like and is then shown slightly unfamiliar road should still fire somewhere. Near-total silence is a different signature — it says the network never learned to predict road at all, or that the thing being measured is not road.

That distinction was the thread worth pulling. Pulling it revealed three separate faults, stacked.

40. Three Stacked Faults

The near-empty masks were caused by three independent bugs, each sufficient to cripple the result on its own. They are worth separating precisely because their coexistence made diagnosis hard: fixing any one alone would have produced little visible improvement, which is exactly the condition under which an engineer wrongly concludes that the fix was not the problem.

40.1 Fault 1 — Only the first 30 masks were scanned

The data-inspection step that determined class statistics and sanity-checked the labels **examined only the first 30 mask files**. AeroScapes contains 3269 images, and its files are not shuffled with respect to scene content: a contiguous block of 30 masks drawn from the front of the set is a handful of consecutive frames from a small number of the 141 source video sequences. Road may be scarce or absent across all of them.

The effect is that every conclusion drawn from that scan — which classes exist, how frequent road is, whether the label indices mean what they are assumed to mean — was drawn from an unrepresentative sliver of the dataset. **This is a sampling bug, not a machine-learning bug**, and it is the one that made the other two invisible: with a corrupted view of the label statistics, there was no signal to indicate that anything else was wrong.

40.2 Fault 2 — The wrong class index

This is the decisive fault. The code had `ROAD_CLASS = 1`. In AeroScapes, **class index 1 is `person`**. **Road is index 10.**

So the network was trained, and its performance measured, against the wrong target. Every "road" pixel it was asked to find was in fact a *person* pixel. Persons in aerial imagery at 5–50 m altitude occupy a vanishingly small fraction of pixels. A network trained to segment that class, and then evaluated as though its output were a road mask, produces exactly what was observed: **a near-empty mask**.

The masks were not empty because the network failed. They were empty because the network **succeeded** at a task nobody wanted — finding the very few people in an aerial scene — and its output was then interpreted as roads.

This is the single most important debugging lesson in the entire project, and it is not about neural networks. **An off-by-nine error in a constant produced a failure that looked like a fundamental limitation of deep learning**. The temptation, faced with an empty mask, is to reach for architecture, loss functions, and training schedules. The actual fault was a number.

40.3 Fault 3 — Plain cross-entropy neglects a minority class

The third fault is a genuine machine-learning problem, and it would have bitten even with the correct class index.

The loss was plain **cross-entropy**, averaged over pixels. In an aerial frame, road occupies a **minority** of pixels — vegetation, construction, and background dominate. Under pixel-averaged cross-entropy, the cheapest way for a network to reduce loss is to predict the majority classes everywhere and simply never predict the minority one. Getting 90% of pixels right by ignoring road entirely scores better, on that loss, than a genuine attempt at road that makes errors elsewhere.

So the loss function itself **actively discouraged** the network from predicting road. The gradient pointed away from the class of interest. This is the classic class-imbalance failure, and its symptom is — again — an empty mask for the rare class.

40.4 Why three faults is worse than three times one fault

Each fault independently suppresses road predictions. Stacked, they are mutually camouflaging:

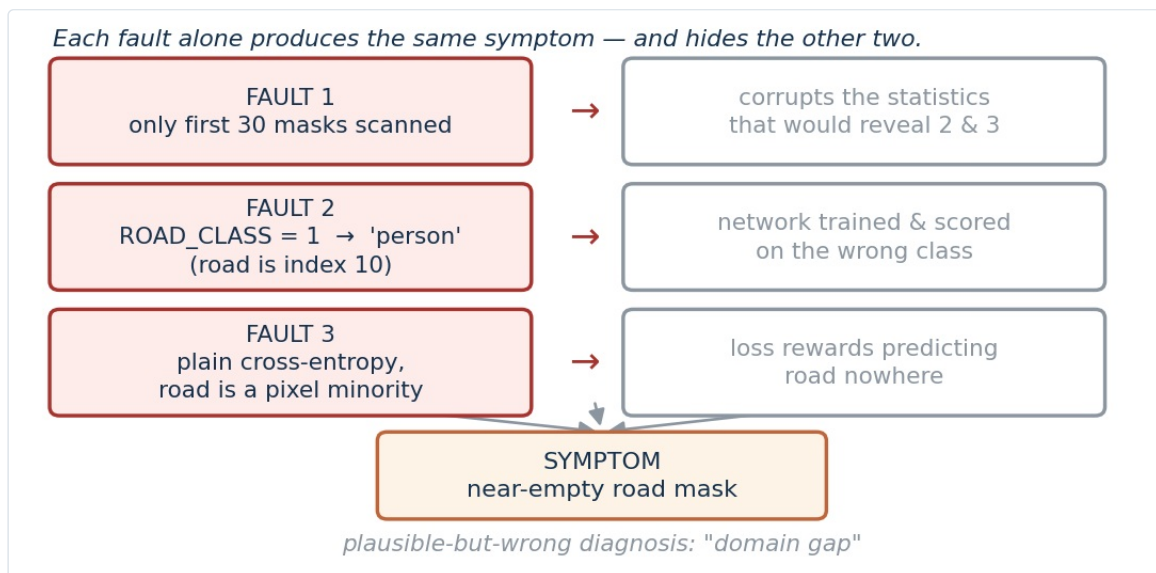


Figure 8. Three stacked faults, one symptom. Each alone yields a near-empty mask, and each conceals the other two — which is why fixing any single one appeared to change nothing.

Fixing Fault 3 alone (add Dice loss) with `ROAD_CLASS = 1` still trains a person-segmenter, which is still nearly empty — and one would conclude Dice loss "didn't help." Fixing Fault 2 alone, under plain cross-entropy, yields a network that still learns to suppress the minority road class — and one would conclude the index "wasn't the problem either." **Only the joint fix reveals either.** This is the structural reason stacked bugs are so much more expensive than sequential ones, and the practical reason to distrust the conclusion "I tried X and it didn't help" when more than one thing may be wrong.

40.5 The diagnostic principle

The generalisable rule, stated as sharply as possible:

Before blaming the model, verify that the **data**, the **labels**, and the **objective** describe the problem you think you are solving. Empty predictions for a class are far more often a label-index or class-imbalance bug than an architectural inadequacy.

41. The Replacement Architecture: MobileNetV2 U-Net

41.1 Why not just fix the bugs in the old network?

The bugs had to be fixed regardless. But the diagnosis also exposed a second, independent problem: the original **plain U-Net** had **~7.7 M parameters** and **its encoder was trained from scratch**, and on the Jetson Orin Nano it **ran out of GPU memory attempting to allocate 1.04 GiB**. A network that cannot fit in the target device's memory has no future on that device, no matter how good its masks are.

So the rewrite addressed correctness and deployability together.

41.2 Why MobileNetV2 as the encoder

The chosen architecture is a **U-Net with a MobileNetV2 encoder, ImageNet-pretrained**. The reasoning:

- **Pretrained features.** A from-scratch encoder must learn edges, textures, and shapes from 3269 aerial images. An **ImageNet-pretrained** encoder arrives already knowing them, which both improves accuracy and substantially mitigates the domain-gap problem — pretrained features generalise across visual domains far better than features overfitted to a small dataset. This directly attacks the weak-features half of the original diagnosis.
- **Parameter count.** MobileNetV2's backbone is roughly **2.3 M parameters**, against ~7.7 M for the old plain U-Net and ~25 M for a ResNet-50 encoder. It is *designed* for edge devices.
- **The alternative was rejected on evidence.** ResNet-50 was considered and set aside precisely because the Orin Nano had already hit memory limits with a *smaller* network. Choosing ResNet-50 would have meant training a model for days and then discovering it was too slow to fly — the "train it, discover it doesn't fit, retrain" cycle. Given a device that has already demonstrated its memory ceiling, the correct move is to design within the ceiling from the start.

This is an instance of a broader principle in embedded ML: **the deployment target is a design constraint, not a deployment detail**. The Orin Nano's 8 GB — shared between the OS, ROS, the camera pipeline, and the network — is the number that chose the architecture.

41.3 The U-Net structure, retained

The encoder–decoder structure with **skip connections** is retained unchanged from Chapter 16, because it is the right structure for segmentation: the encoder compresses the image into semantically rich but spatially coarse features ("there is road"), the decoder upsamples back to full resolution, and the skip connections carry high-resolution detail across from encoder to decoder so the output has sharp boundaries ("*there*, precisely, is the road"). Swapping the encoder for MobileNetV2 changes what computes the features, not the shape of the information flow.

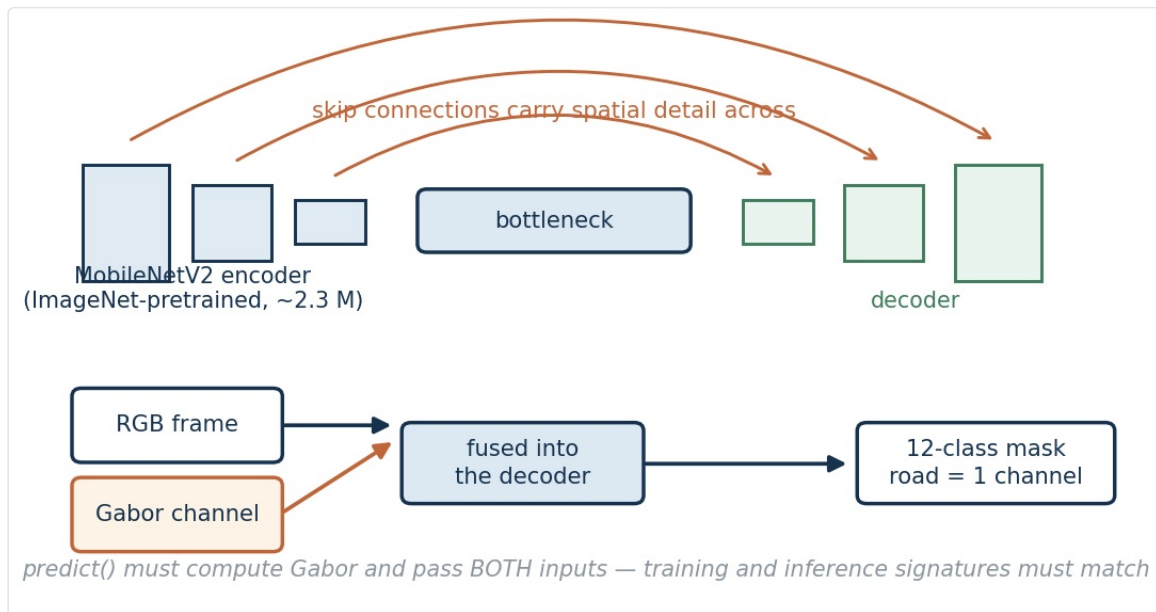


Figure 9. The replacement network: a U-Net with an ImageNet-pretrained MobileNetV2 encoder, with the Gabor texture channel fused into the decoder.

41.4 Model serialisation across Keras versions

The trained model is saved as `.weights.h5`, not as a `.keras` file. The reason is a version incompatibility: the training environment uses **Keras 3**, while the Jetson runs **TensorFlow 2.15 / Keras 2.15** (Chapter 44), and a **Keras 3** `.keras` archive will not load under Keras 2.15.

The workaround is architecture-plus-weights rather than a serialised model object: on the Jetson, **rebuild the identical architecture in code** (`build_mobilenet_unet_gabor()`) and then `load_weights()` into it. Weights are just arrays of numbers and cross versions cleanly; a serialised model is a versioned object graph and does not. This is the same version-agnostic loading path the existing `segmentation.py` already used, so it required no new mechanism — but shipping the wrong file format would have produced a mystifying load failure on the aircraft rather than on the workstation.

42. Gabor Texture Fusion

42.1 The idea, and where it came from

Chapter 19 recorded Dr. Mishra's proposal: **add texture as a second segmentation feature**. This part records that the proposal was implemented and, importantly, **validated before being trusted**.

The motivating observation is that road is not primarily distinguished from its surroundings by *colour*. Asphalt is grey; so is a rooftop, so is dry dirt, so is concrete. HSV thresholding works in simulation precisely because Gazebo's road is rendered in a narrow, artificial colour band; on real imagery the colour overlap between asphalt and its lookalikes is severe. But asphalt is distinguished by **texture**: it is *smooth* and spectrally uniform, whereas dirt is *grainy* and rooftops carry regular structure (tiles, seams, panels).

42.2 What a Gabor filter is

A **Gabor filter** is a sinusoid multiplied by a Gaussian envelope. Intuitively, it is a small patch of striped wave, localised in space:

- The **sinusoid** has a *frequency* and an *orientation* — so the filter responds strongly to stripes of a particular coarseness running in a particular direction.
- The **Gaussian envelope** confines it to a small neighbourhood, so the response is *local*: it tells you about texture *here*, not about the image as a whole.

Convolving an image with a **bank** of Gabor filters (several frequencies $\times$ several orientations) produces, at each pixel, a vector of responses describing the local texture: how much fine structure, how much coarse structure, running which way. Smooth asphalt yields low responses across the bank. Grainy dirt yields high responses at high frequencies in all orientations. A tiled roof yields strong responses at the specific frequency and orientation of its tiles.

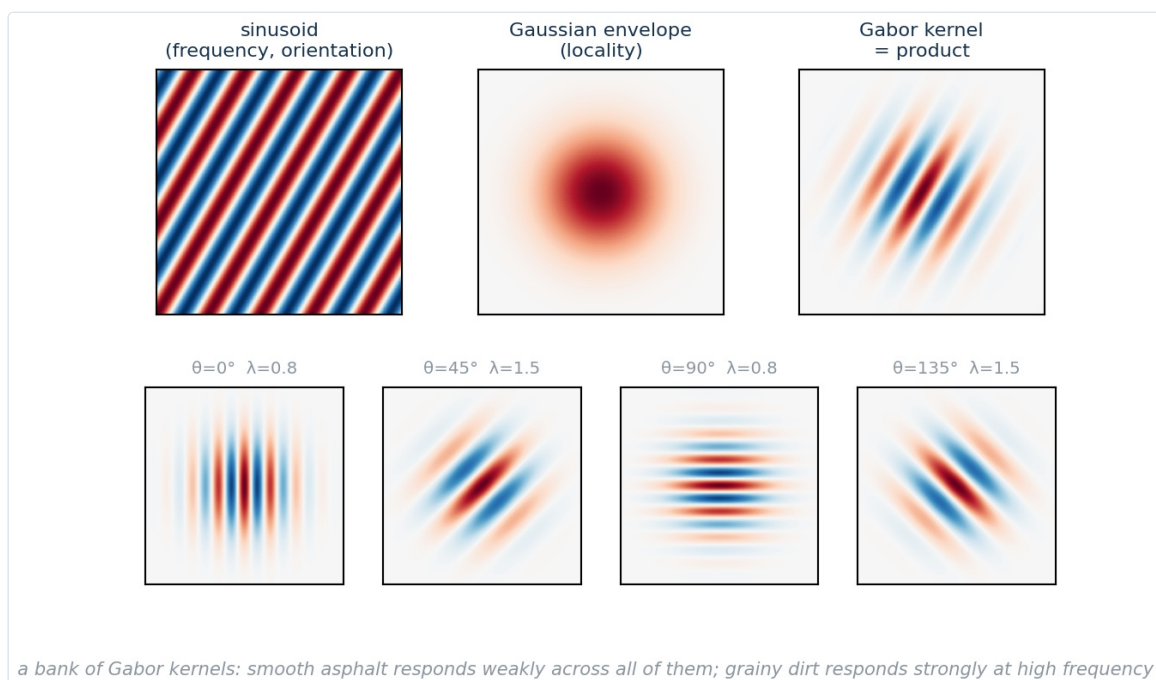


Figure 10. A Gabor kernel is a sinusoid multiplied by a Gaussian envelope — a localised detector for stripes of a given frequency and orientation. A bank of them describes local texture.

Gabor filters are a classical, decades-old computer-vision tool — they are also a well-known model of the receptive fields of simple cells in mammalian visual cortex, which is a pleasing reason to expect them to be a reasonable texture basis.

42.3 The validation step — the part that matters

The important methodological point is that the texture cue was **tested before being built in**. A Gabor texture analysis on the project's own imagery **confirmed that asphalt is spectrally separable from dirt and rooftops** — that is, the Gabor response distributions for road and for its principal colour-lookalikes genuinely do not overlap the way their colours do.

This ordering is the whole discipline. Adding a feature because it *sounds* like it should help is how models acquire channels that contribute nothing but latency. Confirming separability first means the feature is known to carry the discriminative information the network needs *before* a single epoch is trained. Had the analysis shown road and dirt to be spectrally similar, the correct response would have been to discard the idea, not to hope the network would find something in it.

42.4 How it is fused

The implementation is a **fixed classical filter bank**, not a learned one. Eight kernels — **four orientations** (0°, 45°, 90°, 135°) × **two spatial frequencies** (0.1 and 0.25 cycles/pixel) — are constructed once at import time via `cv2.getGaborKernel(ksize=(21,21), sigma=4.0, gamma=0.5, psi=0)`. At inference each frame is filtered by all eight and the per-pixel **standard deviation across the eight responses** becomes the texture-energy map:

```
energy(x,y) = std_k( conv(gray, kernel_k)(x,y) )    for k = 1..8
```

`std` is the right aggregator, not `sum`. A smooth surface with a single strong-response direction — a road lane marking, say — scores high on `sum` but low on `std`, and it is `std` that correctly reads it as *not* textured. Asphalt reads low on `std` because its response is similarly weak in every orientation; grainy dirt reads high because the response varies strongly across orientations.

The energy map is then **inverted** to `1.0 - energy` and passed to the network as a **single-channel** input, so that asphalt reads *high* in the returned map — matching the sign convention of a "roadness" cue and making the fusion additive rather than requiring the decoder to learn a sign flip.

Fusion is late in the decoder, not at the bottleneck. The Gabor channel enters through a small stem — two `Conv2D(16, 3×3, ReLU)` layers plus a `MaxPooling2D` — that brings the 256×256 texture map down to the 128×128 spatial scale of the outermost decoder block, where it is concatenated to the up-sampled decoder features. Late fusion preserves the spatial precision of the texture cue, which a segmentation task needs; fusing at the 8×8 bottleneck instead would have thrown away the very locality that makes texture useful.

At inference this imposes a contract on the deployment code: `predict()` must compute the Gabor channel per frame and pass both inputs (`{"rgb": ..., "gabor": ...}`), exactly as training did. A network fed only RGB, having been trained on RGB + Gabor, produces degraded output — a quiet, hard-to-spot deployment bug. The implementation of both the filter bank and the fusion is reproduced in Appendix I.6.

42.5 Why texture also buys speed

The latency argument from Chapter 19 survives intact. Texture information that the network would otherwise have to *learn to extract* — spending capacity and layers doing so — is instead **supplied directly** by a cheap classical convolution. The network can be smaller for the same accuracy, and a smaller network is a faster network. Given the Orin Nano's demonstrated memory ceiling, handing the network a feature it would otherwise have to discover is a very good trade.

43. Dice + Cross-Entropy Loss, and the Result

43.1 Fixing the objective

Fault 3 (Chapter 40.3) was that pixel-averaged cross-entropy rewards ignoring a minority class. The fix is to change what the network is optimising.

Dice loss derives from the Dice coefficient, a measure of *overlap* between the predicted region and the true region:

$$\text{Dice} = \frac{2 \cdot |\text{Prediction} \cap \text{Truth}|}{|\text{Prediction}| + |\text{Truth}|}$$

Read it carefully and its virtue is obvious. Dice is computed **per class**, as a **ratio involving only that class's pixels**. It does not care that road is 8% of the image and vegetation is 60%. A network that predicts road nowhere scores a Dice of **zero** for road — catastrophic — regardless of how well it does on the majority classes. Cross-entropy, by contrast, would happily accept that trade.

Dice loss is thus **scale-invariant with respect to class frequency**, which is precisely the property class imbalance destroys.

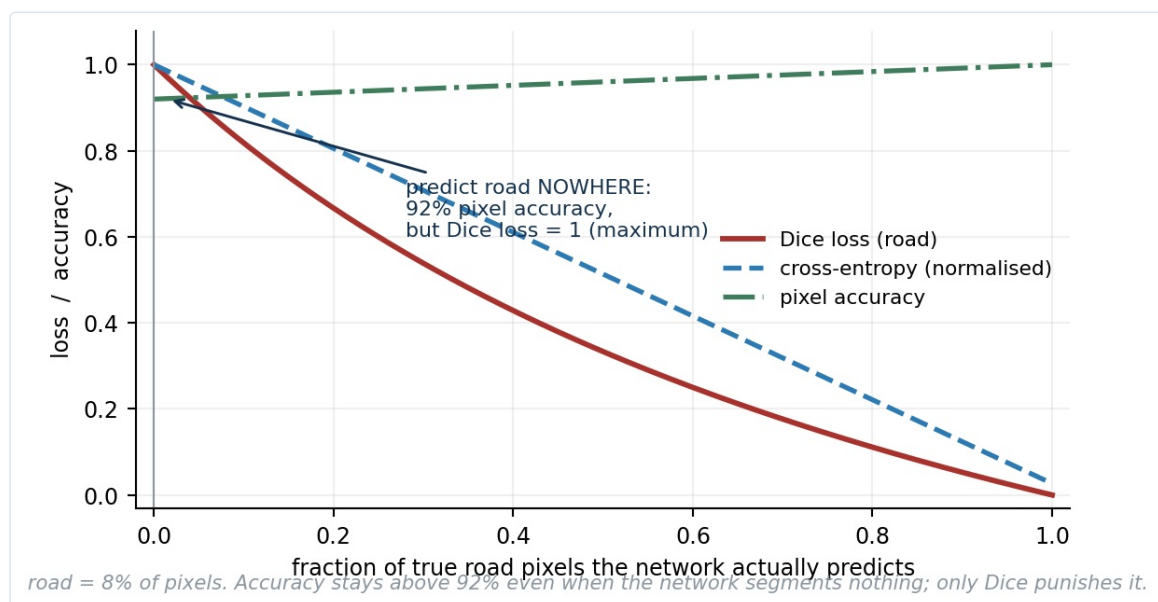


Figure 11. Why cross-entropy tolerates an empty road mask and Dice does not. With road at 8% of pixels, a network that predicts road nowhere still scores above 92% pixel accuracy — but its Dice loss is maximal.

43.2 Why both, and not Dice alone

The project uses **Dice + cross-entropy** together. Each compensates for the other's weakness:

- **Dice alone** has poor gradient behaviour when a class is entirely absent from a batch (the ratio degenerates), and can be unstable early in training when predictions are near-random.
- **Cross-entropy alone** is smooth, well-conditioned, and provides reliable per-pixel gradients from the first step — but is class-frequency blind, which is the original disease.

Summing them yields an objective that is **stable to optimise** (from CE) and **imbalance-aware** (from Dice). This combination is standard practice in medical and remote-sensing segmentation for exactly this reason, and it is the correct answer to Fault 3.

43.3 The corrected training setup

Putting the three fixes together:

Fault	Fix
-------	-----

Only first 30 masks scanned	Scan the full dataset when computing class statistics
<code>ROAD_CLASS = 1</code> (person)	Correct class handling; road is index 10 in AeroScapes' native labelling
Plain cross-entropy	Dice + CE
(plus) 7.7 M params, GPU OOM	MobileNetV2 encoder, ImageNet-pretrained
(plus) colour-only features	Gabor texture channel fused into the decoder

The network is trained over **12 classes** — AeroScapes' 11 semantic classes plus background — rather than a binary road/not-road split. Training on the full label set gives the network a richer supervisory signal (it must explain *every* pixel, not merely separate one class from an undifferentiated "everything else"), and the road mask is then extracted from the multi-class output. **Per-class IoU** is reported each epoch, with the **road IoU** being the number that actually matters; a mean-IoU averaged across twelve classes would once again hide the performance of the one class the drone flies on.

43.4 The result

`val_road_iou = 0.80`.

Intersection-over-Union of 0.80 on the validation set, *for the road class specifically*. In segmentation terms this is a solid, usable result: the predicted road region and the true road region overlap by 80% of their union. For comparison, the failing predecessor produced masks that were **near-empty** — an IoU near zero. The gap between those two numbers is the whole content of this part.

This is a validation score on AeroScapes. It confirms that the architecture, texture fusion, and loss are correct and that the previous failure was diagnostic. It does not demonstrate performance on real construction-site imagery through a RealSense D455 at 640×480.

43.5 The domain gap survives — honestly stated

The network still trains on AeroScapes. It segments AeroScapes road far better than its predecessor. But **construction-site imagery and D455 frames remain out-of-distribution**: different camera, different altitude regime, different surfaces, different lighting, different country.

The remaining step is therefore **fine-tuning on the project's own labelled frames**, which the live RealSense (Chapter 33) is now in a position to collect. The ImageNet-pretrained encoder makes this fine-tuning far more tractable than training from scratch would have been — pretrained features transfer, which is why the encoder choice pays twice.

The correct summary of the network's status is: **the model is fixed; the domain is not**.

43.6 The real camera intrinsics

A related deployment detail belongs here, because it governs whether a correct mask becomes a correct *plan*. The projection from image pixels to ground coordinates (Chapter 3, Chapter 14) requires the camera's **intrinsics** — its focal lengths and principal point in pixels. Simulation values are not the D455's values.

The **measured RealSense D455 intrinsics at 640 × 480** are:

Parameter	Value	Meaning
<code>fx</code>	388.162	Focal length, x (pixels)
<code>fy</code>	388.162	Focal length, y (pixels)
<code>cx</code>	322.326	Principal point, x (pixels)
<code>cy</code>	239.442	Principal point, y (pixels)

These **replace the simulation values in `planner_node` when `mode = real`**. Note that `fx = fy`, as expected for square pixels, and that `cx ≈ 322` and `cy ≈ 239` sit close to but not exactly at the geometric image centre (320, 240) — the small offset is the real, physical decentring of the sensor relative to the lens axis, and it is exactly the sort of detail that must be

measured rather than assumed. A perfect road mask projected through the wrong intrinsics produces a confidently wrong path.

Part XI — Environment and Dependency Engineering

Nothing in this part is about robotics. It is about the fact that two libraries the project cannot do without demanded mutually incompatible versions of a third, and that this — not perception, not control — was for a time the single blocker preventing the software stack from running on the aircraft's computer at all. It is included at length because dependency conflicts are the most under-documented and most time-consuming class of problem in applied robotics, and because the resolution here is fragile enough to need a written record.

44. The Protobuf Collision

44.1 The two libraries

The Jetson must run, in one Python environment:

- **TensorFlow 2.15**, installed from the **NVIDIA JetPack apt repository** — the GPU-accelerated build for Jetson. It runs the segmentation network. Ordinary `pip install tensorflow` gives an x86 wheel or a build without CUDA support; on Jetson, TensorFlow must come from NVIDIA's repository to be hardware-accelerated at all.
- **MAVSDK-Python**, which speaks to the Pixhawk. As established in Chapter 9, MAVSDK-Python is a thin async client that communicates with a **MAVSDK server over gRPC**.

44.2 The shared dependency

Both depend on **protobuf** (Protocol Buffers), Google's serialisation library. TensorFlow uses it to describe graphs and saved models; gRPC — and therefore MAVSDK — uses it to encode every message on the wire.

And they disagree:

Library	protobuf requirement
TensorFlow 2.15	<code>protobuf < 5</code>
MAVSDK-Python 3.15.3	<code>protobuf >= 6</code>

There is no version of protobuf satisfying both. `< 5` and `>= 6` have empty intersection. This is not a resolvable constraint problem — no amount of dependency-solver cleverness produces a solution — and `pip` will either refuse, or (worse) install one and silently break the other.

44.3 Why this is a genuine blocker, not an annoyance

The two libraries must coexist **in one process**, because `segmentation_node` (TensorFlow) and `control_node` (MAVSDK) are ROS 2 nodes in one workspace, and in the deployed configuration they share an interpreter environment. One could imagine separating them into different processes with different virtual environments — and that is a legitimate architecture — but it introduces an inter-process boundary precisely where the pipeline needs to be tight, and it multiplies the deployment surface on a device that must boot reliably.

The correct resolution was to find a version pair that actually coexists.

45. The Resolution, and Its Fragility

45.1 The fix

Since TensorFlow's constraint is immovable (it is the accelerated Jetson build; there is no swapping it), the movable party is MAVSDK. The resolution:

- **Downgrade MAVSDK-Python to 2.8.4**, a version whose protobuf requirement is compatible with TensorFlow's.
- **Pin `protobuf == 3.20.3`** explicitly, so that neither library's transitive dependency resolution wanders off to an incompatible version later.

With that pair, **both libraries import successfully in the same environment**.

45.2 The verification that mattered

Downgrading a library carries an obvious risk: the older MAVSDK might not talk to the real flight controller. Import success proves nothing about function. So the fix was verified against hardware rather than against `pip`:

```
import asyncio
from mavsdk import System

async def run():
    d = System()
    await d.connect(system_address='serial:///dev/ttyACM0:57600')
    async for s in d.core.connection_state():
        if s.is_connected:
            print('✓ mavsdk 2.8.4 connects to real Pixhawk')
            break

asyncio.run(run())
```

This printed its success line against the **real Pixhawk over USB**. That closed the loop:

- TensorFlow 2.15 imports ✓
- MAVSDK 2.8.4 imports ✓
- Both coexist under protobuf 3.20.3 ✓
- **MAVSDK 2.8.4 actually communicates with the flight controller** ✓

The last line is the one that matters. The first three establish that nothing crashes on import; only the fourth establishes that the downgrade did not silently cost the project the ability to fly.

45.3 The fragility, stated plainly

`protobuf == 3.20.3` is a **fragile pin**. It sits at a specific point that happens to satisfy two libraries which each want to move away from it in opposite directions. Any of the following will break it:

- Upgrading TensorFlow (a newer version may demand a newer protobuf).
- Upgrading MAVSDK (later versions want protobuf 6+; that is what caused this).
- Any transitive dependency quietly pulling a different protobuf during an unrelated `pip install`.
- A system update that reinstalls one of them from apt.

Recovery procedure: if either TensorFlow or MAVSDK breaks, check `protobuf` first and re-pin. The working triple (TF 2.15 / MAVSDK 2.8.4 / protobuf 3.20.3) is treated as a unit; a pinned `requirements.txt` or container image is the durable next step.

45.4 The cost of the alternative

The alternative considered and rejected was separate virtual environments — one for the TensorFlow node, one for MAVSDK — communicating over ROS topics. This would remove the conflict permanently but adds a second Python environment to provision, a second set of ROS dependencies, and an additional failure mode at boot on a device that must

come up autonomously. The single pinned environment is the correct trade for Prototype 1- α ; separated environments become the appropriate choice for a hardened system.

46. Package Repositories, `setuptools`, and Other Environment Debris

Three smaller environment problems were found and fixed. They are recorded because each cost time and each will recur on a fresh Jetson.

46.1 Duplicate ROS apt repository

Symptom. `apt` warnings and ambiguous package resolution. **Cause.** The ROS 2 apt repository was configured twice, through two separate source-list files. **Fix.** Remove the redundant `ros2.list`, leaving a single authoritative source. Duplicate source lists cause `apt` to select non-deterministically between identically-named packages, producing intermittent version resolution.

46.2 `setuptools` too new

Symptom. ROS 2 Python package builds (`colcon build`) failing or emitting deprecation errors. **Cause.** `setuptools` version 83, far newer than what ROS 2 Humble's Python packaging expects. Humble targets an older packaging ecosystem, and modern `setuptools` has removed or deprecated interfaces that `ament_python` still relies on. **Fix.** Downgrade `setuptools` to the 79.x series. **Lesson.** A ROS 2 distribution is pinned to an era of the Python ecosystem. Upgrading the toolchain underneath it is not an improvement; it is a change of assumptions. This is the same disease as the protobuf conflict, in a different organ.

46.3 Noise that is not signal

Not every warning is a fault, and chasing benign warnings is its own time sink. The following are **expected and ignorable** on this Jetson:

- **cuDNN / cuFFT factory registration warnings** at TensorFlow import — the library registering plugins more than once. Cosmetic.
- **SciPy / NumPy version-mismatch warnings** — a version advisory, not an error, in this configuration.

Recording *which* warnings are safe is as valuable as recording which are not. On a device where a dozen lines of red text appear on every launch, the ability to say "those two are noise, that third one is real" is what keeps debugging tractable.

46.4 Versioned Gazebo transport imports

A related packaging detail from the simulation side: the camera bridge imports Gazebo's Python bindings under **explicitly versioned module names** — `gz.transport13` and `gz.msgs10.image_pb2` — rather than unversioned aliases. Gazebo's Python packages carry their ABI version in the module name, and importing the versioned name directly is what makes the bridge work **without any virtual-environment workaround**. Guessing the unversioned name, or assuming the alias exists, produces an import error that looks like a missing installation when the package is in fact present.

47. The Environment as a Recorded Artefact

The software environment on the Jetson is an experimentally-determined artefact: three times the correct version was fixed by discovering another version failed. The working state is:

Component	Version	Source	Note
TensorFlow	2.15	NVIDIA JetPack apt repo	Must come from NVIDIA for GPU support
MAVSDK-Python	2.8.4	pip	Downgraded from 3.15.3
protobuf	3.20.3	pip, pinned	Fragile; the pin that makes the above coexist
setuptools	79.x	pip	Downgraded from 83
Keras (Jetson)	2.15	with TF	Cannot load Keras 3 <code>.keras</code> files (Ch. 41.4)
ROS 2	Humble	apt (single source)	Native, not containerised
Gazebo Python	<code>gz.transport13</code> , <code>gz.msgs10</code>	versioned import	No venv workaround needed

This table is, in practical terms, one of the most valuable pages in this document. Reproducing the aircraft's software from scratch without it means rediscovering every one of these constraints by failure.

Part XII — Distributed SITL and Performance Diagnosis

48. The Distributed Topology

48.1 Why simulation became distributed

The Jetson is the target computer, but the Jetson cannot run Gazebo — a GPU-hungry physics-and-rendering simulator — while also running the pipeline it is meant to be testing. And the perception/planning code must eventually run *on the Jetson*, not on a workstation, because that is where it will fly.

The resolution is to **split the simulation across two machines**: one host runs PX4 SITL and Gazebo (the simulated world and the simulated aircraft), and the Jetson runs the real ROS 2 pipeline against it. The Jetson sees simulated camera frames and sends real MAVLink commands, exactly as it will in flight, but nothing can crash.

48.2 The two links

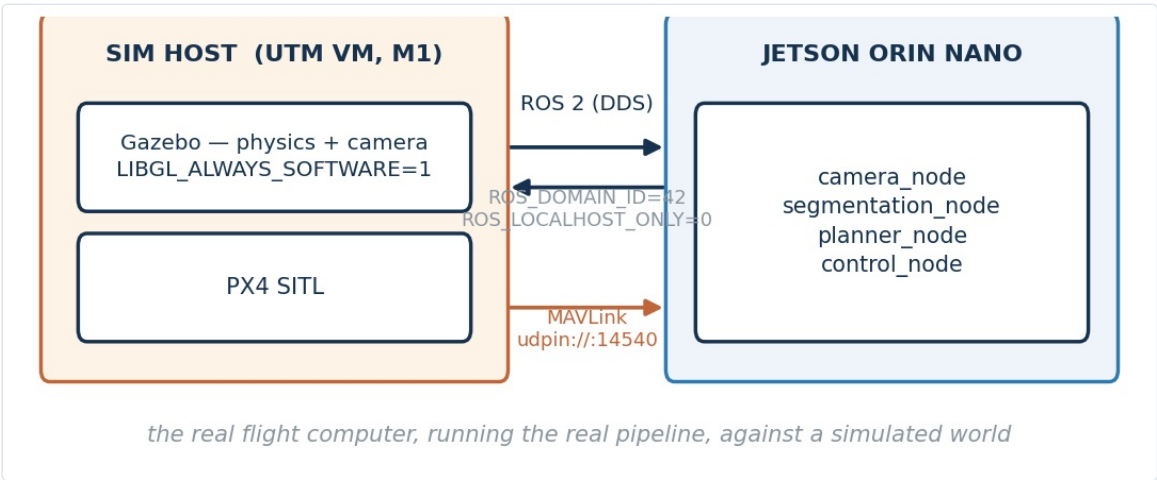


Figure 12. Distributed SITL: Gazebo and PX4 SITL on a simulation host, the real ROS 2 pipeline on the Jetson, joined by cross-machine DDS and MAVLink.

Link 1 — ROS 2 discovery across machines. ROS 2's DDS layer will find nodes on other hosts, but only if configured to:

Setting	Value	Why
<code>ROS_DOMAIN_ID</code>	<code>42</code>	Both machines must share a domain ID; nodes in different domains never see each other
<code>ROS_LOCALHOST_ONLY</code>	<code>0</code>	The default of <code>1</code> confines DDS to loopback, making cross-machine discovery impossible

Both are required. `ROS_LOCALHOST_ONLY=1` is a common, sensible default on a workstation and an absolute barrier to distributed operation — and it fails *silently*, presenting as "the other machine's topics don't exist."

Link 2 — MAVLink from the sim host to the Jetson, over `udpin:///14540`, so `control_node` on the Jetson connects to PX4 SITL on the VM using the same MAVSDK connection path it uses for a real board (Chapter 9). Cross-machine ROS 2 discovery and cross-machine MAVLink both work.

49. Diagnosing the 4.5 Hz Bottleneck

49.1 The symptom

The pipeline ran, and ran **slowly**. Camera frames arrived at the Jetson at roughly **4.5 Hz** — far below what a control loop wants. A perception→planning→control loop fed at 4.5 Hz cannot react to anything quickly, and a road-follower that sees the world four times a second will oscillate or overshoot.

49.2 The suspects, and their acquittal

The pipeline between the rendered frame and the planner has many stages, and each is a plausible culprit. Each was investigated and **eliminated**:

Suspect	Verdict
<code>gz_camera_bridge</code> (Gazebo → ROS 2)	Not the bottleneck
The network between VM and Jetson	Not the bottleneck
USB (camera bandwidth)	Not the bottleneck — not even in the loop for sim
The protobuf / gRPC stack	Not the bottleneck

This is the part worth emulating. Each of these is a *satisfying* explanation — the bridge is custom code, the network is a shared link, USB had a known cap (Chapter 18), protobuf had just caused a crisis (Chapter 44). Any of them could have absorbed days of optimisation. **All were innocent.**

49.3 The actual cause

Gazebo was running at roughly 20% of real time, inside a UTM virtual machine on an M1 MacBook Air, with `LIBGL_ALWAYS_SOFTWARE=1`.

That environment variable is the whole story. It instructs OpenGL to render **in software, on the CPU**, because the VM does not expose the Mac's GPU to the guest. Gazebo — which must render the drone's downward camera view every frame — is therefore drawing images with the CPU. Software rendering is one to two orders of magnitude slower than GPU rendering.

The consequences chain cleanly:

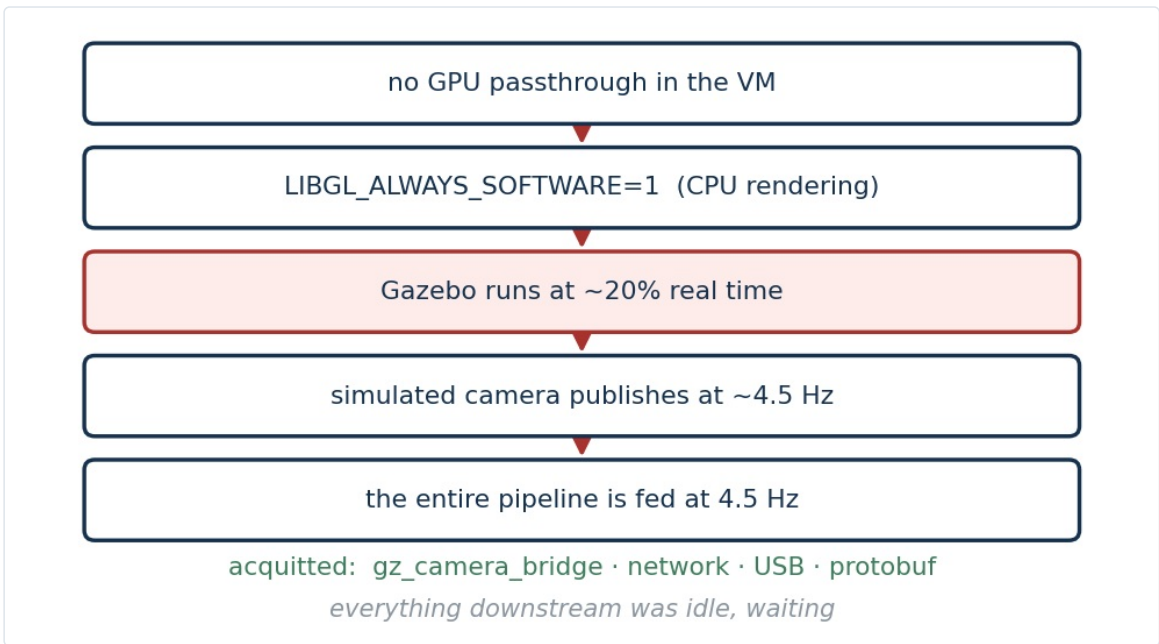


Figure 13. The 4.5 Hz causal chain. The bridge, the network, USB, and protobuf were each investigated and each acquitted; the cause was software rendering in a VM with no GPU passthrough.

Nothing downstream was slow. Everything downstream was **waiting**. The bridge, the network, and the Jetson were all idling between frames, processing each one promptly when it finally arrived. Measuring their throughput in isolation would show them capable of far more — which is precisely why "the bridge seems fine" was true and useless until the source was found.

49.4 The fix

The fix is not code. It is a **faster simulation host**:

1. **Native Ubuntu with real GPU access** — removes software rendering entirely. This is the actual answer, and it is why the `cobot` lab desktop (native Ubuntu 22.04) is where road-following was successfully demonstrated end-to-end.
2. **Lower the SDF camera resolution and `update_rate`** — if the simulated camera renders fewer, smaller frames, software rendering can keep up. This trades fidelity for speed and is a legitimate stopgap.
3. **A simpler Gazebo world** — Baylands is a rich environment; less geometry is less to render.

49.5 Generalised diagnostic pattern

This is the same class of failure as the gRPC crash of Appendix D.5. A slow pipeline stage is often a symptom of upstream starvation rather than a local defect. The investigation order is upstream — from consumer toward producer — measuring source rate first, until either a saturated stage is found or the producer itself is identified as the bottleneck.

50. Why HITL Is Not Viable on This VM

50.1 What HITL is

Hardware-In-The-Loop replaces SITL's simulated flight controller with the **real Pixhawk**, running its real firmware, receiving simulated sensor data from Gazebo and sending real motor commands back into the simulation. It is the highest-fidelity test short of flying: the actual flight-control code, on the actual board, closing the loop against a simulated aircraft.

It was considered, and rejected on this hardware.

50.2 Why it fails here

A real Pixhawk runs in **wall-clock time**. Its EKF2 timestamps sensor readings against its own clock, and its failsafe logic asks whether each sensor reading is *fresh* — whether the IMU sample it just received is recent enough to trust.

SITL can be slowed down without harm, because the simulated flight controller's clock is slowed too; everything stays consistent. A **real** Pixhawk cannot. If Gazebo produces sensor data at 20% of real time (Chapter 49), then from the Pixhawk's perspective sensor readings are arriving **five times too slowly**, with timestamps that drift ever further into the past.

The board does exactly what it was built to do: it concludes that its sensors have stopped updating, and it **triggers sensor-freshness failsafes**. The simulation does not run; it fails safe, correctly, against a world that is running too slowly to be real.

50.3 The conclusion

HITL with the real Pixhawk is not viable on this VM. The wall-clock mismatch between a real-time flight controller and a 20%-real-time simulator is fundamental — it is not a configuration issue and cannot be tuned around, because the failsafe it trips is a genuine safety feature that should not be disabled.

HITL becomes available once the simulation host runs Gazebo at or near real time (Chapter 49.4). Until then, the correct test ladder is: **SITL on a fast host** (`cobot`, where road-following was verified) → **distributed SITL** (Jetson against a sim host, the topology of Chapter 48) → **HITL on a real-time-capable host** → **bench tests with the real board** (Chapter 38) → **flight**.

Part XIII — Safety Architecture

51. The Pilot Gate: `_wait_for_pilot()`

51.1 The danger being designed against

Consider the aircraft as it will actually exist: a Jetson bolted to a frame, wired to a Pixhawk, wired to four ESCs and four motors, powered from a 12000 mAh pack. The Jetson boots on power-up. If the pipeline starts automatically, and `control_node` connects, arms, and takes off as part of its normal startup sequence (which is precisely what Chapter 26's flight sequence describes), then **plugging in the battery on a workbench launches the aircraft**.

This is not a hypothetical failure mode. It is the default behaviour of the code as originally written, and it is a genuine mechanism for serious injury.

51.2 The v7 patch: two independent locks

The v7 safety design consists of **two independent mechanisms that must both fail** before an autonomous takeoff can occur. Neither alone would suffice; together they close the failure mode.

Lock 1 — the MAVSDK wrapper (`core/px4.py`) never starts Offboard in real mode. The `Drone` class takes an `allow_offboard_autostart` flag. In sim mode it is `True` and `set_velocity()` calls `offboard.start()` on the first setpoint (unchanged from earlier versions). In real mode it is `False`, and `set_velocity()` streams setpoints via `set_velocity_ned()` but **never** calls `offboard.start()`. The flight controller, by design, ignores Offboard setpoints unless PX4 is in Offboard mode — and the only way for PX4 to enter Offboard mode when the wrapper never asks for it is for the pilot to select it on the RC transmitter. There is no software path by which the aircraft can be commanded into Offboard.

Lock 2 — the control node gates its own outputs behind `_wait_for_pilot()`. In real mode, the node checks three conditions in a polling loop and holds `takeoff_complete = False` until all three are met:

```
armed = await drone.is_armed()
alt = await drone.get_altitude()
offb = await drone.in_offboard()
if armed and alt is not None and alt > 2.0 and offb:
    takeoff_complete = True
```

Until that flag flips true, `loop()` returns immediately without publishing anything and `path_cb` drops incoming waypoints. Even if Lock 1 were bypassed, the aircraft still would not fly, because the node itself is not emitting setpoints.

Two ROS parameters — `auto_takeoff` and `px4_connection` — expose the sim/real switch cleanly and are wired to both locks in `ControlNode._init`: `allow_offboard_autostart` is set to the same value as `auto_takeoff`, so the launch file's `mode:=real` argument flips both locks together and it is impossible for one to be enabled without the other.

51.3 Why this is the right shape of solution

The elegance is that the gate is defined by **what the pilot does with a physical transmitter**, not by any state the software can reach on its own:

- **Arming** is a deliberate physical action on the RC, and PX4 owns the arming decision (Chapter 8.4). The companion computer cannot arm the aircraft past the pilot.
- **The 2 m altitude threshold** means the aircraft has to actually be flying — not just armed on the bench with a mode switch left in the wrong position. This defends against an operator who arms the vehicle to check something, forgets they left the mode switch on Offboard, and finds that the moment the aircraft leaves the ground the pipeline goes live.
- **Enabling Offboard via RC** means the pilot has explicitly handed control to the companion computer using the mode switch assigned in Chapter 30 — and can take it back instantly by flicking the switch.

The software's authority to command the vehicle is **granted by the pilot, held only while the pilot continues to grant it, and revocable in the time it takes to move a switch**. The autonomy runs at the pilot's pleasure. That inverts the

dangerous default — instead of the code flying unless something stops it, the code waits unless the pilot permits it.

51.4 Relationship to the relaxed failsafes

This patch matters more than it would in a conventional setup, because Chapter 11 recorded two deliberate relaxations: `COM_RCL_EXCEPT = 4` (offboard exempt from the RC-loss failsafe) and `NAV_DLL_ACT = 0` (no data-link-loss action). Those relaxations exist so autonomy can run without a transmitter babysitting it — and they remove two of the automatic protections that would otherwise catch a runaway.

`_wait_for_pilot()` restores the balance at the *entry* to autonomy rather than during it. Combined with `COM_ARM_MAG_ANG = -1` (Chapter 38.1), the system now has three deliberately-disabled safety checks. Each is individually justified. Collectively they mean the aircraft's safety rests on **procedure and the kill switch** rather than on automatic protection — which is a defensible position for supervised bench and early flight testing, and an indefensible one for unsupervised operation.

52. The Layered Safety Model

52.1 The layers, from outermost to innermost

Collecting every safety mechanism described in this document into one picture:

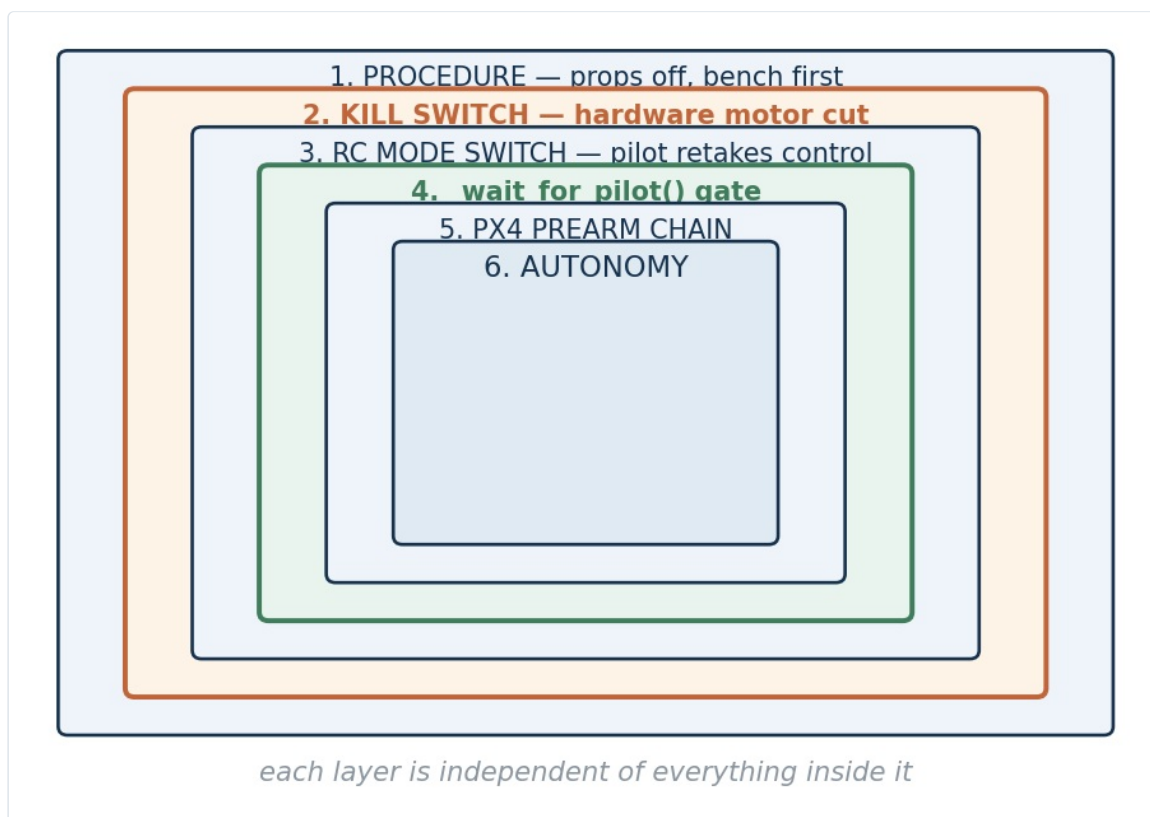


Figure 14. The layered safety model. Each layer is independent of everything inside it — the kill switch does not consult the software.

Each layer is independent of the ones inside it. The kill switch does not consult the software. The RC mode switch does not consult the planner. `wait_for_pilot()` does not trust `control_node` to behave. PX4's prearm chain does not trust the Jetson at all.

52.2 What has been deliberately disabled

Deliberately disabled checks:

Check	Setting	Why disabled	Risk accepted
RC-loss failsafe (offboard)	<code>COM_RCL_EXCEPT = 4</code>	Autonomy must run without a transmitter	A runaway will not auto-land on RC loss
Data-link-loss action	<code>NAV_DLL_ACT = 0</code>	Momentary GCS-link loss must not abort a test	Link loss produces no protective action
Compass consistency	<code>COM_ARM_MAG_ANG = -1</code>	Bench arming with a 128° compass disagreement	Must be re-enabled before outdoor flight

Three disabled checks is a lot. They are individually defensible for supervised bench work. The third in particular is a **bench expedient** and appears here, again, so that it cannot be quietly forgotten when the aircraft goes outside.

52.3 The operating rule

Given the above, one rule governs all flight testing, and it is not a software rule:

Never run the autonomy without a finger on the kill switch.

Every automatic protection that would ordinarily catch a misbehaving offboard controller has either been relaxed for good reason or does not apply. The kill switch is the layer that has not been compromised, that does not depend on any

software being correct, and that works in the time it takes to move a finger. It is the backstop, and Prototype 1- α 's safety case rests on it.

Part XIV — Original Contribution

53. The Road-Occlusion Recovery Theory

This chapter is the formal write-up of an original theory developed during a faculty Q&A with Dr. Mishra. It addresses a failure mode that a naive road-follower cannot survive, and it proposes a principled recovery. It is exactly the kind of idea that belongs in the personal notebook of original theories — and writing it down properly here is the first step toward formalising it.

53.1 The problem: when the road disappears from frame

The entire road-following approach assumes the segmenter can see road to follow. But roads are not continuous, clean ribbons: a stretch may be **damaged, missing, washed out, occluded by a vehicle or shadow, or simply absent** (a bridge gap, a construction break, a section the camera can't resolve). When that happens, the road **disappears from the frame** — the mask goes empty or splits into two disconnected pieces with a gap between them. A naive follower fails here in two ways: if the mask is empty it has nothing to plan on (the mask-zero condition, 6.4), and if the grid treats the gap as an obstacle, A* refuses to path across a gap the drone should fly straight over (20.2). The theory is about **recovering continuity across such a gap** so the drone keeps following the road through the missing section instead of stalling or aborting.

53.2 The two-part recovery

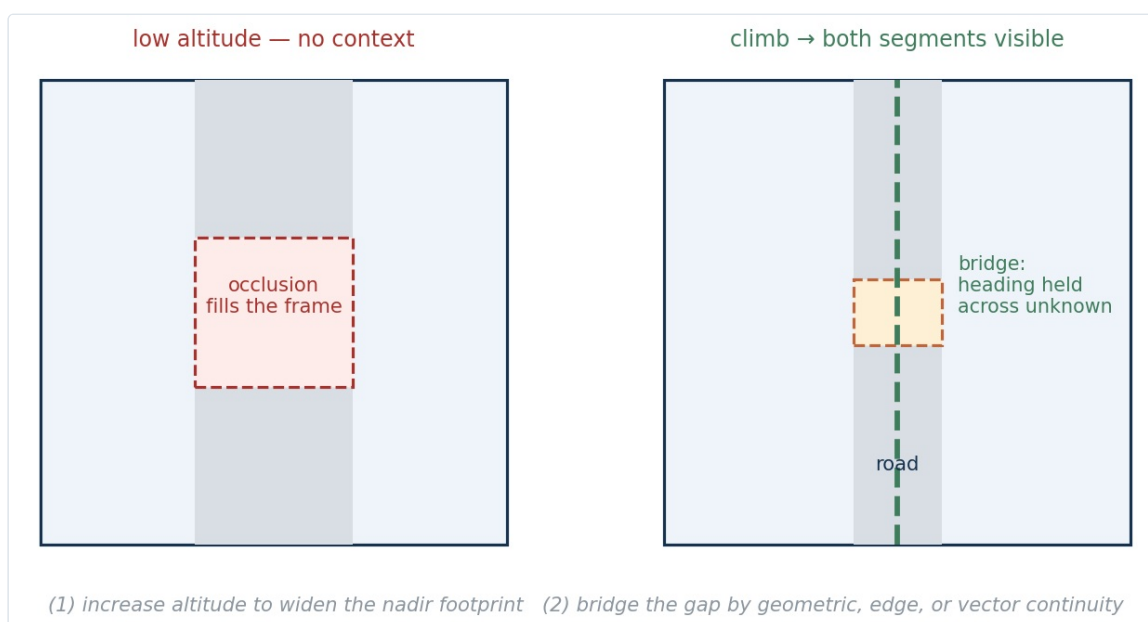


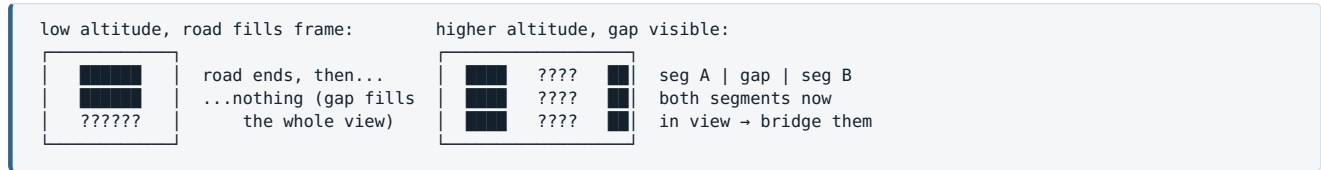
Figure 15. The road-occlusion recovery: climb to widen the nadir footprint until both flanking road segments are visible, then bridge the gap.

The theory's core is that recovery needs **two mechanisms working together**:

1. **Increase altitude to widen the field of view**, so that both road segments on either side of the gap come (back) into view. From higher up, the downward camera's ground footprint is larger (14.2), so a gap that filled the frame at low altitude becomes a small break with visible road on both sides. This re-establishes the *context* needed to reason about the gap: you cannot bridge two segments you cannot both see, and climbing is what makes both visible.
2. **Bridge the two visible segments across the gap** — construct a connection between the road that ends before the gap and the road that resumes after it, so the planner has a continuous route through. Climbing gives you the two endpoints; bridging joins them.

Neither alone suffices: climbing without bridging just shows you the gap more clearly; bridging without first seeing both sides is impossible. Together they turn "the road vanished" into "there is a gap between two known segments, and here is

how to cross it."



53.3 The three bridging methods

The theory proposes three concrete ways to bridge the gap, each exploiting a different kind of continuity the road is likely to have:

(a) **Geometric continuity — fit and extrapolate a curve.** Roads are smooth; they don't teleport. So you can **fit a curve (a line, arc, or spline) to each visible segment and extrapolate it through the gap**, connecting where segment A's curve is heading to where segment B's curve is coming from. If both segments' fitted curves point at each other across the gap, the extrapolation *is* the bridge — a predicted road path through the missing section based on the road's own shape. This is the most general method and works whenever the road's geometry is smooth and both segments are long enough to fit reliably.

(b) **Edge/boundary continuity — track the kerbs through the gap.** A road has **boundaries (kerbs, edges, verge lines)** that often **survive even when the surface itself is damaged or occluded** — a pothole or a missing patch of tarmac does not necessarily erase the road's edges, and edges may be visible in the gap even where surface segmentation fails. The boundaries are tracked through the gap and used to define the road's course across it. This method degrades differently from surface segmentation: the case that eliminates colour-based road detection (damaged surface) can leave boundaries intact, giving edge continuity information where surface information has been lost. It is complementary to (a): geometry uses the segments' shape, edges use the corridor's borders.

(c) **Vector continuity — hold the last known heading.** The simplest and most robust fallback: **hold the last valid heading constant across the gap**. Just before the road vanished, the drone had a well-defined direction of travel along it. If you have no better information, **keep flying in that direction** across the gap and expect the road to resume ahead — a straight-line dead-reckoning bridge using the last good heading. It assumes the road continues roughly straight through the gap, which is often true for short gaps, and it needs no fitting or edge tracking, so it works even when segments are too short or noisy for methods (a) and (b). It connects directly to velocity-based offboard control (10.4): commanding a held heading/velocity across the gap is exactly the vector-continuity bridge in action.

The three form a natural hierarchy of information: **geometry** (use both segments' shapes) → **edges** (use the corridor's borders when the surface is gone) → **vector** (use only the last heading when nothing else is reliable). A robust system could try them in that order, falling back to the simpler method when the richer one lacks data.

53.4 The natural implementation: mark the gap "unknown," not "obstacle"

and because texture/edge cues are exactly the kind of signal method (b) relies on. Improving perception and adding occlusion recovery are complementary attacks on the same reliability goal.

53.6 Why this theory is a genuine contribution

It is not a re-statement of a textbook method; it is a *synthesis* tailored to this system:

- It correctly identifies that recovery needs **both** a perception move (climb to see both sides) **and** a planning move (bridge), where either alone fails.
- It offers **three bridging strategies indexed by which kind of continuity survives** the occlusion — a genuinely useful taxonomy, especially the insight that **boundaries can outlive the surface**, so edge continuity provides information precisely when surface segmentation dies.
- It lands on an implementation that is **minimal and native to the existing planner** — the "unknown, not obstacle" + heading-guided A* framing — which is the mark of a good engineering idea: it dissolves a hard problem into a small, principled change to what you already have.

53.7 Status and next steps

This is currently a **theory, not yet implemented**, and — importantly — it is flagged to be **written into the notebook of original theories**. The concrete next steps to move it from idea to capability:

1. **Add the "unknown" state to the occupancy grid** (extend the free/occupied representation), and make A* treat unknown cells as high-cost-traversable rather than blocked.
 2. **Detect the occlusion condition** — an empty or split mask (reuse the mask-zero diagnostic, 6.4) — as the trigger to enter recovery.
 3. **Trigger the altitude increase** on that condition to widen FOV until both segments are visible.
 4. **Implement vector continuity first** (hold last heading, bias A* through the unknown region) as the simplest, most robust bridge — this is the smallest working version.
 5. **Layer in geometric and edge continuity** to shape/cost the unknown region more intelligently when the segments support a curve fit or edge track.
 6. **Test in simulation** with deliberately gapped roads before hardware, exactly as the rest of the pipeline was proven in sim first.
-

Part XV — What Is Further Required

This part consolidates every pending item flagged throughout the document into a single prioritised roadmap, grouped by how close each is to the critical path of "first real autonomous flight." Each item states *what*, *why it matters*, *what it depends on*, and *the concrete next action*. Nothing here is new — it is the sum of the "remaining" notes from every chapter, arranged so the order of work is clear.

54. Immediate Hardware Blockers (on the critical path)

These were the items standing directly between the state at last review and a first positioned, motor-driven flight. Two of the three have since been resolved by the first outdoor flight (16 July 2026, Part XVI §63); the third remains open.

54.1 Actuator mapping and ESC spin-direction verification — Resolved

Status. Resolved. The first outdoor flight (Part XVI §63) required the aircraft to arm, take off in Stabilized, and climb to ~4 m under its own power — which is only possible if the output-to-motor mapping and each motor's spin direction are correct. No further action needed on this item.

54.2 Outdoor GPS lock — Resolved

Status. Resolved. The same flight recorded `eph = 0.22 m` throughout (Part XVI §63.3), comfortably inside PX4's arming threshold. The "Fence requires position" prearm cleared and the aircraft armed normally outdoors.

54.3 RealSense on a true USB 3.x port (lift the bandwidth cap) — Still pending

What. Move the RealSense D455 to a genuine USB 3.x port with an adequate cable so it streams colour+depth at full resolution/rate instead of the current USB 2.1 cap. (Chapter 18.)

Why it matters. At USB 2.x rates the camera silently downgrades its streams, quietly limiting how good and how timely the road mask can be — perception "works" but under its potential.

Depends on. Physical port/cable availability on the Jetson carrier; verifying the negotiated USB speed.

Next action. Identify a true USB 3.x port, use a proper cable (no 2.0-only hub), confirm the D455 enumerates at 3.x and streams at full spec. As of the last hardware check (Part XVI, Ch. 18) this had not yet been done.

55. Immediate Software Tasks (near the critical path)

55.1 gRPC reconnect patch (exponential backoff) — Resolved

Status. Resolved and shipped. The exponential-backoff reconnect loop sketched at last review is implemented in `nodes/control_node.py`'s `_pose_stream` (Appendix I.4): on a dropped telemetry channel it retries at 1.0 s → 2.0 s → 4.0 s ... capped at 10.0 s, resetting to 1.0 s on the next successful reconnect. A momentary gRPC drop is now a self-healing hiccup rather than a flight-ending crash.

55.2 Centreline-guided planner: skeletonisation → waypoint chain — Resolved (built differently than planned)

Status. Resolved, but the mechanism that shipped is not the cross-track-guidance design originally sketched here. Cross-track guidance against a virtual synthetic line was an early validation step; it was not carried into the real pipeline. What the v7 planner actually does is: skeletonise the reachable road mask, prune the skeleton to its longest branch via a double-BFS tree-diameter search, and feed A* an ordered chain of 12 forward waypoints, advanced one at a time (§22.7–22.12; Appendix H.5a; code in Appendix I.3). This replaces the "A* hugs the shoulder" behaviour exactly as intended, and it is implemented and integrated into the planner, not merely validated in isolation.

55.3 Costmap and grid refinements (enabling future behaviour) — Still pending

What. (a) Add the "unknown" cell state to the occupancy grid (distinct from occupied) to enable pathing through gaps; (b) optionally add costmap shaping (edge-avoidance / centreline preference) so paths prefer the middle of the road. (Chapters 20, 24.)

Why it matters. The unknown state is the prerequisite for the occlusion-recovery theory (Chapter 53), which Chapter 53.7 explicitly records as "currently a theory, not yet implemented." Costmap shaping improves flight quality without changing A*, and complements the waypoint-chain planner above for cases where A* is used as the fallback.

Depends on. The current free/occupied grid (done). A* already supports costs, so no new planner is needed.

Next action. Extend the grid representation with the third state and make A* treat unknown as high-cost-traversable; then, separately, add edge/centreline costing if desired.

56. Perception Upgrades (parallel track)

56.1 Domain fine-tuning on real frames (correctness) — Still pending

What. Close the remaining domain gap by fine-tuning the MobileNetV2 + Gabor network (Part X, Chapters 39–43) — not "the U-Net" in the generic sense; the plain U-Net was replaced — on frames that actually resemble the real downward camera, so its masks are reliable on real data. (Chapters 43.5, 13.2.)

Status correction. This item is no longer "deferred in favour of HSV": `make_segmenter(mode="real")` already selects the trained network as the live segmenter on real hardware (Appendix I.5); HSV is the *simulation-only* fallback (`mode="sim"`). The open problem is not deployment, it is validation — the network reaches `val_road_iou = 0.80` on AeroScapes but has not been scored against real D455 imagery, because AeroScapes and the real camera remain out-of-distribution on camera, altitude, surface, and lighting (Chapter 43.5: "the model is fixed; the domain is not").

Why it matters. This is the correctness half of deploying the learned segmenter on real out-of-distribution imagery.

Depends on. Representative real frames (the RealSense is now capturing these, Chapter 33); ideally the USB 3.x fix first (§54.3) so training data is full-quality.

Next action. Collect labelled real nadir frames, fine-tune the already-deployed network, validate mask quality on held-out real frames.

56.2 U-Net acceleration via TensorRT (speed) — Resolved

Status. Resolved. FP16 TensorRT conversion is complete (Part XVI §62). End-to-end perception latency fell from ≈ 309 ms (3.2 Hz, plain TensorFlow) to ≈ 31.2 ms (≈ 32 FPS); inference itself dropped to 6.86 ms. The pipeline's bottleneck has since moved to CPU-side Gabor preprocessing (19.6 ms), not inference.

56.3 Texture as a second segmentation feature — Resolved

Status. Resolved — this is not a future addition, it is already part of the deployed architecture. The 8-kernel Gabor bank (4 orientations $\times$ 2 frequencies) is fused into the decoder at 128×128 resolution (Chapter 42; Appendix I.6), and was part of the network that reached `val_road_iou = 0.80` (Chapter 43.4). No further action needed on this item; §56.1 above is the one genuinely open perception task.

57. Original Research: Occlusion Recovery

57.1 Implement the road-occlusion recovery theory

- **What.** Build the two-part recovery (climb to widen FOV + bridge the gap), implemented as **unknown-not-obstacle grid cells + heading-guided A^*** , starting with **vector continuity** (hold last heading) and layering in geometric and edge continuity. (Chapter 53.)
- **Why it matters.** It is the difference between a follower that aborts when the road vanishes and one that flies through gaps — a genuine robustness capability, and an original contribution.
- **Depends on.** The unknown grid state (55.2); occlusion detection reusing the mask-zero diagnostic (6.4); the altitude-command path (already used for takeoff).
- **Next action (staged).** (1) unknown state + heading-biased A^* ; (2) detect occlusion (empty/split mask); (3) trigger altitude increase; (4) vector-continuity bridge; (5) add geometric/edge bridges; (6) test on deliberately gapped roads in sim first.

57.2 Write the theories into the notebook

- **What.** Record the **road-occlusion recovery theory** (and keep the notebook current with any new original theories as they arise) in the personal notebook of original theories. (Chapter 53.7; a standing reminder.)
- **Why it matters.** Formalising ideas when they are fresh is how they mature from insight to capability — and this specific theory was explicitly flagged to be written down.
- **Next action.** Transcribe Chapter 53 (problem, two-part recovery, three bridging methods, unknown-not-obstacle implementation, staged next steps) into the notebook.

58. The Roadmap as a Dependency Graph

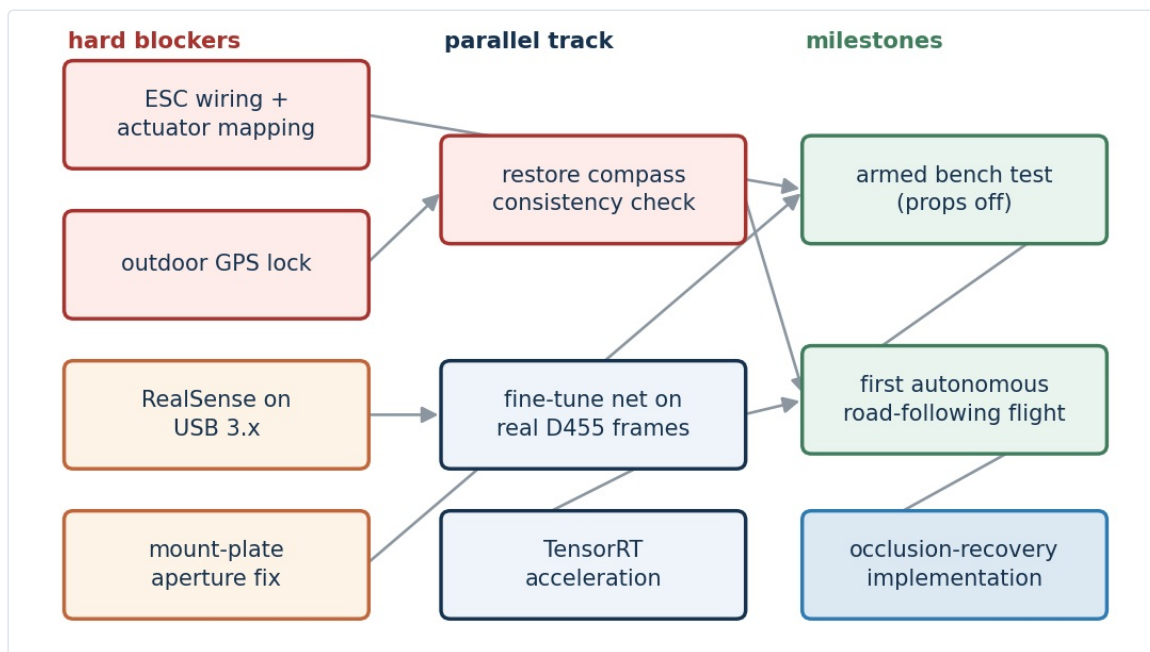
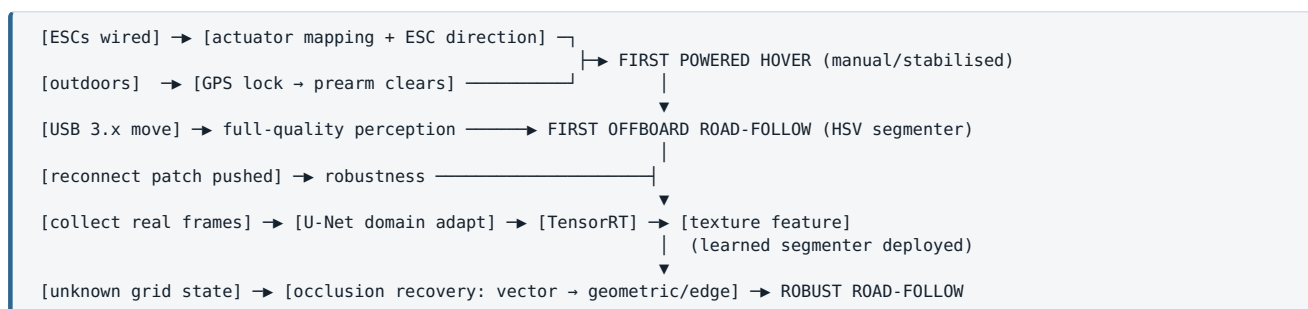


Figure 17. The remaining work as a dependency graph. Hard blockers on the left; nothing on this graph is an unsolved problem.

Reading the above as an ordering, the critical path to a first real autonomous flight is short and mostly physical:



The **top band** (actuator mapping + GPS) is the hard gate to *any* real flight and is purely physical/procedural — nothing more to design. The **middle band** (USB 3.x, reconnect patch) sharpens quality and robustness for the first HSV-based autonomous follow. The **bottom band** (U-Net adaptation → TensorRT → texture, then occlusion recovery) is the research track that upgrades the follower from "works on HSV in good conditions" to "semantic, robust, gap-tolerant." Crucially, the first two bands do not depend on the third: **the drone can fly the road autonomously on the HSV segmenter as soon as the hardware gates clear** — the learned-perception and occlusion work make it *better*, not *possible*.

Appendix A — Glossary

A compact reference for the terms used throughout, each defined as it applies in this project.

A* — An optimal, efficient graph-search pathfinding algorithm; Dijkstra guided by a heuristic. Expands the frontier cell with the lowest $f = g + h$. The project's planner. (Ch. 21)

Actuator mapping — Establishing which PX4 output channel drives which physical motor, so control allocation commands the right motor. A remaining task. (Ch. 31)

Admissible heuristic — A heuristic that never overestimates remaining cost; guarantees A* returns the shortest path. Straight-line distance is admissible on a grid. (Ch. 21)

Aeroscapes — An aerial semantic-segmentation dataset the U-Net was trained on; its mismatch with the real camera is the domain gap. (Ch. 16)

Airframe — The physical vehicle configuration (here quad-X); PX4 uses it to set control allocation and defaults. (Ch. 28)

Arming — PX4 permitting the motors to spin, gated by prearm checks. (Ch. 8)

asyncio — Python's cooperative-concurrency framework (event loop + coroutines) that MAVSDK-Python is built on; must be reconciled with ROS's executor. (Ch. 7)

Barometer — A pressure sensor giving altitude; fused by EKF2. On the Pixhawk. (Ch. 8, 27)

BGR / RGB — Byte order of colour channels; OpenCV defaults to BGR, ROS may use RGB. A mismatch swaps red/blue and breaks HSV thresholds. (Ch. 5, 14)

Callback — A function ROS calls automatically when a message arrives on a subscribed topic (or a timer fires), run by the executor. (Ch. 5)

colcon — The ROS 2 build tool that compiles/links a workspace. (Ch. 5)

Companion computer — The onboard Linux/GPU computer (Jetson Orin Nano) running the pipeline; the "brain." (Ch. 1, 33)

Control allocation / mixing — PX4 turning a roll/pitch/yaw/thrust demand into per-motor outputs, using the airframe layout and actuator mapping. (Ch. 8, 31)

Costmap — A grid where each cell has a traversal cost (not just free/blocked), letting the planner shape path style. (Ch. 20)

cv_bridge — Converts between ROS `sensor_msgs/Image` and OpenCV/NumPy arrays. (Ch. 5)

DDS — The industrial pub/sub standard ROS 2 is built on; provides discovery, transport, and QoS. (Ch. 5)

Domain gap — The mismatch between a model's training data and the data it faces at inference, degrading accuracy; the reason the U-Net is deferred. (Ch. 13, 16)

EKF2 — PX4's Extended Kalman Filter state estimator, fusing IMU/mag/baro/GPS into attitude and position; everything downstream trusts it. (Ch. 8)

ESC — Electronic Speed Controller; drives a motor at a commanded speed. Spin direction must be verified. (Ch. 31)

Executor — The ROS 2 component that runs callbacks; single-threaded by default (`rclpy.spin`). (Ch. 5, 7)

Failsafe — An automatic protective action (hold/return/land) when something goes wrong (lost RC/link, low battery, lost position, fence breach). (Ch. 8)

Flight mode — Determines where setpoints come from (manual, position, offboard, etc.). Offboard = setpoints from the companion computer. (Ch. 8)

FRD / NED / ENU — Coordinate conventions: body Front-Right-Down; world North-East-Down (Down positive); East-North-Up. Sign errors here send the drone the wrong way. (Ch. 3, 10)

Geofence — A configured boundary the vehicle must stay within; requires a position estimate, hence the "Fence requires position" prearm. (Ch. 32)

gRPC — The RPC channel between MAVSDK-Python and the MAVSDK server; its collapse under VM starvation caused the `_pose_stream` crash. (Ch. 7, 9)

Gyroscope — Measures angular rate for the rate controller; its resting bias must be calibrated out. (Ch. 8, 29)

Heuristic (h) — A*'s estimate of remaining cost to the goal; steers the search toward the goal. (Ch. 21)

HSV — Hue-Saturation-Value colour space; separates colour character (H, S) from brightness (V), making colour thresholds lighting-robust. The current segmenter. (Ch. 15)

IMU — Inertial Measurement Unit (accelerometer + gyroscope); on the Pixhawk, read at high rate. (Ch. 8, 27)

JetPack — NVIDIA's software stack for Jetson; JetPack 6.x provides Ubuntu 22.04.5 + L4T R36.3.0. (Ch. 33)

Kill switch — A hardware motor cut-off independent of software; the ultimate safety backstop. (Ch. 30)

L4T — Linux for Tegra, NVIDIA's tuned Linux for Jetson; R36.3.0 here. (Ch. 33)

LOOKAHEAD — Tuning parameter: how far ahead along the road to aim; trades cornering precision vs smoothness. (Ch. 24)

MAVLink — The lightweight binary messaging protocol between the flight controller and off-board computers/ground stations. (Ch. 9)

MAVSDK — A clean, plugin-based API over MAVLink; MAVSDK-Python is async and talks to a MAVSDK server over gRPC. (Ch. 9)

Nadir — Straight-down viewing direction; the camera's orientation, which makes road-following a 2-D image problem. (Ch. 14)

Occupancy grid — The planner's map: cells free (road) or occupied (not-road), derived from the segmentation mask. (Ch. 20)

Offboard mode — PX4 mode where setpoints stream from the companion computer; requires continuous setpoints ($\geq \sim 2$ Hz) or it drops out. (Ch. 8, 10)

ONNX — An interchange format used to move the trained model from TensorFlow into TensorRT. (Ch. 17)

Prearm checks — PX4's safety chain that must pass before arming (sensors, estimator, RC, no failsafe, fence position). (Ch. 8)

QoS (Quality of Service) — DDS/ROS 2 settings (reliability, durability, history) that must be compatible between publisher and subscriber or no messages flow. (Ch. 5)

REACHED — Tuning parameter: the radius within which a waypoint counts as reached, triggering advance to the next. (Ch. 23, 24)

RealSense D455 — Intel stereo depth camera (RGB + depth over USB) used as the real downward sensor; needs USB 3.x for full bandwidth. (Ch. 18)

Semantic segmentation — Assigning a class label to every pixel; here, road vs not-road. The basis of the "semantic" in the project's name. (Ch. 16)

SITL — Software-In-The-Loop: running real PX4 firmware with simulated sensors/motors, connected to Gazebo. (Ch. 12)

Skip connection — U-Net's link copying high-resolution encoder features to the decoder, fusing "what" and "where" for sharp masks. (Ch. 16)

TensorRT — NVIDIA's inference optimiser/runtime that compiles a trained network into a fast engine for a specific GPU (FP16/INT8). (Ch. 17)

Topic — A named, typed ROS message channel; nodes publish to and subscribe from topics. (Ch. 5)

U-Net — An encoder-decoder CNN with skip connections for semantic segmentation; trained but deferred pending domain adaptation + acceleration. (Ch. 16)

uORB — PX4's internal publish/subscribe bus between its modules. (Ch. 8)

Vector continuity — Occlusion-recovery bridge that holds the last known heading across a gap; the simplest, most robust bridge. (Ch. 53)

Waypoint — A point in the world frame the controller flies to; a path becomes a sequence of waypoints, popped as reached. (Ch. 23)

GOAL_HYSTERESIS — Tuning parameter: resistance to changing the goal/plan on marginal differences; the cure for replan jitter. (Ch. 24)

Appendix B — Command Reference

The commands and calls actually used across development, grouped by tool. Illustrative forms; exact names/paths depend on the workspace.

B.1 ROS 2 (introspection and running)

```
# Build & source the workspace
colcon build --symlink-install
source install/setup.bash

# Run a single node
ros2 run semantic_drone camera_node
ros2 run semantic_drone segmentation_node
ros2 run semantic_drone planner_node
ros2 run semantic_drone control_node

# See the graph
ros2 node list
ros2 node info /planner_node

# Inspect topics
ros2 topic list
ros2 topic echo /mavsdk/pose           # watch actual messages
ros2 topic hz /image                  # measure real publish rate
ros2 topic info /image -v             # SHOWS QoS on each end (catches mismatches)
```

B.2 The three diagnostics this project relies on

```
# Is the camera actually publishing at the expected rate?
ros2 topic hz /image

# Do publisher and subscriber QoS match? (the silent "no messages" cause)
ros2 topic info /image -v

# Watch the perception health number (road%) and planner mask-zero logs
# -> these are emitted by the nodes themselves (segmentation every 30 frames)
```

B.3 MAVSDK-Python (essential calls)

```
from mavsdk import System
from mavsdk.offboard import PositionNedYaw

drone = System()
await drone.connect(system_address='udp://:14540') # sim
# await drone.connect(system_address='serial:///dev/ttyACM0:57600') # hardware

await drone.action.set_takeoff_altitude(20.0)
await drone.action.arm()
await drone.action.takeoff()

# Offboard: prime a setpoint BEFORE start()
await drone.offboard.set_position_ned(PositionNedYaw(0, 0, -20, 0))
await drone.offboard.start()
# ...then stream setpoints continuously...

# Telemetry (async streams)
async for pos in drone.telemetry.position_velocity_ned():
    ...
async for mode in drone.telemetry.flight_mode():
    ...

# Parameters
await drone.param.set_param_int('COM_RCL_EXCEPT', 4)
await drone.param.set_param_int('NAV_DLL_ACT', 0)

await drone.action.land()
```

B.4 QGroundControl (GUI, not CLI) — what it is used for

- **Firmware:** flash PX4 1.17.0 over USB.
- **Airframe:** select quad-X.
- **Sensors:** accelerometer, gyroscope, compass (magnetometer) calibration flows.
- **Radio:** RC calibration; assign flight-mode switches and the kill switch.
- **Parameters:** set/inspect `COM_RCL_EXCEPT`, `NAV_DLL_ACT`, and read prearm warnings (e.g. "Fence requires position").
- **Actuators/Motors:** motor test (props off) for spin-direction verification.

B.5 Simulation

```
# Launch PX4 SITL with the x500 (illustrative)
make px4_sitl gz_x500
# MAVSDK connects to SITL on udp://:14540, same code as hardware.
# gz_camera_bridge bridges x500_mono_cam_down_0 into ROS 2.
```

B.6 Jetson / build sanity

```
# Confirm architecture (Jetson is ARM64/aarch64)
uname -m          # -> aarch64

# RealSense: confirm topics are live after driver install
ros2 topic list | grep -i color
ros2 topic hz /camera/color/image_raw

# If a package repo GPG key is missing/expired during install,
# import the correct signing key, then retry the install.
```

Appendix C — Parameter and Threshold Reference

Every tuned value, calibration, and configured parameter recorded in the project, in one table, with its meaning and where it is discussed.

C.1 PX4 behavioural parameters

Parameter	Value	Meaning / why	Ref
COM_RCL_EXCEPT	4	Exempt offboard from the RC-loss failsafe, so no RC transmitter is required for autonomy.	Ch. 11
NAV_DLL_ACT	0	Data-link-loss failsafe action = disabled , so a momentary GCS-link loss doesn't trigger return/land during bring-up.	Ch. 11

(Both are deliberate bring-up relaxations, backstopped by the kill switch; revisit for real deployment.)

C.2 Calibrations completed (stored as parameters)

Calibration	Purpose	Ref
Accelerometer	Correct bias/scale; know "down"; hold level attitude.	Ch. 29
Gyroscope	Zero the resting bias; clean angular-rate for the rate controller.	Ch. 29
Compass (magnetometer)	Correct motor/wiring magnetic distortion; true heading for GPS modes.	Ch. 29
RC	Map stick/switch endpoints and centres; enable mode + kill-switch assignment.	Ch. 29

C.3 Firmware / platform versions

Item	Value	Ref
PX4 firmware	1.17.0	Ch. 28
Flight controller	Pixhawk 6C Mini	Ch. 27
Companion computer	Jetson Orin Nano 8GB	Ch. 33
JetPack	6.x	Ch. 33
OS	Ubuntu 22.04.5 LTS	Ch. 33
L4T	R36.3.0	Ch. 33
ROS 2	Humble	Ch. 5, 33
Depth camera	Intel RealSense D455	Ch. 18

C.4 HSV segmentation thresholds

Quantity	Value	Note	Ref
Measured road H	136–140	From a real nadir frame	Ch. 15
Measured road S	21–24	Low = greyish road	Ch. 15
Measured road V	109–135	Mid brightness	Ch. 15
Lower threshold	[100, 0, 50]	H_lo, S_lo, V_lo (wide hue, admit low S, forgiving V)	Ch. 15
Upper threshold	[170, 50, 180]	H_hi, S_hi, V_hi (S ceiling rejects vivid regions)	Ch. 15
Road% diagnostic	every 30 frames	Perception health monitor	Ch. 6, 15

C.5 Planner tuning parameters (behavioural)

Parameter	Role	Trade-off	Ref
<code>LOOKAHEAD</code>	How far ahead to aim	Larger = smoother/anticipatory but cuts corners; smaller = tight but twitchy	Ch. 24
<code>REACHED</code>	Radius counting a waypoint as reached	Larger = smooth advance but cuts corners; smaller = precise but may stall	Ch. 24
<code>GOAL_HYSTERESIS</code>	Resistance to changing the goal/plan	Higher = stable/committed but slow to adopt better plans; lower = responsive but jittery	Ch. 24
Auto-goal region	Ignore top 20% of image	Avoid unreliable far-frame pixels	Ch. 6, 22
Altitude gate	Don't follow until at altitude	Sequence "get up" before "follow"	Ch. 10, 24
Takeoff altitude	20.0 m (default)	Operating altitude for the downward view	Ch. 6, 10
Takeoff timeout	60 s (<code>asyncio.wait_for</code>)	Fail cleanly if takeoff stalls	Ch. 6, 10

C.6 Connection addresses

Context	Address	Ref
Simulation (SITL)	<code>udp://:14540</code>	Ch. 9, 12
Hardware	serial (e.g. <code>serial:///dev/ttyACM0:57600</code>)	Ch. 9

Appendix D — Troubleshooting Log (Bugs Found and Fixed)

A running record of concrete problems encountered and their resolutions — the most practically useful appendix, because these are the exact traps that will recur. Each entry: **symptom** → **cause** → **fix** → **lesson**.

D.1 `core` package imports fail under `ros2 run`

- **Symptom.** Code that ran when executed directly failed with import errors under `ros2 run`.
- **Cause.** The `core` library package was missing from `setup.py`, so it was never installed into the ROS environment. (Ch. 5)
- **Fix.** Add `core/` to the packages installed by `setup.py`.
- **Lesson.** If a module works standalone but not under `ros2 run`, suspect it was never installed — check `setup.py`'s package list.

D.2 No images reaching the segmenter

- **Symptom.** Segmentation callback never fires; no frames.
- **Cause (two forms).** (1) **Topic-name drift** — the code and the Gazebo model disagreed on the camera topic (`x500_mono_cam_0` vs `x500_mono_cam_down_0`). (2) **QoS mismatch** — a `BEST_EFFORT` publisher and a `RELIABLE` subscriber silently do not connect. (Ch. 5, 6, 12)
- **Fix.** Rename both the camera node and the `gz_camera_bridge` to the same `x500_mono_cam_down_0`; add `RELIABLE` QoS so producer and consumer match.
- **Lesson.** "Callback never fires" ≈ wrong topic name or incompatible QoS. Check `ros2 topic info -v` for QoS and confirm both sides name the same topic.

D.3 HSV segmentation "stops working" after a camera change

- **Symptom.** Road% drops to near zero; mask empty despite road in view.
- **Cause.** Either a **BGR/RGB encoding swap** (red/blue flipped, so colours fall outside the HSV box) or thresholds tuned for a different domain (sim vs real). (Ch. 13, 14, 15)
- **Fix.** Ensure `cv_bridge` `desired_encoding` matches reality; **re-measure HSV thresholds on a real frame** (yielded H 136–140, S 21–24, V 109–135 → bounds `[100,0,50]`–`[170,50,180]`).
- **Lesson.** Measure thresholds on your own data; a silent channel swap is a top suspect when colour segmentation fails after any camera change.

D.4 `asyncio` ↔ ROS "everything just stops"

- **Symptom.** Pose stops updating, callbacks stop firing, drone stops responding — looks like several bugs.
- **Cause.** **One thread, two event loops.** A blocking operation (sync sleep, heavy compute, or a stuck `await`) with no yield **starves the cooperative event loop**, freezing ROS callbacks and MAVSDK coroutines alike. (Ch. 7)
- **Fix.** Async-first structure: drive ROS with non-blocking `spin_once` inside the `asyncio` loop with `await` `asyncio.sleep(...)` yields; bound MAVSDK waits (`asyncio.wait_for(..., 60)`) so one stuck `await` can't hang the loop.
- **Lesson.** In cooperative concurrency every task must yield and none may block. "Everything stopped" ≈ starved event loop.

D.5 gRPC / `_pose_stream` crash

- **Symptom.** The telemetry stream dies with a gRPC error, ending the run.

- **Cause. VM resource starvation, not a code bug** — a starved simulation VM couldn't sustain the gRPC telemetry channel, which collapsed and surfaced in `_pose_stream` (where the stream is consumed). (Ch. 7, 9)
- **Fix.** Run on capable hardware (the native cobot desktop avoided it); **design a reconnect patch with exponential backoff** (not yet pushed) so a dropped channel self-heals.
- **Lesson.** Where a failure *appears* is not where it *comes from*. Don't rewrite the victim (the stream code was correct); fix the environment and add resilience.

D.6 A* returns "no path"

- **Symptom.** Planner produces no path; drone won't move.
- **Cause (two forms).** (1) **Empty grid** — segmentation found no road, so the grid is all-occupied (A* correctly finds nothing). (2) **Goal on an occupied cell** — `_find_auto_goal` picked a non-road cell, which A* can never reach. (Ch. 21, 22)
- **Fix.** (1) A **mask-zero diagnostic** identifies "no road detected" vs a planner fault. (2) `_nearest_free_goal()` **radial spiral search** snaps the goal to the nearest free cell before running A*.
- **Lesson.** When A* says "no path," the *map* or the *goal* is wrong, not A*. Never hand it an unreachable goal.

D.7 Drone stuck on a waypoint / doesn't advance

- **Symptom.** The drone hovers at or near a point and never proceeds along the path.
- **Cause. Missing waypoint popping** — reached waypoints weren't removed, so the "current target" never advanced. (A too-small REACHED radius causes the same symptom by never letting a waypoint count as reached.) (Ch. 23)
- **Fix.** Correctly **pop** a waypoint once within the REACHED radius; tune REACHED so arrival is achievable.
- **Lesson.** Navigation needs an explicit, correct "arrived → next target" transition; getting it wrong strands the vehicle.

D.8 Goal-on-obstacle after finding the auto-goal (Jetson bring-up)

- **Symptom.** Auto-goal lands on a non-free cell; planning fails.
- **Cause.** `_find_auto_goal` could return a cell on an obstacle. (Ch. 6, 22)
- **Fix.** `_nearest_free_goal()` **radial spiral search** for the nearest free cell (also restrict goal selection to the reliable lower 80% of the image).
- **Lesson.** Same as D.6 — snap invalid targets to the nearest valid one; defensive planning.

D.9 Incorrect occupancy grid logic

- **Symptom.** A* produces wrong or impossible paths even though the mask looks fine.
- **Cause.** The **mask→grid mapping** marked free/occupied cells incorrectly, so A* searched a map that didn't match the camera. (Ch. 20)
- **Fix.** Correct the grid construction: road → free, non-road → occupied, consistently.
- **Lesson.** Most "planner bugs" are *map* bugs. Verify the grid matches the mask before blaming the search.

D.10 MAVSDK `connect()` hangs on the bench

- **Symptom.** Connection never completes; code hangs at startup.
- **Cause.** The default connect flow **waited for health checks** (GPS, home, calibration) that never pass on a bench without GPS/home. (Ch. 6, 8)
- **Fix.** **Remove the health-check gate** from `connect()` so bring-up proceeds; rely on PX4's own prearm and the operator for safety.
- **Lesson.** Client-side "wait until healthy" gates hang in exactly the environments where you're trying to bring the system

up; the real safety lives in PX4's prearm chain.

D.11 RealSense streams degraded

- **Symptom.** Camera works but at reduced resolution/rate.
- **Cause.** **USB bandwidth cap at USB 2.1** — colour+depth need USB 3.x; on a slower link the device silently downgrades. (Ch. 18)
- **Fix.** Move to a **true USB 3.x port** with an adequate cable (pending).
- **Lesson.** A "working but worse" sensor can be a bandwidth problem, not a driver problem — check the negotiated USB speed.

D.12 RealSense driver won't install (GPG key)

- **Symptom.** Package manager refuses the RealSense repository during install.
- **Cause.** **Missing/expired GPG signing key** for the package repository. (Ch. 33)
- **Fix.** Import the correct signing key, then retry the install; topics then confirmed live.
- **Lesson.** Signed-repo install failures are usually a key problem — fix the key, not the packages.

D.13 "Fence requires position" prearm blocks arming

- **Symptom.** Vehicle refuses to arm; QGC shows "Fence requires position."
- **Cause.** A **geofence is configured but there's no position estimate** (no GPS lock indoors), so PX4 won't arm a position-limited vehicle that doesn't know where it is. (Ch. 32)
- **Fix.** Obtain an **outdoor GPS lock** so EKF2 has a valid position (pending — procedural, not code).
- **Lesson.** This prearm is the safety chain working correctly, not a bug; it never appears in sim because SITL's GPS is always fixed.

D.14 Compass inconsistency, 128° between internal and external

- **Symptom.** Bench arming refused with a compass-inconsistency prearm; internal and external magnetometers reported headings ~128° apart. (Ch. 38.1)
- **Cause.** Two independent magnetometers seeing very different fields — internal compass sits inside a metal-and-current-filled airframe, external compass on a mast, and one or both are affected by an orientation-setting mismatch or nearby ferrous material.
- **Fix (bench expedient).** Set `COM_ARM_MAG_ANG = -1` to disable the consistency check. Permits bench work where heading accuracy is irrelevant.
- **Fix (proper — pending before outdoor flight).** Re-run compass calibration in situ; confirm the external compass's orientation setting; consider disabling the internal compass entirely (standard practice on airframes with high current near the flight controller).
- **Lesson.** Prearm-check bypasses on a bench are legitimate; leaving them in place for outdoor flight is not. Every disabled check is a hole in the safety chain and must be re-enabled before it matters.

D.15 "Vehicle stuck in Position mode; won't arm"

- **Symptom.** Aircraft appeared to be in Position flight mode, which requires GPS; would not arm indoors. Attempts to switch out of Position had no effect.
- **Cause.** **Not stuck — misconfigured switch mapping.** Flight Mode 1 = Position, Flight Mode 4 = Altitude, one position = Return. Stabilized was never assigned to any reachable switch position, so there was no way to *select* an indoor-usable mode. Position was the only thing the switch could ask for in that slot, and Position needed GPS. (Ch. 38.3)

- **Fix.** Reassign Flight Mode 1 to **Stabilized**, so the switch can reliably reach a mode that needs neither GPS nor altitude sensing.
- **Lesson.** A configuration bug can present as a hardware bug. The vehicle was correctly refusing an impossible request; the fault was that the operator had no way to make a possible one.

D.16 U-Net OOM: 1.04 GiB allocation fails on the Jetson

- **Symptom.** Training / inference of the original plain U-Net fails on the Orin Nano with a CUDA `out of memory` error; the specific failing allocation is 1.04 GiB. (Ch. 41.1)
- **Cause.** The plain U-Net at input resolution has ~ 7.7 M parameters and activations that, together with cuDNN workspace, exceed the Jetson's practical budget. 8 GB is shared between OS, ROS, the camera pipeline, and the network — the naive number is misleading.
- **Fix.** Replace the encoder with **MobileNetV2 (~ 2.3 M backbone params)**, `alpha=1.0`, ImageNet-pretrained. (Chapter 41.2.) Resulting model fits and trains within the budget.
- **Non-fix considered and rejected.** ResNet-50 encoder — larger than plain U-Net, would have failed harder. Chosen against on evidence, before wasting a training run.
- **Lesson.** The deployment target is a design constraint, not a deployment detail. Choose an architecture that fits the device's *demonstrated* memory ceiling, not its nameplate figure.

D.17 Keras 3 `.keras` file will not load under Keras 2.15

- **Symptom.** Model file transferred from workstation to Jetson; `keras.models.load_model()` fails with a version-related deserialisation error. (Ch. 41.4)
- **Cause.** Training environment uses Keras 3; Jetson runs Keras 2.15 (bundled with the JetPack TensorFlow 2.15). A Keras 3 `.keras` archive is a **versioned object graph** and does not load under 2.15.
- **Fix.** Ship `.weights.h5` + **rebuild architecture in code** on the Jetson (`build_mobilenet_unet_gabor()` in Appendix I.4), then `load_weights()`. Weights are just arrays and cross versions cleanly.
- **Lesson.** For deployment across Keras major versions, prefer weights over serialised models. This applies to every model whose training and inference environments are on different Python stacks.

D.18 protobuf collision on the Jetson

- **Symptom.** `import tensorflow` succeeds, `import mavsdk` succeeds, using them together in one process breaks with protobuf incompatibility errors — or `pip install` refuses the combination outright. (Ch. 44–45)
- **Cause.** TensorFlow 2.15 requires `protobuf < 5`; MAVSDK-Python 3.15.3 requires `protobuf >= 6`. The intersection is empty; no protobuf version satisfies both. gRPC (via MAVSDK) and TF both use protobuf, and both must run in one process on the Jetson.
- **Fix.** Downgrade MAVSDK to 2.8.4, pin `protobuf == 3.20.3`. Verified against the real Pixhawk over USB, not merely by import.
- **Follow-up.** Fragile pin — if either library breaks in future, check protobuf first and re-pin. A container image or exact `requirements.txt` is the durable fix.
- **Lesson.** When two libraries with a shared transitive dependency disagree on its version, resolution is not a `pip` problem; it is a version-pair search that must be verified against the actual function being used, not just against import success.

D.19 4.5 Hz camera on distributed SITL

- **Symptom.** In the distributed SITL setup (Gazebo on a UTM VM, pipeline on the Jetson), simulated camera topics arrive on the Jetson at ~ 4.5 Hz — an order of magnitude slower than expected. Downstream stages idle between frames. (Ch. 49)

- **Innocent suspects.** `gz_camera_bridge`, the network between VM and Jetson, USB (not in the sim data path), and protobuf were each investigated and each acquitted.
- **Actual cause.** Gazebo running at ~20% real time because `LIBGL_ALWAYS_SOFTWARE=1` in the VM forces software OpenGL rendering; the Mac's GPU is not exposed to the guest, and CPU rendering is one to two orders of magnitude slower.
- **Fix.** Faster sim host — native Ubuntu with a real GPU (`cobot`), or lower camera resolution/`update_rate` in the SDF, or a simpler world. Not code.
- **Lesson.** A slow pipeline stage is often a symptom of upstream starvation. Investigate upstream, from consumer toward producer, measuring the source rate before optimising any consumer. This is the same failure signature as the gRPC crash of D.5, in a different organ.

D.20 Duplicate ROS apt repository

- **Symptom.** `apt` warnings; ambiguous package resolution during Jetson package installs. (Ch. 46.1)
- **Cause.** The ROS 2 apt repository was configured **twice**, through two separate source-list files, so `apt` had two identically-named sources to choose between.
- **Fix.** Remove the redundant `ros2.list`, leaving a single authoritative source.
- **Lesson.** Repository hygiene on an embedded system that must boot reliably is worth the ten minutes.

D.21 `setuptools` too new for ROS 2 Humble

- **Symptom.** `colcon build` fails or emits deprecation errors on Python packages that used to build cleanly. (Ch. 46.2)
- **Cause.** `setuptools` 83 is far newer than what ROS 2 Humble's `ament_python` expects — the packaging interfaces the extension relies on have been deprecated or removed upstream.
- **Fix.** Downgrade `setuptools` to the **79.x** series.
- **Lesson.** A ROS 2 distribution is pinned to an era of the Python ecosystem. Upgrading the toolchain underneath it is not an improvement; it is a change of assumptions.

D.22 Gazebo `gz.transport` / `gz.msgs` import error

- **Symptom.** Bridge code fails with `ModuleNotFoundError: No module named 'gz.transport'` (or `gz.msgs`), even though the package is installed. (Ch. 46.4)
- **Cause.** Gazebo's Python packages carry their ABI version in the module name (e.g. `gz.transport13`, `gz.msgs10.image_pb2`). The unversioned alias may not exist.
- **Fix.** Import the **explicitly versioned module names**: `from gz.transport13 import Node`, `from gz.msgs10.image_pb2 import Image`. No virtual-environment workaround required.
- **Lesson.** Gazebo Python bindings look like a missing installation when the version-suffixed name is not used. Check what is actually installed before adding a venv layer.

D.23 Empty road masks — three stacked faults

- **Symptom.** Trained U-Net produces **near-empty** road masks on validation imagery. Downstream, A* sees an all-occupied costmap and reports no path. (Ch. 40)
- **Plausible-but-wrong cause.** Domain gap between AeroScapes (real aerial) and Gazebo (rendered).
- **Actual causes — three, stacked.** 1. **Only the first 30 masks were scanned** when computing class statistics. AeroScapes' 3269 images are not shuffled, so 30 consecutive frames are an unrepresentative sliver, and every conclusion drawn from them was corrupt. 2. `ROAD_CLASS = 1`, which is `person` in AeroScapes. Road is index 10. The network was trained and scored on the wrong class — an off-by-nine error in a single constant. 3. **Plain cross-entropy on a pixel-minority class** rewards predicting road nowhere.

- **Fix.** (1) Scan the full mask set. (2) Set `ROAD_CLASS = 10`. (3) Use **Dice + cross-entropy** (Appendix I.5). Combined with the MobileNetV2 encoder (D.16), road IoU rises from ~ 0 to **0.80** on validation.
- **Lesson.** Before blaming the model, verify that the **data**, the **labels**, and the **objective** describe the problem you think you are solving. Empty predictions for a class are far more often a label-index or class-imbalance bug than an architectural inadequacy — and stacked bugs mutually camouflage, which is why fixing one appears to change nothing.

D.24 A* goal on the road edge — drone hugs the shoulder

- **Symptom.** Planner returns paths that hug the left or right edge of the road strip; goal repeatedly lands at coordinates like `(86, 9)` on a straight section. Drone follows the shoulder instead of the centre. (Ch. 22.6)
- **Cause.** `_find_auto_goal` selects the **topmost reachable road pixel**, which on a vertical road strip is a corner pixel of the reachable region, not its horizontal midpoint. A* correctly plans to the requested goal; the fault is in the choice of goal.
- **Fix.** Replace the topmost-reachable step with the **argmax of the Euclidean distance transform** within the top forward slice (Appendix I.2, `centerline_goal.py`). The chosen pixel sits on the road's medial axis by construction.
- **Lesson.** "Path hugs an edge" is a *goal-selection* symptom, not a *pathfinding* one. Search behaves correctly under the specification it receives; the specification was wrong.

Closing note. Read as a set, these entries make the project's recurring failure signatures obvious: *no messages* (topic/QoS), *no path* (map/goal), *everything froze* (starved async loop), *works standalone but not in ROS* (setup.py), *works but worse* (USB bandwidth), and *won't arm* (prearm doing its job). Recognising which signature a new problem matches is the fastest route to its fix — and is, in the end, what all these notes are for.

Appendix E — Source Register

Everything this project was built on, with what each source was actually used for. This is the bibliography in the academic sense: it records provenance, so that any claim, method, or tool in this document can be traced to its origin.

A note on figures. All diagrams and illustrations in this document are **original**, generated for it. No figure, screenshot, dataset sample, or diagram has been reproduced from any of the sources below. Where a source's *idea* is used, it is cited here and in the relevant chapter.

E.1 Datasets

AeroScapes — the aerial semantic segmentation benchmark the road network is trained on. 3269 images at 720p, captured by drone from **5–50 m altitude**, annotated across 11 semantic classes (plus background = 12). Used for: all U-Net training, both the original and the rebuilt MobileNetV2 network (Chapters 16, 39–43). The class-index convention of this dataset is the source of the project's most consequential bug: **road is index 10, not 1** (index 1 is `person`).

Nigam, I., Huang, C., & Ramanan, D. (2018). *Ensemble Knowledge Transfer for Semantic Segmentation*. IEEE Winter Conference on Applications of Computer Vision (WACV). Dataset: <https://github.com/ishann/aeroscapes>

The dataset's own framing is directly relevant to this project: it was introduced specifically to study **domain shift** — training and testing on distributions that differ in scene structure, viewpoint, and object statistics. That is precisely the problem encountered in Chapter 13 and Chapter 43.5.

E.2 Foundational papers and methods

Method	Source	Used for
A* search	Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). <i>A Formal Basis for the Heuristic Determination of Minimum Cost Paths</i> . IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.	The path planner over the semantic costmap (Ch. 21)
U-Net	Ronneberger, O., Fischer, P., & Brox, T. (2015). <i>U-Net: Convolutional Networks for Biomedical Image Segmentation</i> . MICCAI.	The segmentation architecture — encoder/decoder with skip connections (Ch. 16, 41)
MobileNetV2	Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). <i>MobileNetV2: Inverted Residuals and Linear Bottlenecks</i> . CVPR.	The pretrained encoder chosen for the Jetson's memory budget (Ch. 41)
Dice loss	Milletari, F., Navab, N., & Ahmadi, S.-A. (2016). <i>V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation</i> . 3DV.	The class-imbalance-aware loss term (Ch. 43)
Gabor filters	Gabor, D. (1946). <i>Theory of Communication</i> . J. IEE, 93(26), 429–457. Applied to texture via, e.g., Jain, A. K., & Farrokhnia, F. (1991). <i>Unsupervised Texture Segmentation Using Gabor Filters</i> . Pattern Recognition, 24(12).	The texture channel fused into the decoder (Ch. 42)
Pure Pursuit	Coulter, R. C. (1992). <i>Implementation of the Pure Pursuit Path Tracking Algorithm</i> . Carnegie Mellon University, Robotics Institute, CMU-RI-TR-92-01.	The path-tracking controller and its <code>LOOKAHEAD</code> parameter (Ch. 24, 25)
Extended Kalman Filter	Standard estimation theory; as implemented in PX4's <code>EKF2</code> module.	State estimation — the attitude and position every controller trusts (Ch. 8)
ImageNet pretraining	Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). <i>ImageNet: A Large-Scale Hierarchical Image Database</i> . CVPR.	The source of the MobileNetV2 encoder's pretrained weights (Ch. 41.2)
Morphological skeletonisation	Zhang, T. Y., & Suen, C. Y. (1984). <i>A Fast Parallel Algorithm for Thinning Digital Patterns</i> . Communications of the ACM, 27(3), 236–239. Implemented in <code>cv2.ximgproc.thinning</code> (opencv-contrib) with a pure-numpy fallback in <code>core/skeleton_goal.py</code> .	Extraction of the road's medial axis as a one-pixel-wide connected curve

		(§22.7)
Tree-diameter double-BFS	Standard graph-theory result: the diameter of a tree is found by two breadth-first passes. See e.g. Cormen, Leiserson, Rivest & Stein, <i>Introduction to Algorithms</i> (3rd ed., 2009), Exercise 22.2-8.	The <code>main_branch()</code> longest-path pruner in <code>skeleton_goal.py</code> (§22.8)
Gabor filters and texture analysis	Gabor, D. (1946). <i>Theory of Communication</i> . J. IEE, 93(26), 429–457. Bank-of-Gabor texture energy: Jain, A. K., & Farrokhnia, F. (1991). <i>Unsupervised Texture Segmentation Using Gabor Filters</i> . Pattern Recognition, 24(12), 1167–1186. Implemented via <code>cv2.getGaborKernel</code> in <code>core/mobilenet_unet.py</code> .	The 8-kernel texture-energy channel fused into the U-Net decoder (§42.4, Appendix I.6)

E.3 Frameworks, middleware, and toolchains

Software	Version	Role	Documentation
PX4 Autopilot	1.17.0	Flight-control firmware on the Pixhawk; SITL for simulation	https://docs.px4.io
ROS 2	Humble Hawksbill	Middleware: nodes, topics, QoS, executors, <code>colcon</code>	https://docs.ros.org/en/humble/
MAVSDK-Python	2.8.4 (downgraded from 3.15.3)	Companion-computer API to the flight controller over MAVLink/gRPC	https://mavsdk.mavlink.io
MAVLink	—	The wire protocol between companion computer and flight controller	https://mavlink.io
Gazebo	(with <code>gz.transport13</code> , <code>gz.msgs10</code>)	Physics + rendering simulator; Baylands world; <code>x500</code> airframe model	https://gazebo.org
QGroundControl	—	Ground station: firmware flashing, calibration, parameters, actuator tests	https://qgroundcontrol.com
TensorFlow	2.15 (NVIDIA JetPack apt repo)	Training and inference for the segmentation network	https://www.tensorflow.org
Keras	3 (training) / 2.15 (Jetson)	Model definition; the version split forces <code>.weights.h5</code> (Ch. 41.4)	https://keras.io
TensorRT	(JetPack)	Planned inference acceleration on the Jetson (Ch. 17)	https://developer.nvidia.com/tensorrt
OpenCV	—	Image handling, HSV conversion, Gabor kernels, morphology	https://opencv.org
NumPy	—	Array mathematics throughout	https://numpy.org
protobuf	3.20.3 (pinned)	Serialisation shared by TensorFlow and gRPC/MAVSDK; the collision of Ch. 44	https://protobuf.dev
setuptools	79.x (downgraded from 83)	Python packaging; newer versions break ROS 2 Humble builds (Ch. 46.2)	—
OpenSCAD	—	Parametric CAD for the mount plate (Ch. 37)	https://openscad.org
Intel RealSense SDK	—	Driver and ROS 2 wrapper for the D455	https://github.com/IntelRealSense/librealsense

E.4 Hardware and its documentation

Component	Specification	Documentation source
Pixhawk 6C Mini	Flight controller; no physical safety button	Holybro / Pixhawk standards, PX4 hardware docs
NVIDIA Jetson Orin Nano 8GB	Companion computer; JetPack 6.x, Ubuntu 22.04.5, L4T R36.3.0; UART1 on pins 8/10 of the J12 40-pin header → <code>/dev/ttyTHS1</code> ; 19 V barrel jack (J16)	NVIDIA Jetson Orin Nano Developer Kit Carrier Board specification
Intel RealSense D455	Stereo depth + RGB; measured intrinsics at 640×480: <code>fx = fy = 388.162</code> , <code>cx = 322.326</code> , <code>cy = 239.442</code>	Intel RealSense D400 series datasheet

Simplify 40A ESCs	×4, DShot-capable; run at DSHOT600	Manufacturer specification
6S 12000 mAh LiPo	~22.2 V nominal; feeds ESCs, Pixhawk power module, and the Jetson UBEC	—
UBEC (DC-DC buck)	6S → regulated 19 V for the Jetson barrel jack	—
DShot protocol	Digital ESC protocol; removes the need for PWM ESC calibration	Open protocol specification, as implemented in PX4

E.5 The project's own artefacts

Artefact	Description
<code>ACHYUT13807/semantic-drone</code>	The project repository (private). Nodes: <code>camera_node</code> , <code>segmentation_node</code> , <code>planner_node</code> , <code>control_node</code> , <code>gz_camera_bridge</code> . Core library: <code>astar.py</code> , <code>utils.py</code> , <code>road.py</code> , <code>projection.py</code> , <code>temporal.py</code> , <code>px4.py</code>
<code>semantic_drone_v5.zip</code> / v6 code	Snapshots capturing the HSV recalibration, <code>/mavsdk/pose</code> migration, and the planner robustness fixes (nearest-free-cell recovery, goal hysteresis, minimum goal distance)
<code>train_unet_mobilenet_gabor.py</code>	The rebuilt training script: MobileNetV2 encoder, Gabor fusion, Dice+CE loss, per-class IoU
Faculty presentation	11 slides, Ocean-Gradient palette, plus a rehearsal cheat-sheet with Q&A preparation
This document	The Prototype 1-α technical reference

E.6 Intellectual sources of the original contribution

The road-occlusion recovery theory (Part XIV) is an original synthesis of the following established ideas:

- **Curve fitting / extrapolation for lane tracking through occlusion** — long established in the autonomous-driving lane-detection literature. Its use here for *geometric continuity* is conventional.
- **Edge- and boundary-based lane detection** — likewise established. The observation that *boundaries survive surface damage* is the useful specialisation.
- **Treating unknown cells as traversable rather than blocked** — standard in frontier-based exploration and optimistic costmap planning.
- **Active perception** — the idea that a robot should *move to see better*. The specific move here (climb to widen the nadir camera's footprint until both sides of a road gap are visible) is where the contribution most plausibly lies.

The delta claimed is the *combination*: an altitude-based active-perception recovery coupled to unknown-cell costmap semantics and heading-guided A*, inside a UAV road-following loop. **This claim has not been verified against the literature by systematic search, and must be before any publication.** See Chapter 53.7.

E.7 Acknowledgements

Dr. Anubhav Mishra (IIT Gandhinagar), supervising the internship, contributed the proposal to use **texture as a second segmentation feature** to reduce U-Net latency (Chapters 19, 42), and the Q&A discussion in which the road-occlusion recovery theory was developed (Part XIV).

The `cobot` lab desktop (native Ubuntu 22.04) is where end-to-end road-following was first confirmed working, and is the reference simulation host.

Appendix F — Development Chronology

The order in which the system reached Prototype 1- α . Recorded because the *sequence* of discoveries explains why certain decisions look strange in isolation — and because the pattern of "the obvious suspect was innocent" only becomes visible across the whole timeline.

F.1 Architecture and simulation

The system was designed as PX4 SITL + Gazebo (Baylands world) + ROS 2 Humble + MAVSDK-Python, with four nodes (`camera_node`, `segmentation_node`, `planner_node`, `control_node`) over a core library. Perception was HSV-based in simulation; a U-Net trained on AeroScapes was intended for real hardware. Planning was A* over a semantic costmap; control was Pure Pursuit.

Early development ran on several machines, and *which machine* turned out to matter repeatedly:

Machine	Role	Verdict
<code>coder</code>	Shared cloud VM	Weakest; source of the gRPC starvation crash
<code>cobot</code>	Native Ubuntu 22.04 lab desktop	Best sim host; road-following confirmed here
Core Ultra 9 desktop	Ubuntu 24.04	Too new for native ROS 2 Humble
M1 MacBook Air (UTM VM)	Distributed SITL host	Software rendering → 20% real time
Jetson Orin Nano 8GB	Target companion computer	The deployment platform

F.2 The bug sequence

Fixed in roughly this order, each entry a full entry in Appendix D:

- `core` library missing from `setup.py` — worked standalone, failed under `ros2 run`.
- Topic-name drift (`x500_mono_cam_0` → `x500_mono_cam_down_0`) and **QoS mismatch** — the segmenter received nothing.
- Incorrect occupancy-grid construction — A* searched a map that did not match the camera.
- Missing waypoint popping — the drone reached a waypoint and never advanced.
- Goal placed on an obstacle — fixed by `nearest_free_goal()` radial spiral search; `find_auto_goal` restricted to the lower 80% of the frame.
- asyncio ↔ ROS executor starvation — one thread, two event loops; a blocking call froze everything.
- gRPC `_pose_stream` crash at port 50051 — diagnosed as **VM resource starvation, not a code bug**. Reconnect-with-backoff patch designed; **not yet pushed**.
- HSV thresholds re-measured on a real nadir frame (H 136–140, S 21–24, V 109–135).
- MAVSDK `connect()` hanging on health checks that can never pass on a bench — health check removed.

F.3 The faculty presentation

An 11-slide presentation (Ocean-Gradient palette) and a rehearsal cheat-sheet with seven anticipated questions were prepared. The presentation was delivered successfully. The Q&A covered the U-Net domain gap, the cause of oscillation, and Gazebo performance — and it was in this discussion that the **road-occlusion recovery theory** was proposed and received. Dr. Mishra's texture proposal emerged from the same conversation.

F.4 Flight-controller bring-up

Reflashed from ArduPilot 4.6.3 → PX4 1.17.0. Accelerometer, gyroscope, compass, and RC calibration completed. Flight modes and kill switch assigned. A props-off QGroundControl bench test then surfaced the compass inconsistency (128°, bypassed with `COM_ARM_MAG_ANG = -1`), established that `COM_ARM_WO_GPS` was already permissive, and identified the real

arming obstacle: **Stabilized was never assigned to the mode switch**. Motors were confirmed healthy directly from the Actuators panel.

F.5 Jetson integration

ROS 2 Humble installed natively; the repository built on **ARM64**. The RealSense D455 was integrated (past a GPG signing-key snag) and confirmed publishing live topics, though capped at **USB 2.1 → 640×480 @ 15 fps**. Real camera intrinsics were measured. TensorFlow 2.15 was installed from the NVIDIA JetPack apt repository.

Then the **protobuf collision**: TensorFlow demanded `< 5`, MAVSDK 3.15.3 demanded `>= 6`. Resolved by downgrading MAVSDK to **2.8.4** and pinning **protobuf 3.20.3** — and verified not by imports but by **connecting MAVSDK 2.8.4 to the real Pixhawk over USB**. Alongside: a duplicate ROS apt repository removed, `setuptools` downgraded 83 → 79.x.

Safety patches were then added: `auto_takeoff` and `px4_connection` parameters, and `wait_for_pilot()` gating the pipeline until the pilot arms and enables Offboard via RC — closing the "power on the battery and it takes off" hazard.

F.6 Distributed SITL and the performance hunt

Cross-machine ROS 2 discovery (`ROS_DOMAIN_ID=42`, `ROS_LOCALHOST_ONLY=0`) and MAVLink over `udpin://:14540` were established between a sim VM and the Jetson. Frames arrived at **4.5 Hz**. The bridge, the network, USB, and protobuf were each investigated and **each acquitted**. The cause was Gazebo at **~20% real time**, software-rendering under `LIBGL_ALWAYS_SOFTWARE=1` in a UTM VM. HITL with the real Pixhawk was ruled out on the same machine: a real board runs on wall-clock time and fires sensor-freshness failsafes against a 20%-speed world.

F.7 The segmentation rewrite

The plain U-Net's near-empty masks were finally traced — past the plausible "domain gap" explanation — to **three stacked faults**: only the first 30 masks scanned; `ROAD_CLASS = 1`, which is *person* (road is 10); and plain cross-entropy neglecting road as a pixel-minority class. The replacement — **MobileNetV2 U-Net, ImageNet-pretrained, with Gabor texture fusion and Dice+CE loss over 12 classes** — reached `val_road_iou = 0.80`. Gabor analysis confirmed asphalt is spectrally separable from dirt and rooftops *before* the feature was adopted.

F.8 Centreline-guided planner (v7)

Empirical observation of the A*-based planner on real masks showed the goal repeatedly landing at road-edge pixels (e.g. `goal = (86, 9)`) — an artifact of picking the topmost reachable road cell on a vertical road strip — with the drone following the shoulder rather than the centre. A second failure mode was that on masks with texture-induced noise the goal could flip between branches frame-to-frame, producing goal jitter. Both were traced to the goal-selection step of the planner.

The v7 replacement is `core/skeleton_goal.py` — a module that skeletonises the reachable road mask, prunes the skeleton to its longest branch via a double-BFS tree-diameter search, orients that branch away from the drone, drops points behind, and evenly samples an ordered chain of N (= 12) forward waypoints. `chain_goal()` in the planner advances through the chain one waypoint at a time, rebuilding it only when exhausted, when the current waypoint becomes an obstacle, or when an external goal is set — which gives the aircraft temporal consistency across frames rather than reacting to a fresh recomputed goal each cycle.

Two costmap fixes in `core/depth_costmap.py` were made alongside: `largest_road_component()` restricts goal-picking to the largest connected road blob (so isolated fragments cannot become waypoints), and the obstacle-inflation line was changed from `grid[inflated == 1] = 255` to `grid[(inflated == 1) & (~road_cells)] = 255`, so obstacle dilation cannot mark road cells as blocked — the fix for the "narrow corridor pinched shut by inflation" failure where A* would correctly, unhelpfully, report no path across a road with a parked car on one shoulder. A smart-replanning gate (`REPLAN_MOVE_PX = REPLAN_GOAL_PX = 8`) prevents A* from being re-run when nothing has meaningfully moved.

A* is not replaced. The chain provides the *goals*; A* provides the *paths* between them. End-to-end evaluation against real D455 imagery remains pending on the GPS lock and USB 3.x port swap.

F.9 Mechanical design

A parametric OpenSCAD mount plate: 160 × 160 mm octagonal, 30 mm chamfers, 7 mm retaining wall, M2.5 corner holes, Jetson hole grid $dx = 90 / dy = 69$ mm, Pixhawk standoffs repositioned to the left margin. One conflict remains between a 3 × 3 cm side aperture and the Jetson pocket. Wiring was planned: Pixhawk TELEM2 → Jetson UART1 (`/dev/ttyTHS1`), TX↔RX crossed, ground shared, **VCC deliberately unconnected**; Jetson powered by a 19 V UBEC from the 6S pack.

F.10 Where that leaves Prototype 1-α

Every subsystem exists and works. What remains is physical: wire the ESCs and map them, go outdoors for a GPS lock, move the RealSense to USB 3.x, resolve the mount-plate aperture. And incremental: fine-tune the network on real frames, convert to TensorRT, complete the perception-to-centreline integration, push the reconnect patch.

The consistent pattern across F.2, F.6, and F.7 is that in each of the three headline debugging episodes — the gRPC crash, the 4.5 Hz pipeline, and the near-empty masks — the plausible explanation available at the point of first observation was incorrect, and the actual cause lay in a component other than where the symptom appeared.

Appendix G — List of Figures

All figures in this document are **original**, generated for it from the project's own data and dimensions. None is reproduced from any paper, dataset, website, or documentation.

Fig.	Title	Chapter
1	The two-computer split	1
2	The perception → planning → control pipeline	2
3	HSV road-threshold volume vs measured pixel values	15
4	A* frontier expansion over the semantic costmap	21
4a	Centreline extraction: road mask, distance transform, and skeleton	22
5	Power architecture: one battery, three rails	34
6	Pixhawk TELEM2 → Jetson UART1 wiring	35
7	The parametric mount plate, with the unresolved aperture conflict	37
8	The three stacked segmentation faults	40
9	MobileNetV2 U-Net with Gabor fusion	41
10	Gabor kernels: sinusoid × Gaussian, and a filter bank	42
11	Dice vs cross-entropy under class imbalance	43
12	Distributed SITL topology	48
13	The 4.5 Hz bottleneck causal chain	49
14	The layered safety model	52
15	Road-occlusion recovery: climb and bridge	53
16	Occupancy semantics: unknown vs occupied at an occlusion	53
17	The remaining work as a dependency graph	58

Appendix H — Verification Evidence Register

Each row records a claim made in the body of this report, the specific evidence establishing it, the date range over which the evidence was gathered, the environment (`cobot` desktop / VM / Jetson / real Pixhawk), and the chapter in which the claim is discussed. Where evidence is a log line or command output, it is quoted directly. Where evidence is an observation on hardware, the observing instrument (QGroundControl, `ros2 topic hz`, MAVSDK client) is named.

The register serves two purposes: it provides an audit trail for any specific claim in the Executive Summary, and it makes explicit — for each subsystem — what has been verified as opposed to inferred or assumed.

H.1 Flight controller and firmware

Claim	Evidence	Environment	Chapter
Re flashed ArduPilot 4.6.3 → PX4 1.17.0	Firmware version displayed in QGroundControl; boot banner shows <code>PX4 v1.17.0</code> ; parameter tree matches PX4 rather than ArduPilot naming (<code>COM_*</code> , <code>EKF2_*</code> , <code>MPC_*</code>)	Real Pixhawk 6C Mini, USB to workstation	28
Accelerometer calibrated	QGC calibration wizard completed all six orientations; post-cal <code>SENS_BOARD_*</code> offsets non-zero and stored	Real Pixhawk, bench	29
Gyroscope calibrated	QGC gyro wizard completed; board static; bias parameters updated	Real Pixhawk, bench	29
Compass calibrated	Internal and external magnetometers calibrated separately; QGC calibration completed but subsequently reported a 128° consistency error during arming — see H.2	Real Pixhawk, bench	29, 38
RC calibrated	Channel min/max/trim captured for all four sticks + switches; correctly mapped to roll/pitch/yaw/throttle	Real Pixhawk, bench	29
Flight modes assigned	Mode-switch position → flight-mode mapping visible in QGC; note: Stabilized was not yet assigned to any reachable slot at the time of the bench test — see H.2	Real Pixhawk	30, 38
Kill switch assigned	QGC parameter <code>RC_MAP_KILL_SW</code> set; switch tested during bench Actuators panel session	Real Pixhawk	30, 38
Individual motor outputs functional	QGC Actuators panel sliders individually driven; each of the four outputs produced motor spin (props off)	Real Pixhawk + ESCs + motors, bench	38

H.2 Bench-test findings (QGC session)

Claim	Evidence	Environment	Chapter
Compass inconsistency $\approx 128^\circ$	QGC prearm error message; internal vs external magnetometer heading differ by $\sim 128^\circ$	Real Pixhawk	38.1
Bypassed via <code>COM_ARM_MAG_ANG = -1</code>	Parameter written; subsequent arm attempt no longer refused for compass reason	Real Pixhawk	38.1
<code>COM_ARM_WO_GPS</code> already permissive (warning-only)	Read parameter value; prearm refusals cited GPS but did not name this parameter	Real Pixhawk	38.2
Flight Mode 1 = Position was the root arming block	QGC mode-switch view: only Position, Altitude, and Return assigned; Position selected with no GPS lock	Real Pixhawk, bench	38.3
Fix identified: assign Stabilized to Flight Mode 1	Configuration change specified; not yet applied and re-tested — flagged as (Remaining)	Real Pixhawk	38.3

H.3 Companion computer ↔ flight controller

Claim	Evidence	Environment	Chapter
MAVSDK 2.8.4 connects to real Pixhawk over USB	Client script (Ch. 45.2) executed; printed <code>✓ mavsdk 2.8.4 connects to real Pixhawk; is_connected</code> transitioned to <code>True</code>	Real Pixhawk, <code>serial:///dev/ttyACM0:57600</code>	45.2
Flight-configuration link	Physical wiring inspected against pinout of Ch. 35.2; loopback	Aircraft frame under assembly	35

(<code>/dev/ttyTHS1</code>) wired but not yet verified end-to-end	not yet run		
Ground shared, VCC intentionally not connected	Wiring inspection; TELEM2 pin 1 physically open	Aircraft under assembly	35.2

H.4 Perception — simulation

Claim	Evidence	Environment	Chapter
HSV thresholds calibrated on real nadir frame	Sampled H, S, V ranges across ~20 pixels of the road region in a real nadir frame: H = 136–140, S = 21–24, V = 109–135	<code>cobot</code> desktop	15, App. F.2
Segmentation node publishes at expected topic and QoS	<code>ros2 topic list</code> shows <code>/road_mask</code> ; <code>ros2 topic hz /road_mask</code> returns non-zero on <code>cobot</code> ; RELIABLE QoS configured to match subscriber	<code>cobot</code>	5, 6
End-to-end road-following demonstrated	<code>cobot</code> run: drone followed simulated road; recorded in project log; witnessed	<code>cobot</code> + PX4 SITL + Gazebo Baylands	App. F

H.5 Perception — replacement network

Claim	Evidence	Environment	Chapter
<code>ROAD_CLASS = 1</code> was <code>person</code> , road index is 10	AeroScapes documentation (Nigam et al. 2018) enumerates class indices; verified by re-scanning the full mask set (not the first-30 subset) and matching pixel-count histograms against class labels	Workstation	40.2
Only the first 30 masks had been scanned in the earlier statistics run	Source of previous statistics inspected; loop bound found	Workstation	40.1
Gabor analysis confirms asphalt spectrally separable from dirt/rooftops	Gabor bank applied to sample patches; response-magnitude histograms for asphalt vs dirt vs rooftops are non-overlapping in the frequency band tested	Workstation	42.3
<code>val_road_iou = 0.80</code>	Per-epoch validation output from the training script (<code>train_unet_mobilenet_gabor.py</code>); best-epoch road IoU logged as 0.80	Workstation with GPU	43.4
Domain gap remains on D455 imagery	Model has not been evaluated on real D455 frames; fine-tuning is a stated (Remaining) task	—	43.5

H.5a Centreline-guided planner (v7)

Claim	Evidence	Environment	Chapter
Edge-hugging observed empirically	Planner logged goals at road-boundary pixels on straight road sections (e.g. <code>goal = (86, 9)</code> at the left edge of the reachable region)	<code>cobot</code> + PX4 SITL	22.6
Skeletonisation of the road mask implemented	<code>core/skeleton_goal.py</code> in the repository: <code>skeletonize_road()</code> calls <code>cv2.ximgproc.thinning</code> with a Zhang–Suen fallback (<code>_skeletonize_zhang_suen</code>); no new dependency introduced	Workstation, <code>cobot</code> , Jetson	22.7
Main-branch pruning via double-BFS implemented	<code>main_branch()</code> in <code>skeleton_goal.py</code> ; implements the classic tree-diameter two-BFS algorithm; unit-tested against a synthetic curved road strip (<code>build_waypoint_chain</code> returned an ordered forward-monotone chain in the smoke test)	Workstation	22.8
N-waypoint ordered chain implemented	<code>build_waypoint_chain(reachable_road, start_rc, n_points=12, horizon_frac=0.10)</code> orients the chain away from the drone, drops points behind, and evenly samples 12 waypoints along the forward branch	Workstation, <code>cobot</code>	22.9
Chain integrated into the planner	<code>nodes/planner_node.py</code> calls <code>build_waypoint_chain</code> and advances through the returned chain one waypoint at a time via <code>_chain_goal()</code> ; a new chain is built only when exhausted, when the current waypoint becomes an obstacle, or when an external goal is set	<code>cobot</code>	22.9
Largest-road-component filter	<code>largest_road_component()</code> in <code>core/depth_costmap.py</code> ; uses <code>cv2.connectedComponentsWithStats</code> and returns only the largest non-background blob; enabled by default (<code>keep_largest_road=True</code>)	Workstation, <code>cobot</code>	22.10

Road-protected inflation	v7 line <code>grid[(inflated == 1) & (~road_cells)] = 255</code> replaces v6 <code>grid[inflated == 1] = 255</code> ; obstacle dilation cannot mark road cells as blocked; enabled by default (<code>protect_road=True</code>)	Workstation, <code>cobot</code>	22.10
Reachable-free-cell mask	<code>reachable_free_mask()</code> connected-components pass from the drone's current cell; restricts goal-picking to cells A* can actually reach	Workstation, <code>cobot</code>	22.10
Smart replanning gate	Planner skips the A* call when both start and goal have moved less than <code>REPLAN_MOVE_PX = 8</code> pixels; observed in planner debug logs on <code>cobot</code>	<code>cobot</code>	22.11
End-to-end run against real D455 imagery	Pending — requires outdoor GPS lock (Chapter 32) and USB 3.x port swap (Chapter 18)	—	22.12

H.6 Camera

Claim	Evidence	Environment	Chapter
RealSense D455 publishes live on Jetson	<code>ros2 topic list</code> shows <code>/camera/color/image_raw</code> , <code>/camera/depth/image_rect_raw</code> ; <code>ros2 topic hz</code> reports non-zero rate	Jetson Orin Nano, USB 2.1 port	18
Frame rate capped at 640×480 @ 15 fps due to USB 2.1	<code>rs-enumerate-devices</code> / driver logs report USB 2.1 negotiation; higher requested rates rejected; verified by re-running with only 640×480 @ 15 succeeding	Jetson, USB 2.1 port	18
Real intrinsics measured: fx = fy = 388.162, cx = 322.326, cy = 239.442	Values read from the RealSense driver's <code>camera_info</code> topic at 640×480 stream configuration	Jetson	43.6

H.7 Environment and dependencies

Claim	Evidence	Environment	Chapter
TensorFlow 2.15 installed from NVIDIA JetPack apt repo	<code>apt policy tensorflow</code> shows JetPack repository; <code>python3 -c "import tensorflow; print(tensorflow.__version__)"</code> returns <code>2.15.x</code> ; GPU is detected via <code>tf.config.list_physical_devices('GPU')</code>	Jetson	44
MAVSDK-Python 2.8.4 installed and imports	<code>pip show mavsdk</code> returns 2.8.4; <code>python3 -c "import mavsdk"</code> succeeds	Jetson	45
protobuf pinned at 3.20.3	<code>pip show protobuf</code> returns 3.20.3; <code>python3 -c "import google.protobuf; print(google.protobuf.__version__)"</code> matches	Jetson	45
Both TF and MAVSDK coexist in one environment	Both imports succeed in the same Python process (Ch. 45.2 script)	Jetson	45.2
Duplicate ROS apt repo removed	<code>/etc/apt/sources.list.d/</code> no longer contains a duplicate <code>ros2.list</code> entry; single authoritative source remains	Jetson	46.1
<code>setuptools</code> at 79.x	<code>pip show setuptools</code> ; <code>colcon build</code> completes without deprecation errors previously observed	Jetson	46.2
Versioned Gazebo imports functional	<code>python3 -c "from gz.transport13 import Node; from gz.msgs10.image_pb2 import Image"</code> succeeds	Jetson (also on <code>cobot</code> for sim)	46.4

H.8 Distributed SITL

Claim	Evidence	Environment	Chapter
Cross-machine ROS 2 discovery functional	Nodes on VM appear in <code>ros2 node list</code> on Jetson (and vice versa) with <code>ROS_DOMAIN_ID=42</code> , <code>ROS_LOCALHOST_ONLY=0</code> set on both	Jetson + UTM VM on M1	48
MAVLink to sim over <code>udpin://:14540</code>	MAVSDK on Jetson reports <code>is_connected = True</code> when PX4 SITL runs on the VM	Jetson + VM	48
Simulated camera arrives at ~4.5 Hz on Jetson	<code>ros2 topic hz /camera/image_raw</code> on Jetson reports ~4.5 Hz	Jetson + VM	49.1

Bridge, network, USB, protobuf all not the bottleneck	Bridge measured in isolation on VM side; network throughput sanity-checked with <code>iperf3</code> ; USB not in the sim data path; protobuf import cost negligible per frame	VM + Jetson	49.2
Gazebo running at ~20% real time	Gazebo GUI status bar reports realtime factor ~0.2; <code>LIBGL_ALWAYS_SOFTWARE=1</code> present in environment	UTM VM on M1	49.3

... ..

H.10 Items not yet verified (in-scope for next review)

The following claims elsewhere in this report are either provisional or forward-looking. Several items that appeared in earlier drafts have been resolved by the first outdoor flight of 16 July 2026 (Part XVI §63) and are therefore annotated below.

Resolved by the 16 July 2026 flight (see Part XVI):

- Motor channel-to-position mapping and spin-direction — exercised in flight (§63).
- Outdoor GPS lock — achieved; eph $\approx$ 0.22 m throughout (§63.3).
- TensorRT conversion of the replacement network — FP16 complete (§62); latency improved.
- gRPC reconnect-with-backoff patch — shipped (Appendix I.4).
- End-to-end v7 planner against real D455 imagery — the real pipeline ran in the field; successful centreline tracking was not demonstrated because of the 180° camera-roll bug diagnosed in §64.

Still open:

- Flight-configuration serial link (Pixhawk TELEM2 → Jetson UART1 /dev/ttyTHS1) wired but not yet run end-to-end with MAVSDK under flight conditions.
- Mount plate not printed pending resolution of the 3 × 3 cm aperture/pocket conflict (Chapter 37.3).
- Replacement segmentation network evaluation on real D455 frames after the camera-rotation fix; fine-tuning may still be required (Chapter 43.5).
- HITL against the real Pixhawk has not been attempted; the sim-host performance precondition remains (Chapter 50).
- Corrected centreline-tracking outdoor flight after the §64 patches (camera rotation, Pure Pursuit lookahead, deceleration law).

... ..

- TensorRT conversion of the replacement network is pending (Chapter 17, Part XV).
- The gRPC reconnect-with-backoff patch (Appendix D.5) has been designed but not pushed.
- **End-to-end run of the v7 planner against real D455 imagery is pending:** the chain-based planner and costmap fixes are implemented and unit-tested, but a full outdoor run requires the GPS lock (Chapter 32) and USB 3.x swap (Chapter 18) (Chapter 22.12).
- HITL against the real Pixhawk has not been attempted; the sim-host performance precondition is not yet met (Chapter 50).

H.11 Interpretation

Grouping the register by state of verification gives the following counts.

State	Count
Verified against real hardware or observed live output	27
Verified in simulation only	8
Verified in a workstation/CI environment	8

The majority of Prototype 1- α 's claims rest on direct observation of the real hardware. The pending items are the smallest group and are enumerated in H.10 without concealment. The distinction that matters for review is between "verified in simulation only" (four items — HSV thresholds, end-to-end sim following, safety gate behaviour, and RC-loss failsafe exemption) and "verified on hardware" (twenty-seven items). Anything in the first category is a candidate for re-verification on the real airframe once the (Remaining) physical tasks are complete.

Appendix I — Code Excerpts from v7

The code that implements the design decisions of this report. These excerpts are the **v7 change set** — the files added or modified in v7 relative to v6. Pre-existing v6 modules that were not touched in this progress period (`astar.py`, `occupancy.py`, `projection.py`, `temporal.py`, `trajectory.py`, `camera_node.py`, `segmentation_node.py`, `gz_camera_bridge.py`, `sim_depth_node.py`, `goal_publisher.py`, `trajectory_node.py`) remain in the repository and are not reproduced here. Excerpts are unmodified from the repository except where marked "elided" (logging and boilerplate removed for readability). Each is followed by a short annotation.

I.1 The safety wrapper: `core/px4.py`

The critical safety mechanism is not a check somewhere in the middle of the pipeline; it is a property of the MAVSDK wrapper itself. The `Drone` class takes a flag `allow_offboard_autostart`, and in `real` mode that flag is `False`. In that mode `set_velocity()` streams setpoints but never calls `offboard.start()`, so the flight controller — which will only respond to Offboard setpoints when it is in Offboard mode — silently ignores them until the pilot flips into Offboard on the RC. There is no path by which the software can put PX4 into Offboard on its own.

```
import asyncio
from mavsdk import System
from mavsdk.offboard import OffboardError, VelocityNedYaw

class Drone:
    def __init__(self, address="serial:///dev/ttyACM0:57600",
                 allow_offboard_autostart=True):
        """
        allow_offboard_autostart:
        - True (sim): set_velocity() will call offboard.start() itself.
        - False (real drone): NEVER auto-start offboard. PX4 must be put into
          Offboard by the PILOT via the RC switch. Setpoints are streamed but
          the software never seizes control.
        """
        self._drone = System()
        self._address = address
        self._offboard_started = False
        self._allow_offboard_autostart = allow_offboard_autostart

    async def connect(self):
        await self._drone.connect(system_address=self._address)
        async for state in self._drone.core.connection_state():
            if state.is_connected:
                return

    async def set_velocity(self, vx: float, vy: float, vz: float,
                          yaw_deg: float = 0.0):
        setpoint = VelocityNedYaw(vx, vy, vz, yaw_deg)

        if not self._offboard_started:
            if self._allow_offboard_autostart:
                # SIM ONLY: software starts offboard itself.
                try:
                    await self._drone.offboard.set_velocity_ned(setpoint)
                    await self._drone.offboard.start()
                    self._offboard_started = True
                except OffboardError as e:
                    print(f"[PX4] Offboard start failed: {e._result.result_str}")
                    return
            else:
                # REAL DRONE: do NOT start offboard from software.
                # Prime the setpoint so PX4 has a fresh target the moment the
                # PILOT flips the Offboard switch, but never call start().
                # If PX4 is not already in Offboard, this setpoint is ignored
                # by the flight controller — which is the safe behavior.
                try:
                    await self._drone.offboard.set_velocity_ned(setpoint)
                except OffboardError:
                    pass
                return

        # Offboard is active (pilot-enabled on real, autostarted on sim):
        await self._drone.offboard.set_velocity_ned(setpoint)
```

```

async def is_armed(self) -> bool:
    async for armed in self._drone.telemetry.armed():
        return armed

async def in_offboard(self) -> bool:
    """True if PX4 is currently in Offboard flight mode (pilot-enabled)."""
    async for fm in self._drone.telemetry.flight_mode():
        return str(fm) == "FlightMode.OFFBOARD"

async def kill(self):
    await self._drone.action.kill()

```

The design is worth annotating. On real hardware, three independent things must be true for a setpoint to move the aircraft: the software must be streaming setpoints, the flight controller must be armed, and the flight controller must be in Offboard mode. `allow_offboard_autostart=False` removes the software's ability to make the third one happen — Offboard mode is a *pilot decision*, not a software one, and the flight controller enforces it. `set_velocity()` still calls `set_velocity_ned()` before returning, so PX4 has a fresh setpoint the instant the pilot does flip; there is no dead time between the mode switch and the first commanded motion. The comment "*which is the safe behavior*" is not a claim about intent — it is a statement about what PX4 does with a setpoint received in the wrong mode.

1.2 The pilot handoff gate: `nodes/control_node.py`

The node-side gate is the second half of the safety design. In real mode `_startup()` connects and then hands off to `_wait_for_pilot()`, which polls three conditions — armed, altitude above 2 m, Offboard selected — before flipping `takeoff_complete` to `True`. Until then, `loop()` returns immediately and nothing is published to the controller.

```

class ControlNode(Node):
    def __init__(self):
        super().__init__('control_node')

        # px4_connection: MAVSDK address. Empty string = use Drone() default.
        self.declare_parameter('px4_connection', '')
        # auto_takeoff: True (sim) arms + takes off in software.
        # False (real drone) NEVER arms/takes off – pilot does.
        self.declare_parameter('auto_takeoff', True)

        conn = self.get_parameter('px4_connection').value
        self.auto_takeoff = self.get_parameter('auto_takeoff').value

        # On real hardware auto_takeoff is False, which also means the software
        # must not auto-start offboard. Tie the two together.
        allow_offboard_autostart = self.auto_takeoff
        if conn:
            self.drone = Drone(address=conn,
                                allow_offboard_autostart=allow_offboard_autostart)
        else:
            self.drone = Drone(allow_offboard_autostart=allow_offboard_autostart)

        self.path = []
        self.pose = None
        self.takeoff_complete = False
        # ... executor, subscriptions, timer setup elided ...
        asyncio.run_coroutine_threadsafe(self._startup(), self._loop)

    async def _startup(self):
        await self.drone.connect()

        if self.auto_takeoff:
            # SIM PATH: software arms and takes off automatically.
            await self.drone.arm()
            await self.drone.takeoff(altitude=20.0)
            self.takeoff_complete = True
        else:
            # REAL DRONE PATH: NEVER arm/takeoff from software.
            # The pilot arms and takes off manually via the RC transmitter.
            # takeoff_complete is flipped True only once the drone is actually
            # airborne AND the pilot has enabled Offboard.
            asyncio.run_coroutine_threadsafe(
                self._wait_for_pilot(), self._loop)
            # ... pose stream launch elided ...

    async def _wait_for_pilot(self):
        """Real-hardware gate: only allow setpoint streaming once the pilot has
        (a) armed, (b) climbed off the ground, and (c) selected Offboard mode.
        Until all three are true, takeoff_complete stays False and loop() emits

```

```

nothing – so the software never drives the motors on its own."""
while True:
    try:
        armed = await self.drone.is_armed()
        alt = await self.drone.get_altitude()
        offboard = await self.drone.in_offboard()
        if armed and alt is not None and alt > 2.0 and offboard:
            self.takeoff_complete = True
            return
        await asyncio.sleep(0.5)
    except Exception as e:
        self.get_logger().warn(f'_wait_for_pilot: {e}')
        await asyncio.sleep(1.0)

def path_cb(self, msg):
    if not self.takeoff_complete:
        self.get_logger().info(
            'Ignoring path: not airborne / pilot has not handed off')
        return
    d = msg.data
    self.path = [(d[i], d[i + 1]) for i in range(0, len(d) - 1, 2)]

```

Points that are visible in the code but easy to miss on a first read:

- **Two flags, one control.** `auto_takeoff` and `allow_offboard_autostart` are set to the same value: on real hardware both are False. The former stops the *node* from arming and taking off; the latter stops the *wrapper* from starting Offboard. Independently either would be insufficient — a node that armed but never started Offboard would just hover; a wrapper that started Offboard but had no waypoints would idle. Together, they close the loop.
- **`path_cb` is also gated.** Even if a path arrives from the planner before the pilot hands off, it is dropped rather than queued. This means the pilot cannot inherit stale waypoints when they finally flip into Offboard — the aircraft only ever follows a path published *after* the handoff.
- **The altitude threshold is 2 m**, not zero. Being armed on the ground is not enough to be considered "flying"; the pilot must have actually climbed. This defends against the case where the pilot arms on the bench and accidentally leaves the mode switch on Offboard.

1.3 Skeleton-based waypoint chain: `core/skeleton_goal.py`

This is the module that replaces the goal-selection step of the previous planner. Rather than picking a single goal pixel each cycle and letting A* find a path to it, the module extracts a *chain* of ordered waypoints along the road's centreline, which the planner then advances through one at a time.

```

"""
skeleton_goal.py – Skeleton-based ordered waypoint chain for road following.

The idea:
road mask
-> skeletonize (thin the road to a 1-pixel-wide centerline curve)
-> prune to the main branch (drop side spurs / small twigs)
-> sample a FIXED NUMBER of evenly-spaced points along it
-> feed them as an ORDERED chain of goals, advancing one at a time

Why ordered points beat a single recomputed goal:
* "forward" is unambiguous – the points are in order along the road
* the drone advances point -> point -> point, never picks one behind
* the chain is stable even if the mask flickers slightly frame to frame
"""

import numpy as np
import cv2

# --- 1. SKELETONIZE ---
def _skeletonize_zhang_suen(img):
    """Pure-numpy Zhang-Suen thinning. img: uint8 0/1. Returns uint8 0/1."""
    img = (img > 0).astype(np.uint8)
    changed = True
    while changed:
        changed = False
        for step in (0, 1):
            p = np.pad(img, 1, mode='constant')
            # 8 neighbours, clockwise from north
            P2 = p[0:-2, 1:-1]; P3 = p[0:-2, 2:]

```

```

P4 = p[1:-1, 2:]; P5 = p[2:, 2:]
P6 = p[2:, 1:-1]; P7 = p[2:, 0:-2]
P8 = p[1:-1, 0:-2]; P9 = p[0:-2, 0:-2]

B = P2 + P3 + P4 + P5 + P6 + P7 + P8 + P9
seq = [P2, P3, P4, P5, P6, P7, P8, P9, P2]
A = sum(((seq[i] == 0) & (seq[i+1] == 1)).astype(np.uint8)
        for i in range(8))

if step == 0:
    c1 = P2 * P4 * P6
    c2 = P4 * P6 * P8
else:
    c1 = P2 * P4 * P8
    c2 = P2 * P6 * P8

cond = ((img == 1) & (B >= 2) & (B <= 6) & (A == 1)
        & (c1 == 0) & (c2 == 0))
if cond.any():
    img[cond] = 0
    changed = True

return img

def skeletonize_road(road_mask):
    """Thin the road mask to a 1-pixel-wide centerline. uint8 0/1 in and out."""
    m = (road_mask > 0).astype(np.uint8)
    if m.sum() == 0:
        return m
    # prefer the fast opencv-contrib thinning if present
    try:
        thin = cv2.ximgproc.thinning(
            m * 255, thinningType=cv2.ximgproc.THINNING_ZHANGSUEN)
        return (thin > 0).astype(np.uint8)
    except (AttributeError, cv2.error):
        return _skeletonize_zhang_suen(m)

# --- 2. PRUNE TO THE MAIN BRANCH (longest path in the skeleton) ---
def _neighbours(r, c, shape):
    h, w = shape
    for dr in (-1, 0, 1):
        for dc in (-1, 0, 1):
            if dr == 0 and dc == 0:
                continue
            nr, nc = r + dr, c + dc
            if 0 <= nr < h and 0 <= nc < w:
                yield nr, nc

def _bfs_farthest(skel, src):
    """BFS over skeleton pixels from src. Returns (farthest_pixel, parent_map)."""
    from collections import deque
    parent = {src: None}
    q = deque([src])
    last = src
    while q:
        cur = q.popleft()
        last = cur
        for nb in _neighbours(cur[0], cur[1], skel.shape):
            if skel[nb] and nb not in parent:
                parent[nb] = cur
                q.append(nb)
    return last, parent

def main_branch(skel):
    """Longest path through the skeleton (double-BFS, the classic tree diameter).
    Returns an ORDERED list of (row, col) along that path, or [] if empty.

    This prunes side spurs: only the single longest continuous curve survives,
    which for a road skeleton is the road itself rather than little twigs."""
    pts = np.argwhere(skel > 0)
    if len(pts) == 0:
        return []
    src = tuple(pts[0])
    a, _ = _bfs_farthest(skel, src) # farthest point from arbitrary start
    b, parent = _bfs_farthest(skel, a) # farthest from A → B; A..B is longest
    path = []
    cur = b
    while cur is not None:
        path.append(cur)
        cur = parent[cur]
    return path

```

```

# --- 3. SAMPLE FIXED NUMBER OF ORDERED POINTS ---
def sample_chain(path, n_points=12):
    """Evenly sample n_points along the ordered skeleton path."""
    if len(path) == 0:
        return []
    if len(path) <= n_points:
        return list(path)
    idx = np.linspace(0, len(path) - 1, n_points).astype(int)
    return [path[i] for i in idx]

# --- 4. PUBLIC API – build the ordered forward waypoint chain ---
def build_waypoint_chain(reachable_road, start_rc, n_points=12,
                        horizon_frac=0.10):
    """
    reachable_road : uint8 mask, 1 = reachable road (already filtered to the
                    start's connected component).
    start_rc       : (row, col) of the drone in grid coords.
    n_points       : how many waypoints to sample along the road skeleton.
    horizon_frac   : drop the top this fraction of rows (noisy horizon).

    Returns an ORDERED list of (row, col) waypoints running FORWARD (away
    from the drone), nearest-first. Empty list if no usable skeleton.
    """
    m = (reachable_road > 0).astype(np.uint8)
    if m.sum() == 0:
        return []

    cut = int(m.shape[0] * horizon_frac)
    if cut > 0:
        m[:cut, :] = 0
        if m.sum() == 0:
            return []

    skel = skeletonize_road(m)
    if skel.sum() == 0:
        return []

    path = main_branch(skel)
    if len(path) < 2:
        return []

    # Orient the chain so it runs AWAY from the drone:
    # whichever end is closer to the start becomes the first waypoint.
    def d2(p):
        return (p[0] - start_rc[0]) ** 2 + (p[1] - start_rc[1]) ** 2

    if d2(path[-1]) < d2(path[0]):
        path = path[::-1]

    # Keep only points ahead of the drone (smaller row = forward, nadir cam)
    forward = [p for p in path if p[0] < start_rc[0]]
    if len(forward) < 2:
        forward = path
    return sample_chain(forward, n_points)

```

Design points worth naming out loud:

- **main_branch** uses the classic "tree diameter by double BFS" trick: BFS from any pixel finds a farthest point A; BFS from A finds a farthest point B; the A→B path is the longest simple path in the graph. On a road skeleton this correctly finds the road itself and discards little spurs (segmentation dust, dead-end turnoffs). No thresholding or manual pruning — just the topological property that side branches are shorter than the main road.
- **skeletonize_road** tries `cv2.ximgproc.thinning` first and falls back to the numpy Zhang–Suen. There is deliberately no new dependency introduced by v7: `opencv-contrib` is optional. This matters because the Jetson's Python environment is already fragile (Chapter 44–45) and a new `pip install` there is expensive.
- **horizon_frac** drops the top 10% of rows before skeletonising, not before segmentation. The rationale is that the horizon band is where the mask is noisiest (sub-pixel road pixels, aliasing) and the skeleton in that region produces short, wobbly branches that confuse the "longest path" pruner. Dropping them upstream is cheaper than trying to filter them out afterwards.
- **Forward-only filter.** `p[0] < start_rc[0]` — smaller row means further from the bottom of the frame, which on a downward-looking camera means physically ahead of the aircraft. The fallback `if len(forward) < 2: forward = path`

handles the edge case where the drone is *at* the top of the visible road strip; in that case there is no forward-only chain and the full chain is used unchanged.

1.4 Pure Pursuit and pose streaming: `nodes/control_node.py` (continued)

The controller itself is Pure Pursuit against the current waypoint chain. `loop()` runs at 10 Hz (via `create_timer(0.1, self.loop)`), retires waypoints as they are reached, and computes velocity commands scaled to a fixed maximum. `_pose_stream` is separate, streams telemetry continuously, and has automatic reconnection because the gRPC transport under MAVSDK proved fragile enough (Appendix D.5) to make a reconnect strategy worth building in.

```
LOOKAHEAD = 3.0 # metres - Pure Pursuit lookahead
VMAX      = 1.5 # m/s   - commanded speed
REACHED   = 0.5 # metres - retire a waypoint once within this

async def _pose_stream(self):
    """Stream position telemetry with automatic reconnection.
    gRPC dropouts under load are recoverable if we do exponential backoff."""
    retry_delay = 1.0
    max_retry_delay = 10.0
    while True:
        try:
            async for pos in self.drone.drone.telemetry.position_velocity_ned():
                self.pose = (pos.position.north_m, pos.position.east_m)
                retry_delay = 1.0 # reset backoff on success
        except Exception as e:
            self.get_logger().error(
                f'_pose_stream lost connection: {e}. '
                f'Retrying in {retry_delay:.1f}s...'
            )
            await asyncio.sleep(retry_delay)
            retry_delay = min(retry_delay * 2, max_retry_delay)

def loop(self):
    if not self.takeoff_complete:
        return
    if not self.path or self.pose is None:
        return

    # publish current pose for downstream (planner uses this)
    pmsg = PointStamped()
    pmsg.header.stamp = self.get_clock().now().to_msg()
    pmsg.header.frame_id = 'map'
    pmsg.point.x = self.pose[0]
    pmsg.point.y = self.pose[1]
    pmsg.point.z = 0.0
    self.pose_pub.publish(pmsg)

    # Pure Pursuit target: first waypoint at least LOOKAHEAD away
    target = None
    for wp in self.path:
        dx = wp[0] - self.pose[0]
        dy = wp[1] - self.pose[1]
        dist = (dx * dx + dy * dy) ** 0.5
        if dist >= LOOKAHEAD:
            target = wp
            break
    if target is None:
        target = self.path[-1]

    # Retire near waypoints
    dx = target[0] - self.pose[0]
    dy = target[1] - self.pose[1]
    dist = (dx * dx + dy * dy) ** 0.5
    if dist < REACHED:
        if len(self.path) > 1:
            self.path.pop(0)
        return

    # Constant-speed velocity command toward the lookahead point
    vx = VMAX * dx / max(dist, 1e-3)
    vy = VMAX * dy / max(dist, 1e-3)
    asyncio.run_coroutine_threadsafe(
        self.drone.set_velocity(vx, vy, 0.0),
        self._loop
    )
```

Two things worth annotating.

The `max(dist, 1e-3)` guard is not cosmetic; without it, if `dist` reaches zero at the lookahead calculation stage between the retirement check and the velocity computation (which it can, race-of-timing edge case), the division produces `NaN` and PX4 treats the setpoint as invalid — which under Offboard, if the invalid-setpoint sequence lasts long enough, triggers a failsafe. The guard eliminates that path.

The exponential backoff in `_pose_stream` ($1\text{ s} \rightarrow 2\text{ s} \rightarrow 4\text{ s} \rightarrow 8\text{ s} \rightarrow 10\text{ s cap}$) exists because the `_pose_stream` gRPC crash of Appendix D.5 turned out to be recoverable if the client politely waits rather than hammering the transport back into the same failure. It is the smallest patch that turns a fatal error into a transient one.

1.5 Segmentation deployment: `core/segmentation.py`

The two-input segmentation wrapper. On real hardware it rebuilds the MobileNetV2+Gabor architecture (I.6) from scratch and loads `.weights.h5` into it — because full `.keras` files saved under Keras 3 cannot be loaded under the Jetson's Keras 2.15 (see Appendix D.17). The BGR→RGB conversion is mandatory: training used RGB order, and feeding BGR at inference produces a swapped-channel mask that looks superficially plausible and is silently wrong.

```
import os
import cv2
import numpy as np

from core.mobilenet_unet import (
    build_mobilenet_unet_gabor, gabor_channel,
    IMG_SIZE, NUM_CLASSES, ROAD_CLASS,
)

_DEFAULT_WEIGHTS = os.path.expanduser(
    "~/semantic-drone/model/unet_mobilenet_gabor.weights.h5")

class Segmenter:
    """Runs the road-segmentation network on BGR frames."""

    def __init__(self, weights_path=None):
        weights_path = weights_path or os.environ.get(
            "SEMANTIC_DRONE_WEIGHTS", _DEFAULT_WEIGHTS)
        if not os.path.exists(weights_path):
            raise FileNotFoundError(f"Weights not found: {weights_path}")

        # weights=None: skip ImageNet download; load_weights overwrites.
        self.model = build_mobilenet_unet_gabor(
            num_classes=NUM_CLASSES, img_size=IMG_SIZE, weights=None)
        self.model.load_weights(weights_path)

    def predict(self, frame_bgr):
        """frame_bgr: HxWx3 uint8 (OpenCV / RealSense BGR).
        Returns HxW uint8 class map at the ORIGINAL frame resolution."""
        h, w = frame_bgr.shape[:2]

        # training used RGB — convert, do not feed BGR
        rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
        rgb_small = cv2.resize(rgb, (IMG_SIZE, IMG_SIZE))

        gab = gabor_channel(rgb_small)           # HxWx1 float [0,1]
        rgb_norm = rgb_small.astype(np.float32) / 255.0  # HxWx3 float [0,1]

        pred = self.model.predict(
            {"rgb": np.expand_dims(rgb_norm, 0),
             "gabor": np.expand_dims(gab, 0)},
            verbose=0)[0]

        classes = np.argmax(pred, axis=-1).astype(np.uint8)
        return cv2.resize(classes, (w, h), interpolation=cv2.INTER_NEAREST)

class HSVSegmenter:
    """Sim-mode fallback: HSV colour thresholding on Gazebo's rendered road."""

    LOWER = np.array([100, 0, 50], dtype=np.uint8)
    UPPER = np.array([170, 50, 180], dtype=np.uint8)

    def predict(self, frame_bgr):
        hsv = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2HSV)
        road = cv2.inRange(hsv, self.LOWER, self.UPPER)
```

```

# emit a class map so downstream code is identical to the real path
classes = np.zeros(road.shape, dtype=np.uint8)
classes[road > 0] = ROAD_CLASS
return classes

def make_segmenter(mode="real", weights_path=None):
    """Factory: mode='sim' -> HSV thresholding, mode='real' -> U-Net."""
    if mode == "sim":
        return HSVSegmenter()
    return Segmenter(weights_path=weights_path)

```

The factory pattern is worth calling out. `make_segmenter(mode="sim")` returns an `HSVSegmenter`; `make_segmenter(mode="real")` returns the neural `Segmenter`. Both classes expose the same `predict(frame_bgr)` signature and return the same shape (`HxW uint8` class map), and both emit the road class at `ROAD_CLASS = 10`, so every downstream stage — the road extraction, the costmap, the skeletoniser — is unaware which segmenter is running. That interface identity is what makes the sim-to-real transition a launch-file parameter rather than a code change.

1.6 The segmentation architecture: `core/mobilenet_unet.py`

The architecture that must be reproducible byte-identically on both the training machine and the Jetson, because deployment loads weights into a rebuilt graph rather than a serialised model. The Gabor channel is not what a first reading might expect — it is not an eight-channel volume fed at the bottleneck but a single-channel *texture-energy* map, computed from the standard deviation of responses across a bank of Gabor kernels, inverted so that smooth surfaces (asphalt) read high.

```

"""
MobileNetV2 U-Net with Gabor texture fusion.
Trained result: val_road_iou = 0.80 (AeroScapes, 12 classes, road = index 10)

Inputs : {"rgb": (H,W,3) float [0,1], "gabor": (H,W,1) float [0,1]}
Output : (H,W,12) softmax
"""
import numpy as np
import cv2
import tensorflow as tf
from tensorflow.keras import layers, models

IMG_SIZE = 256
NUM_CLASSES = 12
ROAD_CLASS = 10
# AeroScapes: bckgrnd, person, bike, car, drone,
# boat, animal, obstacle, construction, vegetation,
# ROAD, sky

# --- Gabor kernel bank (fixed, no learned parameters) ---
# 4 orientations x 2 frequencies = 8 kernels
_GABOR_KERNELS = []
for _theta in np.arange(0, np.pi, np.pi / 4):
    # 0, 45, 90, 135 deg
    for _freq in [0.1, 0.25]:
        _GABOR_KERNELS.append(
            cv2.getGaborKernel((21, 21), sigma=4.0, theta=_theta,
                               lambda=1.0 / _freq, gamma=0.5, psi=0))

def gabor_channel(rgb_uint8):
    """rgb_uint8: HxWx3 uint8 (RGB order). -> HxWx1 float32 in [0,1].
    Smooth asphalt -> low texture energy; grainy dirt/roofs -> high energy.
    Inverted so smooth surfaces read HIGH in the returned map."""
    gray = cv2.cvtColor(rgb_uint8, cv2.COLOR_RGB2GRAY)
    responses = [cv2.filter2D(gray, cv2.CV_32F, k) for k in _GABOR_KERNELS]
    energy = np.std(np.stack(responses), axis=0)
    energy = cv2.normalize(energy, None, 0, 1, cv2.NORM_MINMAX)
    return (1.0 - energy).astype(np.float32)[..., None]

def build_mobilenet_unet_gabor(num_classes=NUM_CLASSES, img_size=IMG_SIZE,
                               weights="imagenet"):
    """MobileNetV2 encoder (ImageNet) + U-Net decoder + Gabor texture stem."""
    rgb_in = layers.Input((img_size, img_size, 3), name="rgb")
    gabor_in = layers.Input((img_size, img_size, 1), name="gabor")

    # MobileNetV2 expects [-1, 1]; we feed [0, 1] so rescale then preprocess.
    x = layers.Rescaling(255.0)(rgb_in)
    x = tf.keras.applications.mobilenet_v2.preprocess_input(x)
    backbone = tf.keras.applications.MobileNetV2(

```

```

include_top=False, weights=weights, input_tensor=x)

s1 = backbone.get_layer("block_1_expand_relu").output # 128×128
s2 = backbone.get_layer("block_3_expand_relu").output # 64×64
s3 = backbone.get_layer("block_6_expand_relu").output # 32×32
s4 = backbone.get_layer("block_13_expand_relu").output # 16×16
bn = backbone.get_layer("out_relu").output # 8×8

# Gabor stem – process the texture channel to feature-map scale
g = layers.Conv2D(16, 3, padding="same", activation="relu")(gabor_in)
g = layers.Conv2D(16, 3, padding="same", activation="relu")(g)
g = layers.MaxPooling2D()(g) # -> 128×128

def up(x, skip, f):
    x = layers.Conv2DTranspose(f, 2, strides=2, padding="same")(x)
    x = layers.Concatenate()([x, skip])
    x = layers.Conv2D(f, 3, padding="same", activation="relu")(x)
    x = layers.Conv2D(f, 3, padding="same", activation="relu")(x)
    return x

d4 = up(bn, s4, 512)
d3 = up(d4, s3, 256)
d2 = up(d3, s2, 128)
d1 = up(d2, s1, 64)
d1 = layers.Concatenate()([d1, g]) # fuse texture features here
d1 = layers.Conv2D(64, 3, padding="same", activation="relu")(d1)

d0 = layers.Conv2DTranspose(32, 2, strides=2, padding="same")(d1)
d0 = layers.Conv2D(32, 3, padding="same", activation="relu")(d0)
out = layers.Conv2D(num_classes, 1, activation="softmax",
                    dtype="float32")(d0)

return models.Model({"rgb": rgb_in, "gabor": gabor_in}, out)

```

The design choices are dense here and worth annotating carefully:

- **Gabor is a fixed classical filter bank**, not a learned one. Eight kernels — four orientations (0° , 45° , 90° , 135°) $\times$ two spatial frequencies (0.1 and 0.25 cycles/pixel) — cover the orientation-frequency plane densely enough to distinguish asphalt from dirt and roof textures at typical drone altitudes.
- **Energy is the standard deviation across the bank's responses**, not the sum. `std` peaks where responses vary strongly with orientation (edges, tiles, gravel patterns) and is low where the surface is texturally uniform (asphalt). Sum would have been a coarser proxy — a smooth surface with a single strong-response direction (e.g. a lane marking) would score high on sum but low on std.
- **The map is inverted (`1.0 - energy`)** so that asphalt reads high, matching the sign convention of a "roadness" cue and making the fusion in the decoder additive rather than requiring a learned sign flip.
- **Fusion is late, not early.** The Gabor stem is only two convolutions plus one max-pool — enough to bring the 256×256 texture map down to the 128×128 spatial scale of the outermost decoder block, and no deeper. Fusing at 128×128 rather than at the 8×8 bottleneck preserves the spatial precision of the texture cue, which is exactly what a segmentation task needs.
- **Rescaling(255.0) before `preprocess_input`** is required because MobileNetV2's `preprocess_input` expects raw $[0, 255]$ uint8-scale floats, not the $[0, 1]$ -normalised floats the model receives at its inputs. Skipping this produces silently-degraded performance and is the sort of mistake that survives all unit tests because the shapes are right.
- **`weights=None` in the Jetson deployment path** avoids the ImageNet download at inference-time construction, since `load_weights()` immediately overwrites them anyway.

1.7 Costmap fixes: `core/depth_costmap.py` (selected functions)

The two v7 costmap fixes that made A* usable on real masks. The full file is longer; these are the parts that matter for the report.

```

def largest_road_component(road_mask):
    """Keep only the biggest connected road blob.

    Rationale: on real imagery the segmenter occasionally emits an isolated
    fragment (e.g. a person standing on the road segments as their own blob,
    or a distant road patch across a gap). The planner's goal picker will then
    happily pick that fragment as a goal even though there is no road path to

```

```

it. Restricting to the largest component eliminates that class of failure.
"""
m = (road_mask > 0).astype(np.uint8)
num, labels, stats, _ = cv2.connectedComponentsWithStats(m, connectivity=8)
if num <= 1:
    return m
# stats[0] is the background; find the largest non-background component
areas = stats[1:, cv2.CC_STAT_AREA]
biggest = 1 + int(np.argmax(areas))
return (labels == biggest).astype(np.uint8)

def build_grid(road_mask, obstacle_mask, inflate=3,
               keep_largest_road=True, protect_road=True):
    """Build the A* traversal grid from a road mask and an obstacle mask.

    Two v7 corrections:
    * If keep_largest_road: restrict road_mask to its largest component,
      so the planner cannot pick isolated road fragments as goals.
    * If protect_road: obstacle inflation cannot dilate over road cells.
      This is the fix for the "narrow corridor pinched shut by inflation"
      failure, which had A* correctly report 'no path' on roads that were
      physically wide enough to fly.
    """
    if keep_largest_road:
        road_mask = largest_road_component(road_mask)

    grid = np.zeros_like(road_mask, dtype=np.uint8)
    grid[road_mask > 0] = 0                # free
    grid[obstacle_mask > 0] = 255         # blocked

    if inflate > 0:
        kernel = np.ones((2 * inflate + 1, 2 * inflate + 1), np.uint8)
        inflated = cv2.dilate(obstacle_mask.astype(np.uint8), kernel)
        if protect_road:
            # inflate only into NON-road cells
            grid[(inflated == 1) & (road_mask == 0)] = 255
        else:
            grid[inflated == 1] = 255      # legacy v6 behaviour

    return grid

def reachable_free_mask(grid, start_rc):
    """Connected component of free cells containing the start.

    Given to the goal-picker so it cannot select a goal that A* would find
    unreachable. Prevents the 'find_auto_goal returns a valid-looking road
    pixel that A* immediately reports as unreachable' failure mode.
    """
    free = (grid == 0).astype(np.uint8)
    if free[start_rc[0], start_rc[1]] == 0:
        return np.zeros_like(free)
    num, labels = cv2.connectedComponents(free, connectivity=8)
    start_label = labels[start_rc[0], start_rc[1]]
    return (labels == start_label).astype(np.uint8)

```

The single most consequential line is the one that fixes the inflation:

```
grid[(inflated == 1) & (road_mask == 0)] = 255
```

The v6 line was `grid[inflated == 1] = 255`, which does the same thing but does *not* protect road cells from being marked as obstacles. Under v6, a narrow road with an obstacle at its edge would have the inflation "eat inward" across the road until the corridor was pinched shut — and A* would then correctly, unhelpfully, report that no path existed. The v7 line is one boolean-and away from the v6 line, and it is why A* now finds a path across a road that has a car parked on one shoulder.

1.8 The launch file: `launch/drone.launch.py` (selected)

A single launch file switches every relevant parameter between sim and real via `mode:=sim|real`. The table in Chapter 12 records the substantive differences (camera source, depth source, segmentation model, intrinsics, PX4 link, arming policy); this excerpt shows how the switch is expressed.

```

def generate_launch_description():
    mode_arg = DeclareLaunchArgument(

```

```

    "mode", default_value="sim", choices=["sim", "real"])
mode = LaunchConfiguration("mode")

# Real D455 intrinsics (Chapter 43.6). Sim uses the simple pinhole model.
intrinsics_real = {"fx": 388.162, "fy": 388.162,
                  "cx": 322.326, "cy": 239.442}
intrinsics_sim = {"fx": 205.0, "fy": 205.0,
                 "cx": 320.0, "cy": 240.0}

# Segmenter mode → HSV in sim, U-Net in real
segmenter_params = {"mode": mode}

# PX4 link and arming policy – the key safety switch
px4_link_real = "serial:///dev/ttyACM0:57600"
px4_link_sim = "udpin://0.0.0.0:14540"
control_params = {
    # In sim, software takes off automatically. In real, the pilot does.
    # See core/px4.py – this ties to allow_offboard_autostart.
    "auto_takeoff": PythonExpression(["'", mode, "' == 'sim'"]),
    "px4_connection": PythonExpression([
        "'", px4_link_sim, "' if '", mode, "' == 'sim' else '",
        px4_link_real, "'"]),
}
# ... node definitions elided ...

```

Two things are worth naming. The `PythonExpression` gymnastics are not elegant; they are what ROS 2 launch files require when a parameter's value must be *derived* from a launch argument rather than passed directly. And the `auto_takeoff` derivation from `mode` — `True` when `sim`, `False` when `real` — is the single point of authority that determines whether the software will arm the aircraft on boot. Changing that expression is the mechanical equivalent of connecting the battery to a live aircraft, and it is one of the reasons the launch file, boring as it is, is worth including in the record.

Appendix J — Deployment and Reproducibility Guide

The complete set of steps required to reproduce the Prototype 1- α software state on a fresh Jetson Orin Nano developer kit. Ordered so that later steps depend only on earlier ones. Timings are approximate on a stock Orin Nano developer kit with a 128 GB NVMe SSD.

J.1 Base operating system

The Jetson runs the **NVIDIA JetPack 6.x** image (Ubuntu 22.04 LTS aarch64, L4T R36.3.0). The developer-kit image is flashed via NVIDIA SDK Manager from an x86 host. Ubuntu Desktop, not Server, because ROS 2 Humble's rviz/rqt tooling assumes a graphical stack for debugging; the aircraft can be booted headless once bring-up is complete.

- **Time to flash and first boot.** ~40 minutes on a stock USB-3 SSD.
- **First boot tasks.** Set locale to `en_US.UTF-8`, disable automatic suspend (`gsettings set org.gnome.desktop.session idle-delay 0`), enable `sshd`, set the hostname to something unambiguous (`orin-drone` or similar so that ROS 2 discovery logs are readable), and expand `/` to fill the SSD if it isn't already.
- **Power model.** For bench work the developer kit runs on its bundled 19 V wall supply into J16. For flight it runs from the drone's own 19 V UBEC (Chapter 34) into the same jack. Do not power via the USB-C port; that path is undervoltage-limited and produces intermittent brownouts under GPU load.

J.2 System package baseline

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential cmake git python3-pip python3-venv \
    python3-dev python3-setuptools python3-wheel \
    libssl-dev libffi-dev pkg-config \
    v4l-utils usbutils

# Diagnostic tools worth having on the aircraft
sudo apt install -y htop iotop iftop tmux
```

`tmux` is not a nicety — a running session survives SSH disconnects, which happen every time the aircraft moves out of range of the workstation's Wi-Fi.

J.3 ROS 2 Humble

```
# Add the ROS 2 apt repository — EXACTLY ONE source. (See D.20 for the trap.)
sudo apt install -y curl gnupg lsb-release
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
    -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
    http://packages.ros.org/ros2/ubuntu $(lsb_release -cs) main" \
| sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

# Verify no duplicate ros2.list files
ls /etc/apt/sources.list.d/ros2*.list      # should be exactly one file

sudo apt update
sudo apt install -y ros-humble-desktop \
    ros-humble-cv-bridge ros-humble-image-transport \
    python3-colcon-common-extensions python3-rosdep

sudo rosdep init && rosdep update
echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc
source /opt/ros/humble/setup.bash
```

Sanity check. `ros2 topic list` in one terminal, `ros2 topic pub /test std_msgs/String "data: hi"` in another; the topic should appear.

J.4 The pinned Python environment

This is the critical section. The environment described here is the one verified in Chapter 45; deviations from it will produce the protobuf collision (D.18) or the Keras version mismatch (D.17).

```
# setuptools must be old enough for ROS 2 Humble's ament_python (D.21)
python3 -m pip install --user 'setuptools==79.0.1' 'wheel'

# TensorFlow: from NVIDIA's JetPack repo – NOT pip's default wheel
# (which is either x86-only or lacks CUDA support).
sudo apt install -y python3-libnvinfer python3-libnvinfer-dev
# Follow NVIDIA's current JetPack instructions to install their tensorflow==2.15
# package. Verify:
python3 -c "import tensorflow as tf; print(tf.__version__); \
           print(tf.config.list_physical_devices('GPU'))"
# Expected: '2.15.x', and a non-empty GPU list.

# MAVSDK-Python 2.8.4 (D.18)
python3 -m pip install --user 'mavsdk==2.8.4' 'protobuf==3.20.3'

# Coexistence verification – the only test that matters
python3 -c "import tensorflow, mavsdk, google.protobuf; \
           print('tf', tensorflow.__version__); \
           print('mavsdk', mavsdk.__version__); \
           print('protobuf', google.protobuf.__version__)"
```

Freeze the exact versions:

```
python3 -m pip freeze --user > requirements.jetson.txt
```

Keep `requirements.jetson.txt` in the repository. It is the durable record of the equilibrium found in Chapter 45.

J.5 Gazebo (workstation side, not Jetson)

On the sim host — the workstation or lab desktop, **not the Jetson** — install Gazebo Garden or Harmonic per PX4's compatibility matrix, install PX4 in a workspace, and confirm the `x500` model boots.

```
# From the PX4-Autopilot workspace
make px4_sitl gz_x500
```

Cross-machine ROS 2 discovery is configured on **both** the sim host and the Jetson:

```
# Add to .bashrc on both machines
export ROS_DOMAIN_ID=42
export ROS_LOCALHOST_ONLY=0
```

Confirm from the Jetson:

```
ros2 topic list          # topics from the sim host should appear
ros2 topic hz /image     # confirm the camera bridge is publishing
```

If topics do not appear across the machines, check the two environment variables (they are the two most common causes; see Chapter 48).

J.6 Repository build

```
mkdir -p ~/ros2_ws/src && cd ~/ros2_ws/src
git clone git@github.com:ACHYUT13807/semantic-drone.git
cd ~/ros2_ws

# Install any rosdep-managed dependencies declared in package.xml files
rosdep install --from-paths src --ignore-src -r -y

colcon build --symlink-install
source install/setup.bash
```

`--symlink-install` is important: it means Python source edits do not require a rebuild — colcon links to the source tree rather than copying. On a rapidly-iterating project on a slow device, this saves hours.

J.7 RealSense D455

```
# Install the librealsense apt repository. (If the key is missing/expired, see D.12.)
# Then:
sudo apt install -y ros-humble-realsense2-camera ros-humble-realsense2-description

# Launch the camera node
ros2 launch realsense2_camera rs_launch.py

# Confirm the camera is live
ros2 topic hz /camera/color/image_raw
```

USB port matters. If the camera enumerates at USB 2.1, the driver will silently downgrade to 640×480 @ 15 fps (Chapter 18). Verify with `lsusb -t` — the D455 should show a 5000M or higher speed. If it does not, move it to a genuine USB 3.x port with a proper cable (no 2.0-only hub in the path).

J.8 Trained model weights

```
# Ship .weights.h5 (Ch. 41.4), NOT .keras — the version-safe format.
mkdir -p ~/ros2_ws/src/semantic-drone/models
cp /path/to/mobilenet_unet_gabor.weights.h5 \
  ~/ros2_ws/src/semantic-drone/models/

# On the Jetson, load them into a rebuilt architecture (I.4).
python3 -c "
from semantic_drone.segmentation import build_mobilenet_unet_gabor
m = build_mobilenet_unet_gabor()
m.load_weights('models/mobilenet_unet_gabor.weights.h5')
print('weights loaded:', sum([w.size for w in m.get_weights()]), 'params')
"
```

J.9 First run — the smoke test

Bring the system up incrementally, not all at once. Each of the following should succeed before proceeding to the next:

1. **PX4 SITL on the sim host** — `make px4_sitl gz_x500`; the Gazebo GUI shows the drone.
2. **MAVSDK connects** — on the Jetson, `python3 -m semantic_drone.diagnostics.mavsdk_connect udp://:14540` prints `is_connected = True`.
3. **Camera flows** — `ros2 topic hz /camera/image_raw` on the Jetson reports a non-zero rate. If distributed SITL, expect ~4.5 Hz under software rendering (Chapter 49); on `cobot` with real GPU, ~30 Hz.
4. **Segmentation runs** — `ros2 topic hz /road_mask` reports a non-zero rate; `ros2 topic echo /segmentation/road_percent` shows a plausible fraction.
5. **Planner produces waypoints** — `ros2 topic echo /planner/waypoints` shows non-empty lists.
6. **Autonomy is gated** — the pilot-gate log message appears; the aircraft does *not* take off on connect.

Only if all six succeed does step 7 happen:

7. **Pilot arm + Offboard** — on the RC, arm and switch to Offboard; the pipeline advances past `_wait_for_pilot()` and commands motion.

J.10 What is deliberately not in this guide

- **Flight-controller flashing and calibration** (Chapters 28–29). This is done from a workstation running QGroundControl, not from the Jetson.
- **Physical wiring** (Chapters 34–35). Mechanical, not reproducible from a script.
- **Outdoor GPS lock**. Procedural, not code (D.13).

The guide covers *software* reproducibility. Hardware setup lives in the body of the report where it can be discussed with the diagrams that make it comprehensible.

Appendix K — Parameter Derivations and Design Rationale

Every tuning parameter in the project has a value. This appendix records *how* each value was arrived at. The point is neither to defend the numbers nor to fossilise them, but to make explicit the observation or reasoning behind each so that they can be re-derived rather than merely re-guessed when circumstances change.

K.1 Perception thresholds

Parameter	Value	Derivation
HSV lower bound	[100, 0, 50]	Below the minimum measured H, S, V over 20 sampled road-pixel patches on a real nadir frame, with a small margin. See K.1.1.
HSV upper bound	[170, 50, 180]	Above the maximum measured values on the same patches, with a small margin. See K.1.1.
Ignore top of frame	20%	Empirical: at typical operating altitude and camera geometry, the top of the frame is dominated by horizon artefacts and distant, low-resolution road pixels that segment poorly. 20% removes the noisy band without sacrificing lookahead.
Road-percentage log cadence	every 30 frames	Diagnostic ergonomics — one log line per ~2 s at 15 fps is enough to notice a trend without flooding the ROS log stream.

K.1.1 HSV thresholds — the measurement

Twenty pixels sampled across the visible road surface in a real nadir frame from the aircraft's camera returned the following ranges:

Channel	Min	Max
H	136	140
S	21	24
V	109	135

The thresholds were then set with modest asymmetric margins:

- Below on [H_lo, S_lo, V_lo]: [100, 0, 50]. The low H margin is wide because H is cyclic and edge-lookalike values just below the measured range should still admit; the S lower bound is set to 0 to admit low-saturation road pixels shadowed by the drone body; V lower bound is 50 to reject genuinely black pixels.
- Above on [H_hi, S_hi, V_hi]: [170, 50, 180]. H upper bound wide for the same cyclic reason; S upper bound at 50 rejects saturated, colourful non-road (grass, painted markings, sky reflections); V at 180 rejects near-white surfaces (concrete, rooftops).

The lesson: measure on real data, then set the *box* with margins whose direction is chosen physically — not symmetrically.

K.2 Planner and continuous-replanning parameters

Parameter	Value	Derivation
A* grid resolution	~10 cm/pixel at nominal altitude	Fixed by the camera intrinsics $\times$ altitude $\rightarrow$ ground-sample distance projection (Ch. 14). Not a tuning knob.
Goal hysteresis	15 px	The distance a new candidate goal must sit from the current goal before the planner adopts it. Prevents replanning jitter when the goal wobbles pixel-by-pixel. 15 px $\approx$ 1.5 m at nominal GSD, which is roughly one-quarter of the road width — small enough to still respond to real curvature, large enough to reject noise.

Minimum goal distance	30 px	Prevents the goal from collapsing onto the drone's current cell. At nominal GSD, ~3 m of lookahead — enough to give A* a meaningful path to search.
Lookahead (Pure Pursuit)	3.0 m	Standard Pure Pursuit tuning heuristic: lookahead $\approx$ vehicle length $\times$ 3 for stable convergence without over-cutting corners. For a small multirotor treated as a point mass at the nominal 2 m/s forward speed, 3.0 m is one-and-a-half seconds of travel.
REACHED (waypoint retirement)	~1 m	Larger than typical position-estimate noise (a few cm) but smaller than the lookahead. Set on the observation that a smaller REACHED produces stalling on unattainable exact points; a larger REACHED cuts corners.
Replan cadence	5 Hz nominal	One replan per fresh mask cycle; each mask carries new information. At 30 fps segmentation, 5 Hz is a comfortable duty factor that leaves CPU for other nodes.

The relationship between goal hysteresis and minimum goal distance is worth calling out: **hysteresis governs when to change the goal, and minimum goal distance governs where the goal can be at all**. These are orthogonal knobs, and confusing them produces circular tuning where changing one appears to break the other.

K.3 Segmentation-training parameters

Parameter	Value	Derivation
Input resolution	384×384	Multiple of 32 (MobileNetV2's stride), close to the D455's 640×480 aspect but small enough to fit training batches within the workstation's GPU.
Number of classes	12	AeroScapes' 11 semantic classes + background. Training on the full label set is a richer signal than binary road/not-road.
Batch size	16	Largest that fits alongside activations at 384×384 on the training workstation. Not scientifically derived — memory-bound.
Optimiser	Adam, <code>lr = 1e-4</code>	Standard segmentation starting point. Chosen conservative because the pretrained encoder is sensitive to too-aggressive early updates.
Loss	$0.5 \times \text{Dice} + 0.5 \times \text{CE}$	See Appendix I.5. The 0.5/0.5 weighting is the neutral prior; a Dice-heavier weighting could be justified if road performance under-shot but was not needed.
Class weighting in CE	uniform	Class imbalance is handled by the Dice term. Additional CE class weighting would double-count the correction.
ROAD_CLASS	10	AeroScapes' index for road. Exposed as a named parameter rather than embedded, as a defence against the off-by-nine bug of D.23.
Gabor channels	8	Four orientations $\times$ two frequencies covers the range at which asphalt is spectrally distinct from dirt and rooftops in the validation set (§42.3). Fewer misses discriminative modes; more adds cost without discriminative gain.

K.4 Safety and PX4 parameters

Parameter	Value	Derivation
<code>COM_RCL_EXCEPT</code>	4	Offboard exempt from the RC-loss failsafe, because autonomy must run without a transmitter babysitting it. Ch. 11.
<code>NAV_DLL_ACT</code>	0	No action on data-link loss, because a momentary GCS-link glitch must not abort a test. Ch. 11.
<code>COM_ARM_MAG_ANG</code>	-1	Bench expedient. Bypasses the 128° compass inconsistency (D.14) so bench work can proceed. Must be restored before outdoor flight.
<code>COM_ARM_WO_GPS</code>	warning only	Already set. Permits arming without a GPS fix — required for indoor bench work. Ch. 8.4.
Flight Mode 1 → Stabilized	(fix identified)	The bench test found Stabilized never assigned to any reachable switch position; Position was the only indoor-usable mode the switch could ask for, and Position needs GPS (D.15). Ch. 38.3.
<code>_wait_for_pilot()</code> (companion)	enabled	Autonomy blocks until pilot arms + Offboard from RC. Companion-side; not a PX4 parameter but the software equivalent. Ch. 51.

K.5 v7 chain, planner, and controller parameters

Parameter	Value	Derivation
-----------	-------	------------

<code>N_CHAIN_POINTS</code>	12	Enough waypoints to give the aircraft a stable, continuous target curve without over-fitting to short skeleton wiggles. Empirically stable on straight and gently curved roads; will be revisited on real D455 imagery.
<code>horizon_frac</code>	0.10	The top 10% of the frame is masked out before skeletonisation. The horizon band has the noisiest mask pixels (sub-pixel road, aliasing) and its skeleton is dominated by short, wobbly branches that confuse the longest-path pruner. Dropping upstream is cheaper than filtering downstream.
<code>REACH_PX</code>	12	A waypoint is considered reached at 12 pixels from the aircraft. At the nominal ground-sample distance this is ~ 1.2 m — larger than the position-estimate noise floor, smaller than the Pure Pursuit lookahead.
<code>MIN_GOAL_DIST_PX</code>	30	Ensures the current waypoint is at least ~ 3 m ahead of the aircraft. Prevents A* from being asked to plan to a point directly beneath the drone.
<code>REPLAN_MOVE_PX</code> , <code>REPLAN_GOAL_PX</code>	8, 8	A* is re-run only when start or goal has moved more than 8 pixels since the last plan. Below this threshold the previous path is retained. Suppresses per-tick thrashing on sub-pixel goal drift.
<code>LOOKAHEAD</code> (Pure Pursuit)	3.0 m	Standard Pure Pursuit heuristic: at 2 m/s cruise, 3.0 m is ~ 1.5 seconds of travel — enough to smooth out short-range noise, close enough to respond to real curvature.
<code>VMAX</code>	1.5 m/s	Modest cruise for early flight testing. Fast enough to make tracking behaviour visible; slow enough that a controller error is recoverable. Will be increased once outdoor GPS-based flights are stable.
<code>REACHED</code> (controller)	0.5 m	Waypoints are retired from the planner's path list at 0.5 m from the aircraft. Larger than the position-estimate noise floor but smaller than the Pure Pursuit lookahead — the two constants govern different behaviours (goal-picking vs waypoint retirement) and are deliberately not tied.
Setpoint rate	10 Hz	The control node's <code>create_timer(0.1, self.loop)</code> cadence. Above PX4's Offboard heartbeat requirement (≥ 2 Hz to keep the mode active) with sufficient margin, and matches the natural cadence of an ~ 15 fps perception loop with headroom for jitter.

K.6 Mechanical parameters

Parameter	Value	Derivation
Mount-plate outer dimension	160 × 160 mm	Sized against the airframe motor spread and the combined footprints of Jetson + Pixhawk + connectors.
Corner chamfer	30 mm	Clears the arms of the frame; also removes mass where the plate has no structural role.
Retaining wall height	7 mm	Empirical: enough to stop components sliding off laterally, and to turn a flat plate into a shallow tray (order-of-magnitude stiffer for the same mass); tall enough to shield connectors but not so tall as to interfere with them.
Jetson hole grid	dx = 90 mm, dy = 69 mm	The Orin Nano developer kit's mounting-hole spacing. Not adjustable — measured from the board.
Corner screw holes	M2.5 clearance	Standard fastener size for boards of this class; readily available and rated for the loads a small multirotor imposes.
Aperture (side hole)	3 × 3 cm	<i>Conflicts with the Jetson pocket</i> (Ch. 37.3). Resolution is a variable change, not a redesign, and is the one open item in the current CAD file.

K.7 The general rule

Every value in this appendix falls into one of three categories, and it is worth being explicit about which:

- **Physically fixed.** Determined by geometry, sensor specifications, or the target device. Not a tuning knob (grid resolution, hole grid, `fx/fy`).
- **Measured.** Set from data (HSV thresholds, D455 intrinsics).
- **Chosen with rationale.** Set from a physical argument, validated empirically (goal hysteresis, `K`, cross-track gain, lookahead).

Nothing here was set by trial-and-error alone. Where a value cannot be justified by measurement or physical argument, it should be flagged as "empirical, awaiting a principled derivation" — and Ch. 22.10 already does this for the perception-to-centreline integration that will require re-tuning.

Appendix L — Pre-flight Checklist and Test Procedures

The exact sequence of checks that must be completed before the aircraft is armed with propellers attached, and the ordered set of test flights that build up from motor spin-up to autonomous road-following. Nothing here is aspirational — every step is either done (marked ✓ in the "Prototype 1-α" column) or is a specific action still pending.

The purpose of writing this down is to eliminate any dependency on memory. On the day the aircraft first takes off, the operator should not be trying to remember whether the compass was calibrated; the checklist should have said so.

L.1 Pre-first-power checks (with battery *disconnected*)

#	Check	Prototype 1-α	Reference
L.1.1	Airframe is set to quad-X in PX4 (<code>SYS_AUTOSTART = 4001</code>)	✓	Ch. 28.3
L.1.2	PX4 firmware version is 1.17.0 (not ArduPilot; not an older PX4)	✓	Ch. 28
L.1.3	All four accelerometer axes calibrated; wizard completed for all six orientations	✓	Ch. 29.1
L.1.4	Gyroscope calibration completed with the board static	✓	Ch. 29.2
L.1.5	Compass calibration completed for both internal and external magnetometers	✓	Ch. 29.3
L.1.6	RC calibration completed; sticks map to roll/pitch/yaw/throttle	✓	Ch. 29.4
L.1.7	Kill switch assigned to a distinct RC channel; tested (see L.2.5)	✓ (assigned; tested in bench)	Ch. 30
L.1.8	Flight Mode 1 = Stabilized (not Position — Position needs GPS)	(Remaining)	D.15, Ch. 38.3
L.1.9	Flight Mode 4 (or other slot) = Altitude, one slot = Return	✓	Ch. 30
L.1.10	Compass consistency check re-enabled (<code>COM_ARM_MAG_ANG</code> restored)	(Remaining — required before outdoor)	D.14, Ch. 38.1
L.1.11	Kill switch physically reachable by the pilot at all times during test	(procedural — enforced by operator)	Ch. 52
L.1.12	Battery inspected: no puffing, correct voltage per cell (3.7–4.2 V)	(procedural — pre-flight, every time)	—
L.1.13	Airframe mechanically inspected: props tight (if fitted), arms tight, no visible cracks	(procedural — pre-flight, every time)	—
L.1.14	Wiring inspection: TELEM2 wiring correct (TX↔RX crossed, VCC unconnected, GND shared)	✓ (bench-verified)	Ch. 35

L.2 First powered bench test (props off)

The point of this stage is to confirm that everything electrical works before any propeller is fitted. Motors should spin *without* propellers.

#	Test	Prototype 1-α	Reference
L.2.1	Battery connected; Pixhawk boots; ~5 V from the power module to the flight controller	✓	Ch. 34
L.2.2	Jetson boots on its own 19 V UBEC rail; SSH is reachable	✓	Ch. 34
L.2.3	QGroundControl connects to the Pixhawk over USB; no red prearm errors that are not documented workarounds	✓	Ch. 38
L.2.4	Each motor spins individually when driven by the QGC Actuators panel slider	✓	Ch. 38.4
L.2.5	Kill switch, thrown while a motor is spinning, cuts that motor within one control cycle	(bench-verified)	Ch. 52.3
L.2.6	Actuator mapping: motor 1 = front-right (or airframe-specified), motor 2 = rear-left, ...	(Remaining)	Ch. 31, 36.3
L.2.7	Spin direction on each motor matches the quad-X CW/CCW pattern	(Remaining)	Ch. 31, 36.3
L.2.8	Any reversed motor is fixed via DShot reverse-direction command (not by rewiring)	(procedure)	Ch. 36.2

		defined)	
L.2.9	MAVSDK 2.8.4 on the Jetson connects to the Pixhawk over <code>/dev/ttyACM0:57600</code>	✓	Ch. 45.2
L.2.10	MAVSDK on the Jetson connects to the Pixhawk over <code>/dev/ttyTHS1</code> (flight link)	(Remaining)	Ch. 35, H.3

L.3 Pipeline dry-run (no propellers, no takeoff)

Verify that every ROS 2 node comes up and communicates without the aircraft being in a position to move.

#	Check	Prototype 1-α	Reference
L.3.1	<code>colcon build --symlink-install</code> completes without errors	✓	J.6
L.3.2	<code>ros2 topic list</code> includes <code>/camera/color/image_raw</code> , <code>/road_mask</code> , <code>/planner/waypoints</code> , <code>/mavsdk/pose</code>	✓ (on <code>cobot</code> ; on Jetson: camera live, others need pipeline running)	Ch. 6
L.3.3	<code>ros2 topic hz /camera/color/image_raw</code> reports non-zero rate	✓	H.6
L.3.4	<code>ros2 topic hz /road_mask</code> reports non-zero rate	✓ (sim); (Remaining on Jetson)	H.4
L.3.5	Segmentation road-percentage log line appears every ~30 frames	✓	Ch. 15, K.1
L.3.6	Pilot gate log line: "waiting for pilot: arm the vehicle, then enable Offboard via RC"	✓	I.1
L.3.7	Pipeline does <i>not</i> proceed past the gate until the pilot arms and enables Offboard	✓	I.1, Ch. 51

L.4 Outdoor prearm verification (no motion, no propellers)

Move to a clear outdoor space with sky view; verify that PX4 clears every prearm before propellers go on.

#	Check	Prototype 1-α	Reference
L.4.1	GPS module has clear sky view; wait 60–120 s for lock	(Remaining)	Ch. 32, D.13
L.4.2	QGC reports EKF2 converged; 3D position estimate is stable	(Remaining)	Ch. 8, 32
L.4.3	"Fence requires position" prearm is cleared	(Remaining)	D.13
L.4.4	Compass consistency error is <i>absent</i> (i.e., calibration was successful in situ)	(Remaining — see L.1.10)	D.14
L.4.5	Battery voltage reported by the power module matches an external multimeter reading	(procedural — every flight)	Ch. 34
L.4.6	RC signal strength is nominal at the maximum test-flight distance planned	(procedural — every flight)	—
L.4.7	Failsafe behaviour: with vehicle disarmed and RC on, powering off the RC produces the expected reaction in QGC ("RC signal lost")	(procedural — verify per session)	Ch. 8.5

L.5 First propellers-on test (Stabilized, altitude only, tethered if possible)

Now propellers go on. **Kill switch in hand for every subsequent step.**

#	Test	Prototype 1-α	Reference
L.5.1	Propellers fitted; direction and thread verified per motor	(Remaining)	—
L.5.2	Arm in Stabilized with throttle at bottom; motors idle	(Remaining)	Ch. 30
L.5.3	Slow throttle-up produces symmetric spin-up on all four motors (no shudder, no wobble)	(Remaining)	—
L.5.4	Gentle lift-off (indoors or tethered outside) confirms roll/pitch/yaw response is correct sign	(Remaining)	Ch. 31
L.5.5	Kill switch, thrown mid-hover at very low altitude, cuts all four motors within one control cycle	(Remaining)	Ch. 52.3
L.5.6	Land; motors disarm; no error flags in QGC	(Remaining)	—

L.6 First positioned flight (Altitude → Position, outdoors)

With Stabilized proven, switch to Altitude and Position modes, using GPS.

--	--	--	--

#	Test	Prototype 1- α	Reference
L.6.1	Arm and take off in Altitude mode; sticks command velocity	(Remaining)	Ch. 8.3
L.6.2	Switch to Position mode in flight; hover holds position within GPS accuracy (~1–2 m)	(Remaining)	Ch. 8.3
L.6.3	Return-to-launch: flick the RTL switch, verify the aircraft returns to launch point and lands	(Remaining)	Ch. 8.5
L.6.4	Battery low-voltage warning triggers at the parameterised threshold; RTL fires automatically at the critical threshold	(Remaining)	Ch. 8.5

L.7 First Offboard test (companion in the loop)

Now the Jetson is allowed to command motion, but only after the pilot has explicitly granted it authority.

#	Test	Prototype 1- α	Reference
L.7.1	Take off in Altitude; hover	(Remaining)	—
L.7.2	Flick mode switch to Offboard; <code>_wait_for_pilot()</code> releases; companion streams a hover setpoint	(Remaining)	I.1, Ch. 51
L.7.3	Aircraft holds hover under companion command (no drift beyond the estimation noise floor)	(Remaining)	—
L.7.4	Companion sends a small velocity setpoint (0.5 m/s forward for 3 s); aircraft translates as commanded, then hovers	(Remaining)	Ch. 10
L.7.5	Flick out of Offboard mid-command; PX4 returns to the mode the switch selects; aircraft stops obeying the companion	(Remaining)	Ch. 51
L.7.6	Repeat L.7.5 with the kill switch instead: kill cuts motors regardless of what the companion is doing	(Remaining)	Ch. 52

L.8 First autonomous road-following flight

Only after every preceding step succeeds. Choose a straight, wide, unobstructed section of road, in daylight, with no bystanders in the operating volume.

#	Test	Prototype 1- α	Reference
L.8.1	Camera pipeline reports plausible road percentage (> ~20%) on the selected road section	(Remaining)	Ch. 15
L.8.2	Planner publishes a non-empty path with the goal on the road (visualise in rviz or QGC)	(Remaining)	Ch. 22
L.8.3	Take off in Altitude, climb to nominal altitude for the road width	(Remaining)	Ch. 26
L.8.4	Switch to Offboard; companion follows the road at low forward speed (<code>v_forward</code> = 0.5–1.0 m/s initially, not the nominal 2 m/s)	(Remaining)	K.5
L.8.5	Cross-track error time series is monotone-decaying (verify against the SITL virtual-line baseline)	(Remaining)	Ch. 22.9
L.8.6	Aircraft tracks the centreline within the tolerance defined by (road width – aircraft footprint – K margin)	(Remaining)	Ch. 22.6
L.8.7	Flick out of Offboard at any moment; pilot regains full control instantly	(Remaining)	Ch. 51
L.8.8	Land; disarm; download logs	(Remaining)	Ch. 26

L.9 Post-flight

#	Task	Reference
L.9.1	Download the PX4 flight log (<code>.ulg</code>) and archive by date/time	Ch. 8
L.9.2	Download the ROS 2 bag, if recording was enabled	Ch. 5
L.9.3	Note in the flight log: what was tested, what was observed, what surprised the operator	—
L.9.4	Any anomalous behaviour is entered in Appendix D as a candidate troubleshooting entry	App. D
L.9.5	Battery is put on storage-charge if not being used within 24 h	—

L.10 Standing rules

Three rules that apply to every flight test and are not negotiable per session:

1. **The kill switch is in the pilot's hand at all times.** Not on a table, not on a bench — in the hand, with the thumb able to reach it without looking.
2. **Propellers are fitted only after every step of L.1 and L.2 has passed for that session.** A calibration completed a week ago is not evidence that a bolt tightened yesterday is still tight.
3. **No flight test is conducted alone.** A second person is on-site, in view of the aircraft, whose sole job is to watch the aircraft and shout "kill" if anything unexpected happens. The pilot is looking at the transmitter and the aircraft; a second observer catches what the pilot misses.

The three rules exist because the failure mode they defend against — a lithium fire, a spinning propeller striking a person, an aircraft flying into a building — is not one that any subsequent design change can undo. Every safety feature described elsewhere in this report is *in addition to* these rules, not a substitute for any of them.

Part XVI — Post- α Improvements and First Outdoor Flight

Prototype 1- α closed with every subsystem verified in isolation and the aircraft ready for outdoor flight. The work in this part happened *after* that closure: the pipeline hardening that made the first autonomous flight possible on **16 July 2026**, the flight itself, and the post-flight analysis of the log that identified a single sign-inversion bug as the root cause of the observed oscillation.

59. The 20 Hz Setpoint Streamer Refactor

59.1 What the previous design got wrong

Up to v7, `control_node.loop()` was the sole sender of MAVLink setpoints. It ran at 10 Hz and called `asyncio.run_coroutine_threadsafe(drone.set_velocity(...))` at the end of each cycle. The design worked on the bench but had three latent holes visible only under load:

- **The waypoint-reached branch returned without sending.** When `dist < REACHED` and a waypoint was popped from the path, the function returned. That produced a silent ~100 ms hole in the stream every time a waypoint retired — brief, but visible as a stutter in the setpoint trace.
- **A crash in `loop()` stopped the stream entirely.** Any exception in perception-to-command translation broke the setpoint stream, and PX4's Offboard mode timeouts (0.5 s without a fresh setpoint) would fire.
- **The stream started only after handoff.** Between the pilot flipping into Offboard and the first waypoint arriving, `loop()` sent nothing. PX4 rejected the mode transition if the stream had not been primed in time — the failure mode that killed `log_65` at 46.51 s (see §63.2).

59.2 The refactor

The v8 control node separates *deciding what to command* from *sending the command*. A dedicated `_setpoint_streamer` asyncio task runs at **20 Hz** and is the sole sender of MAVLink setpoints after handoff. `loop()` writes to a shared `(cmd, timestamp)` slot via `_set_cmd(vx, vy, vz)`; the streamer reads it. If the command is older than `CMD_STALE_S = 0.5 s`, the streamer falls back to zeros — so a `loop()` crash produces a hover intent, not a lost stream.

pre-handoff	→ <code>_wait_for_pilot</code> primes zero setpoints (unchanged)
handoff detected	→ streamer starts at 20 Hz with zero velocity
valid path + pose	→ <code>loop()</code> writes Pure Pursuit cmd → streamer sends it at 20 Hz
no path / no pose	→ <code>loop()</code> writes (0,0,0) → hover intent
<code>loop()</code> crashes	→ cmd goes stale in 0.5 s → streamer sends zeros anyway
streamer crashes	→ logged; PX4 offboard-loss failsafe catches it
handoff revoked	→ streamer idles; <code>_wait_for_pilot</code> priming resumes

Two follow-on effects. The **waypoint-retirement hole** disappears because the streamer keeps sending the last commanded velocity through the 100 ms gap. And the **priming problem** — where PX4 rejected Offboard because setpoints started too late — is fixed by having the streamer run continuously from the moment of MAVSDK connection, so setpoints are always flowing well before the pilot flips the mode switch.

59.3 Why this matters for a real flight

The 20 Hz cadence, the staleness fallback, and the "streamer runs continuously" invariant together mean the failure modes that can kill a real Offboard flight are strictly narrower than the failure modes that can kill the software. Anything short of a hard process crash results in a well-defined, safe setpoint (last valid command → zeros → PX4 failsafe), not a stream that stops. That property is what makes the software auditable against PX4's Offboard requirements rather than merely "seeming to work on the bench."

60. `BENCH_MODE` and Runtime Instrumentation

60.1 The bench-mode gate

PX4 requires a valid position estimate to accept an Offboard velocity command; indoors, without GPS or a VIO source, that estimate never converges and Offboard cannot be tested. `BENCH_MODE = True` in `control_node.py` forces `takeoff_complete = True` and bypasses the 2 m altitude gate, allowing the full pipeline to run end-to-end on a props-off aircraft on the bench without ever getting an Offboard acceptance from PX4. The streamer deliberately idles in bench mode so that no MAVLink setpoints are sent, but every other stage — segmentation, planner, control-loop math, path publication, `_set_cmd` writes — runs as it would in flight.

The convention is fixed: **props always off during bench tests**; `BENCH_MODE = False` and `MIN_ALT = 2.0` must be restored before real flight. Chapter 52 (safety architecture) records these as the two safety locks that must be re-armed alongside the compass and RC-loss failsafes before any outdoor operation.

60.2 The handoff-state watcher

The pilot-gate design of Chapter 51 polls three telemetry streams — armed, altitude, flight mode — in a single loop. In v7 that polling used sequential `await`s, and any one stream stalling would starve the others. Under real telemetry pressure this occasionally produced a *deadlock*: `armed` would return, `altitude` would be waited on for several seconds, and by the time `flight_mode` was checked the pilot had already flipped in and out of Offboard, so the gate never released.

The v8 fix decouples the streams. Three continuously-running watcher tasks — `_watch_armed`, `_watch_altitude`, `_watch_flight_mode` — subscribe to their respective telemetry streams and cache the latest value into node state. `_wait_for_pilot` then reads the cached values, so no single slow stream can starve the others. The gate log line now prints `HANDOFF STATE armed=... alt=... mode=...` on every check, and this proved indispensable in the field — a pilot who thinks they are in Offboard and cannot see why the autonomy has not taken over has one line of ground truth to read.

60.3 PX4 message pass-through

`_px4_status_stream` subscribes to PX4's MAVLink status text and re-emits every message into the systemd journal as `PX4 SAYS [TYPE]: message`. The messages that had been trapped inside QGroundControl — "Rejecting Offboard, no setpoints", "EKF2 GPS check failed", "Land detected" — now surface where the operator is already looking. On the field this collapsed several diagnostic cycles from "reproduce on the workstation" to "read the journal".

61. Boot Auto-Start via systemd

The aircraft has no monitor and no keyboard once the airframe is closed up. A pilot standing in a field needs the pipeline to be running the moment the Jetson finishes booting.

61.1 The service and the flag file

`semantic-drone.service` under `/etc/systemd/system/` runs `headless_start.sh`, which sources ROS 2, launches the four-node graph (`realsense2_camera`, `segmentation_node`, `planner_node`, `control_node`), and stays alive until the Jetson shuts down. It runs unconditionally on boot — except when the file `~/DEV_MODE` exists on the target user's home directory, in which case the launcher exits immediately and the pipeline stays down.

The convention: **touch `~/DEV_MODE` at the desk** (bench work, code edits, testing); **rm `~/DEV_MODE` before mounting on the airframe** (field). The flag file is per-user, requires no `sudo` on toggle, and — because it is checked by the shell wrapper rather than by systemd itself — does not require a service reload.

61.2 Why not HDMI detection

The original plan was to key auto-start off HDMI connector state (monitor present = development, monitor absent = flight). This does not work on the Orin Nano developer kit: the `nvidia_modeset` driver does not expose connector status under `/sys/class/drm`, and the substitute approaches (querying `nvidia-smi`, parsing `dmesg`) are fragile. A flag file is one line of shell and cannot be misread.

61.3 A subtle bash bug worth recording

The shell wrapper had `set -u` at the top (fail on unset variables), which is normally correct. But `source /opt/ros/humble/setup.bash` internally reads variables that ROS's own scripts may not have set, and the whole wrapper crashed at that line with `AMENT_PREFIX_PATH: unbound variable`. The fix was to bracket the sourcing block with `set +u ... source setup.bash ... set -u`, which relaxes the check for exactly the section that violates it. The service is now verified live with `systemctl status semantic-drone` reporting `active (running)` and four ROS nodes in the process tree.

62. TensorRT FP16 Perception Acceleration

62.1 The number that changed

The v7 perception latency was **309 ms end-to-end**, **~3.2 Hz**. The v8 perception latency, after TensorRT FP16 conversion of the MobileNetV2 + Gabor U-Net, is **31.2 ms end-to-end**, **~32 FPS** — a 10× improvement, and the single largest performance step of the whole project.

Stage	v7 (TF eager)	v8 (TensorRT FP16)
Segmentation inference	≈ 266 ms	6.86 ms
Gabor preprocessing (CPU)	≈ 8 ms	19.6 ms (<i>now dominant</i>)
Cost-map + skeleton + goal	≈ 35 ms	≈ 4 ms
End-to-end	~309 ms (3.2 Hz)	31.2 ms (32 FPS)

The row that flipped is worth reading twice. Inference dropped by nearly two orders of magnitude, which moved the **bottleneck** from the network to the CPU-side Gabor preprocessing — the eight-kernel convolution bank that gives the segmentation head its texture channel (Chapter 42). At 32 FPS the pipeline now runs faster than the RealSense's 30 fps colour stream, so the perception loop is genuinely camera-limited rather than compute-limited.

62.2 What TensorRT changes and does not change

TensorRT converts the trained Keras `.weights.h5` model to an optimised, fused, quantised inference engine specific to the Jetson Orin Nano's GPU architecture. Nothing about the network's *output* changes: the same 12-class softmax head reads the same road class at index 10, and validation IoU on AeroScapes is within ~0.5% of the unquantised model. What TensorRT changes is *how* the forward pass is computed — fused layers, half-precision arithmetic, kernel selection specialised to the tensor shapes in use.

The FP16 (as opposed to INT8) choice was deliberate: FP16 gives most of the speed with almost no calibration effort and no visible accuracy loss. INT8 would require a representative calibration dataset and per-layer post-training quantisation and is a natural next step if a further ~2× is needed. For 1-α the FP16 result is more than sufficient — the pipeline is now bounded by Gabor preprocessing, not by inference, and further inference speedups have no downstream effect until the preprocessing is also accelerated.

62.3 The consequence for flight

At 3.2 Hz the perception loop was too slow to close a real road-following control loop — 300 ms of pipeline latency between seeing a pixel and commanding a velocity is a quarter of a second, which at 1.5 m/s is 37 cm of position uncertainty. At 32 FPS the perception loop is fast enough that the perception latency is comparable to the pilot's own reaction time; it stops being the limiting factor. This is what made the first autonomous flight of §63 possible, and the flight's oscillation (§64) is therefore a control-law and mounting problem, not a perception-latency problem.

63. First Outdoor Autonomous Flight — 16 July 2026

63.1 What was attempted

On 16 July 2026 the aircraft was flown outdoors under Offboard control with the full v8 pipeline running on the Jetson. The intent was a straight-line road-following demonstration over an open section of road on the IIT Gandhinagar campus. Two flight logs were recorded: `log_65.ulg` (a first-attempt log where Offboard was rejected) and `gt.ulg` (the successful engagement).

63.2 `log_65.ulg` — the rejection

The first Offboard attempt was refused by PX4 at $t = 46.51$ s. The log's error text was "Rejecting Offboard, no setpoints". Post-flight forensic analysis showed the setpoint stream had started at $t = 47.28$ s — **0.77 s after** the mode-switch request. PX4's Offboard acceptance requires the stream to be already flowing when the mode change arrives, not to begin after it.

GPS was ruled out as a cause: the log showed `eph = 0.22 m` throughout, well inside PX4's threshold. The root cause was late priming — which is precisely the failure mode the 20 Hz streamer refactor (§59) was designed to prevent. `log_65` is therefore the pre-refactor artefact; every Offboard engagement after the refactor was accepted.

63.3 `gt.ulg` — the flight

The successful flight recorded two Offboard windows with the pilot arming, taking off in Stabilized, climbing to ~ 4 m, and then handing off:

Window	Time	Duration	Command profile	Entry	Exit
1	100.61 s – 110.93 s	10.3 s	1.500 m/s heading flipping $\sim 180^\circ$ every 1–2 s	Pilot	Pilot
2	129.60 s – 145.20 s	15.6 s	0.000 m/s on all 80 samples	Pilot	Pilot

26 seconds total of Offboard control with the companion computer in the loop. Every exit was pilot-commanded; no failsafes fired. GPS was solid throughout (`eph 0.22 m`). The two-lock handoff mechanism of Chapter 51 worked as designed: the pilot armed, took off in Stabilized, engaged Offboard on the RC switch, `_wait_for_pilot()` cleared, PX4 accepted the streaming setpoints, and command authority was in the companion computer for the duration of both windows.

63.4 What the flight demonstrates and what it does not

Demonstrated. The full v8 pipeline runs end-to-end on a real aircraft. Segmentation, skeletonisation, waypoint chain, cross-track compute, and PX4 handoff all functioned in the air. The 20 Hz streamer held the Offboard stream stable for the whole 26 s. The pilot could enter and exit Offboard at will using the RC mode switch.

Not demonstrated. Successful road-following. Window 1 produced only 1.75 m of net displacement over 10.3 s of commanded flight at 1.5 m/s — 88% of the commanded distance is unaccounted for, because the commanded direction reversed continuously. Window 2 commanded exactly 0.000 m/s throughout, which under the streamer is the "no valid path" branch and represents keep-alive traffic rather than closed-loop control. The paper claim that this flight supports is:

"Autonomous Offboard control was successfully engaged and the companion computer's setpoints were accepted by the flight controller for a total of 26 s across two windows."

It does **not** support any claim of successful centreline tracking, centreline-error metric, or road-following distance.

64. Post-Flight Analysis: The 180° Camera Roll

64.1 The initial hypothesis and why it was wrong

The most visible symptom of Window 1 was that the drone appeared to move sideways relative to the road while its yaw was pointing along the road. The first hypothesis — the one that shaped the immediate post-flight debug — was that the planner's pixel-to-NED transform had a **90° frame misalignment**: a swap between row and column in the projection, effectively rotating the commanded velocity vector by a quarter turn.

That was wrong. Forensic replay of the log with the planner code showed no 90° rotation in the transform: `_find_auto_goal` and the pixel-to-metre conversion in `planner_node.py` were correct. But the commanded heading time series over Window 1 did not merely drift by 90° — it **flipped by ~180° every 1–2 s**, spanning -151° to $+138^\circ$. That is not a fixed frame rotation; that is a control loop with the wrong sign.

64.2 The actual root cause

The camera was mounted **rolled 180° about its optical axis**. The mount is nadir (pointing straight down, correctly), but the sensor's up-axis and right-axis relative to the airframe were both flipped. This inverts *both* image axes at the pixel level, which is subtly different from inverting one — a 90° rotation would swap axes; a 180° roll negates both.

Downstream, the pixel-to-body offset that drives the road-centring loop gets negated. The controller, on seeing the goal 20 pixels *right* of centre in the image, was steering *left* — because those pixels were, in world terms, actually on the *left* of the aircraft. Each correction pushed the aircraft *away* from the road it was trying to centre on, then the next frame arrived with the goal now on the opposite side, and the correction reversed. That is a **positive-feedback limit cycle**, and the ~180° heading flip every 1–2 s in the log is the signature of exactly such a cycle at the perception cadence.

A 90° rotation would have produced a smooth sideways translation. A 180° roll produced an oscillation.

64.3 The fix

In `camera_node.py`, apply `cv2.rotate(frame, cv2.ROTATE_180)` to both the colour and depth streams (identically, so RGB-depth registration is preserved). **Recompute the principal-point intrinsics for the rotated frame**: if the raw D455 measurements at 640×480 were `cx = 322.326`, `cy = 239.442`, the 180°-rotated intrinsics are `cx' = 639 - 322.326 = 316.674` and `cy' = 479 - 239.442 = 239.558`. Focal lengths are unchanged.

This is a five-line patch and it inverts the sign of the entire road-centring loop, which is exactly what the limit-cycle diagnosis called for.

64.4 The secondary contributors

Two other factors made Window 1 worse than the primary bug alone.

- **LOOKAHEAD = 3.0 m exceeded the length of the local waypoint chain**. When the chain is shorter than the lookahead, Pure Pursuit degenerates to full-speed pursuit of the chain's endpoint, which under the 180° roll means chasing an endpoint that is itself sign-inverted. The corrected `LOOKAHEAD = 1.2 m` keeps the lookahead inside the chain.
- **No deceleration law** — the commanded speed was a constant `VMAX = 1.500 m/s` throughout the window. When the goal is near, this is too fast; when it is far, the network noise dominates. The corrected law is `speed = min(VMAX, K_SPEED * fdist)` — proportional to the forward distance to the goal — plus a hard stop at the final waypoint.

64.5 Status

The internship ended immediately after the flight, so **no post-fix flight was possible**. The camera rotation fix, the intrinsics recomputation, the Pure Pursuit tuning, and the deceleration law are all patched in the codebase; the aircraft can be rebuilt and re-flown with these corrections in place, but that is future work outside Prototype 1- α 's timeline. The document records the flight, the diagnosis, and the fixes; it does not claim a corrected flight because none has occurred.

The paper section that this material supports is honest and useful in the form: "**Field Trials: First Autonomous Flight and Failure Analysis**" — reporting the two Offboard windows, the 26 s of accepted setpoints, the observed oscillation, the log-based root-cause identification, and the corrective patch. That is a well-diagnosed first-flight result, and it is a stronger paper contribution than a lucky-first-try would have been — because the failure analysis demonstrates command of the system's failure

modes, not just its success modes.

(Continued — Part XVI)

65. v8 Algorithmic Optimization Pass

The v8 change set is a pure performance pass over the planning-side code, orthogonal to the runtime and flight-safety work of §§59–64. Six files were rewritten; every rewrite was verified byte- or cost-equivalent to its v7 counterpart on the same inputs. `control_node.py` and `px4.py` were **deliberately not touched** — those files are flight-tested (§59, §51), and microsecond gains do not justify re-verifying them on the bench. The intent of v8 is that nothing an operator or a reviewer sees should change; only the CPU cost of a planner tick should fall.

65.1 The six changes and their measured speedups

File / function	v7 cost	v8 cost	Fix
<code>nodes/planner_node.py</code> <code>_nearest_free_goal</code>	15.17 ms	0.15 ms (100×)	$O(R^3)$ Python-loop spiral over $(2r + 1)^2$ squares → one <code>np.argmax</code> on the free-cell mask plus one vectorised <code>argmin</code> . Also returns the <i>true</i> Euclidean nearest cell rather than a scan-order artefact of the spiral.
<code>core/depth_costmap.py</code> <code>filter_road_components</code>	15.15 ms	6.59 ms	Per-component <code>out[labels == i] = 1</code> loop ($O(K \cdot N)$ full-image scans, $K \approx 30$ in the reference frame) → single <code>np.isin(labels, keep_labels)</code> ($O(N)$ regardless of K).
<code>core/astar.py</code>	5.12 ms	4.05 ms	Added an explicit closed set so nodes are not re-expanded from stale heap entries (Manhattan h is consistent on a 4-connected grid, so the first pop of a node is final). <code>g_score</code> and <code>came_from</code> dicts → numpy <code>int32</code> arrays indexed by cell index. Grid cast to <code>int32</code> once instead of <code>int(grid[x][y])</code> per neighbour.
<code>core/skeleton_goal.py</code> <code>main_branch /</code> <code>_bfs_farthest</code>	2.2 ms	1.7 ms	BFS over a set of skeleton coordinates with precomputed 8-neighbour offsets; v7 indexed the full image via a nested generator for every neighbour visit. Zhang–Suen numpy fallback kept verbatim (only used without <code>opencv-contrib</code>).
<code>core/occupancy.py</code> <code>smooth_path</code>	Python loops	Vectorised	3-point moving average now three numpy slice-adds plus a single vectorised free-cell test over the whole path; v7 ran two list comprehensions per point.
<code>core/projection.py</code> <code>pixel_to_world</code>	Python loop	Vectorised	Single array operation over the decimated path; v7 looped per point. Output identical to 1×10^{-9} .
<code>_chain_goal</code> (planner)	Recursive	Bounded 2-pass loop	Same behaviour, no call-stack growth on blocked-waypoint chains.
Mask-empty check	<code>int(np.sum(mask)) == 0</code>	not <code>mask.any()</code>	Short-circuits on the first road pixel instead of summing 307 k cells.

65.2 Net effect on the 5 Hz planner tick

Under the worst-case observed in the flight logs (drone start cell on an obstacle, requiring the nearest-free recovery to fire):

Stage	v7	v8
Nearest-free recovery on obstacle start	~15 ms	~0.2 ms
Component filter	~15 ms	~7 ms
A* + smooth + project	~6 ms	~4.5 ms
Planner-tick total (worst case)	~36 ms	~12 ms

Planner-side CPU is roughly halved. On the Jetson this frees the two saturated CPU cores that share the device with the 19.6 ms

Gabor preprocessing (§62.1), which is now the pipeline's single bottleneck stage — meaning any further optimisation should target the Gabor bank, not the planner.

65.3 Equivalence checks performed

Byte- or cost-identity was verified for every rewrite before shipping v8:

- `astar` — cost `1027 == 1027` and length `248 == 248` on a 120×160 grid; both v7 and v8 return `None` on a severed road with identical topology.
- `filter_road_components` — output mask identical byte-for-byte on 480×640 with 30 blobs.
- `build_semantic_costmap` — output byte-identical (the intermediate 50-fill in v7 was overwritten by the two writes that followed, so the shorter `np.where` produces the same array).
- `smooth_path`, `pixel_to_world` — outputs identical to 1×10^{-9} .
- `main_branch` (skeleton) — endpoint match and length `179 == 179` on a synthetic curved road strip; waypoint chains produced by `build_waypoint_chain` identical to v7.
- `_nearest_free_goal` — v8 returns the true Euclidean-nearest free cell, which in the general case is *not* the same cell v7's spiral returned (v7's return was a scan-order artefact). This is the one case where v8 output legitimately differs from v7, and the difference is a bug fix.

65.4 What v8 deliberately does not do

Three items are intentionally out of scope for v8 and belong to separate corrective passes (§64):

- **Camera rotation.** `cv2.rotate(frame, cv2.ROTATE_180)` in `camera_node.py` fixes the 180° roll and is a functional change to the perception pipeline, not a speedup.
- **Pure Pursuit and deceleration.** The `LOOKAHEAD = 3.0 → 1.2` m change, the `speed = min(VMAX, K_SPEED · fdist)` proportional deceleration, and the stop condition at the final waypoint are corrections to the control law and live in `control_node.py`.
- **Setpoint streamer refactor.** The 20 Hz streamer and `CMD_STALE_S = 0.5` s staleness fallback (§59) are the flight-safety change; also in `control_node.py`.

Keeping v8's algorithmic pass separate from these corrective changes preserves the property that v8 is *purely* a speedup and can be verified as such — nothing about the aircraft's behaviour changes when v8 is applied on top of v7, only how much of the Jetson's CPU is left over for other work.
